

Container Cloud Native Security



Facebook interface for the page **somkiat.cc**. The top navigation bar includes the Facebook logo, the page name **somkiat.cc**, a search icon, and user profile icons for **Somkiat** and **Home**. The main navigation bar features **Page**, **Messages**, **Notifications** (with a red badge showing 3), **Insights**, **Publishing Tools**, **Settings**, and **Help**.

The page content area displays a large video of a man in a white Superman t-shirt with "SOMKIAT.CC" printed on it, posing against a white wall. A smaller thumbnail of the same video is shown in the left sidebar. The sidebar also includes the page name **somkiat.cc**, the handle **@somkiat.cc**, and a menu with **Home**, **Posts**, **Videos**, and **Photos**.

Below the video, there are interaction buttons: **Liked**, **Following**, **Share**, and a three-dot menu. A blue call-to-action button **+ Add a Button** is also present. A blue tooltip message reads: "Help people take action on this Page."



Section 1

1. Container design patterns
2. Image and Container security
3. Best practices to building images
4. Write Dockerfile
5. Workshop with docker



Section 2

1. Introduction to Kubernetes
2. Kubernetes security
3. Identity and RBAC (Role-base access control)
4. Network policy
5. Secret and credential management
6. Workshop with Kubernetes (miniKube)



Section 3

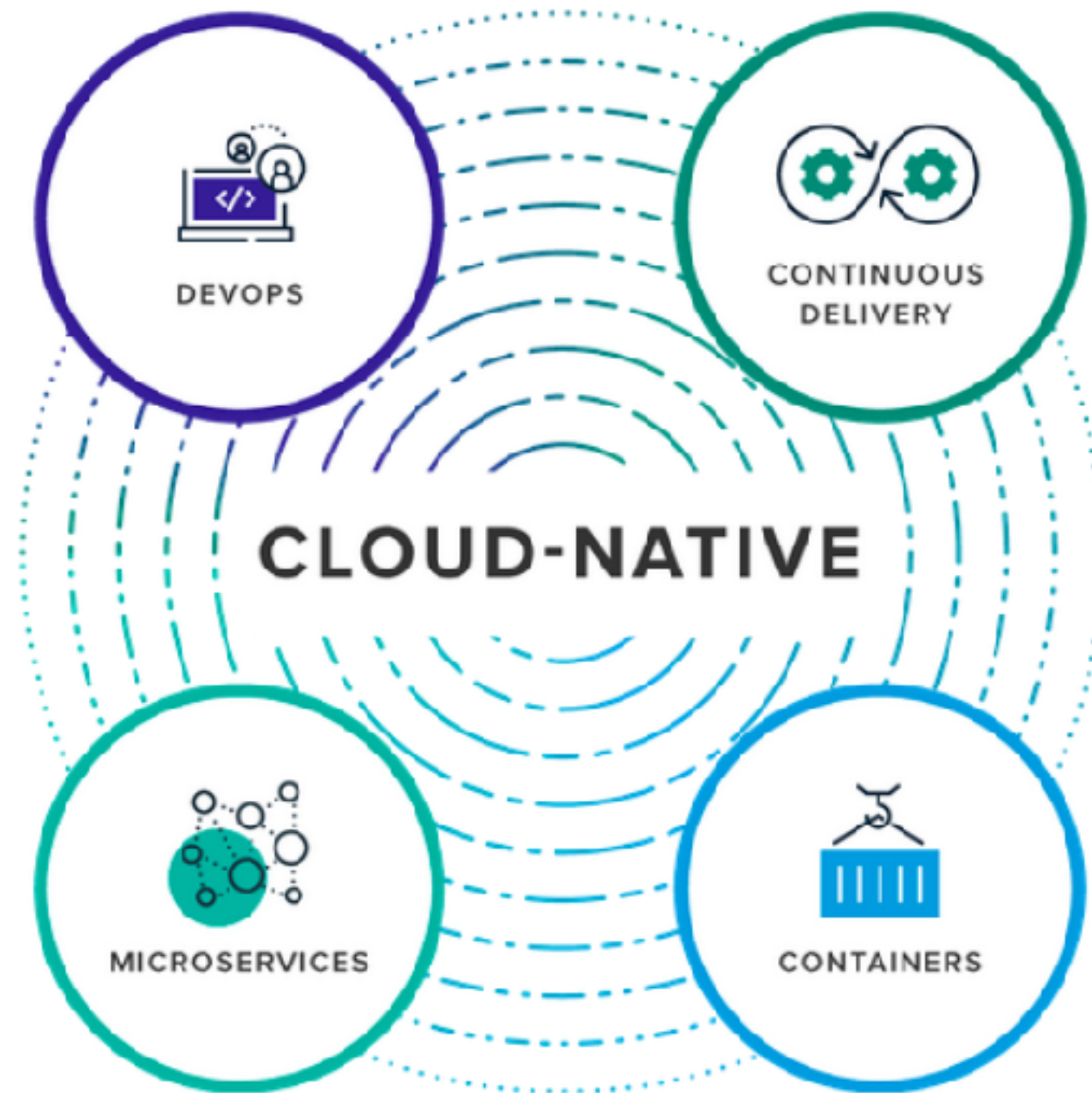
1. Supply chain security
2. Shift-left security (DevSecOps)
3. Working with SAST (Static analysis)
4. Working with SAST (Dynamic analysis)
5. Working with SBOM
6. Working with Dependency check and tracking
7. Workshop with CI/CD



Container Design Patterns



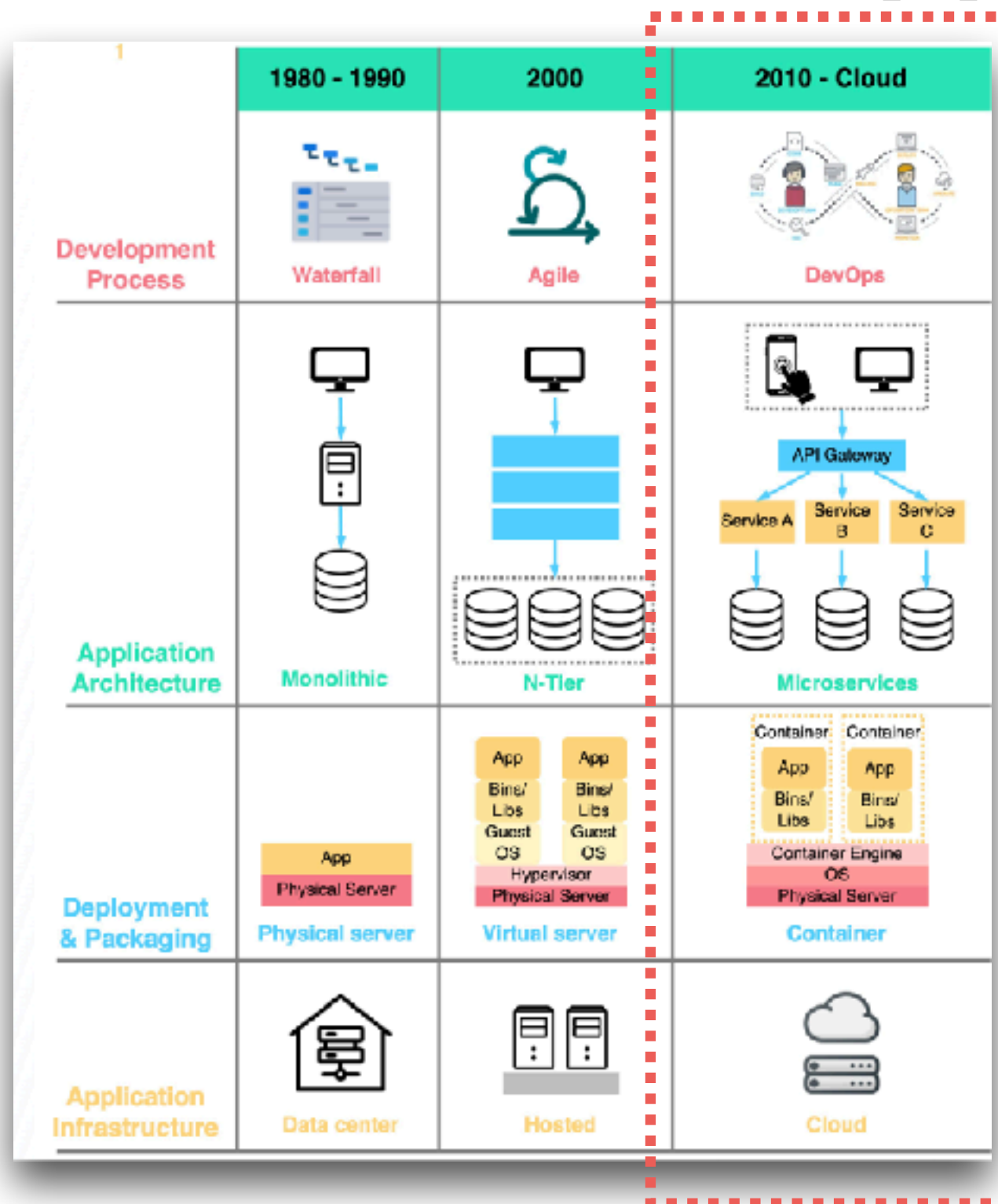
Cloud Native App



<https://kubernetes.io/blog/2018/03/principles-of-container-app-design/>



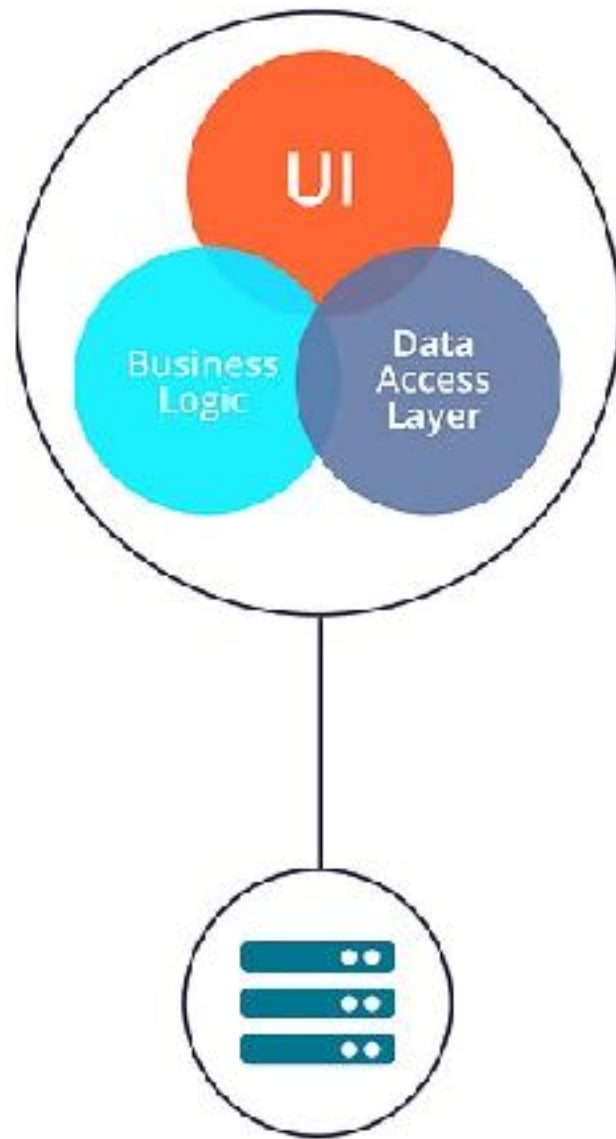
Cloud Native App



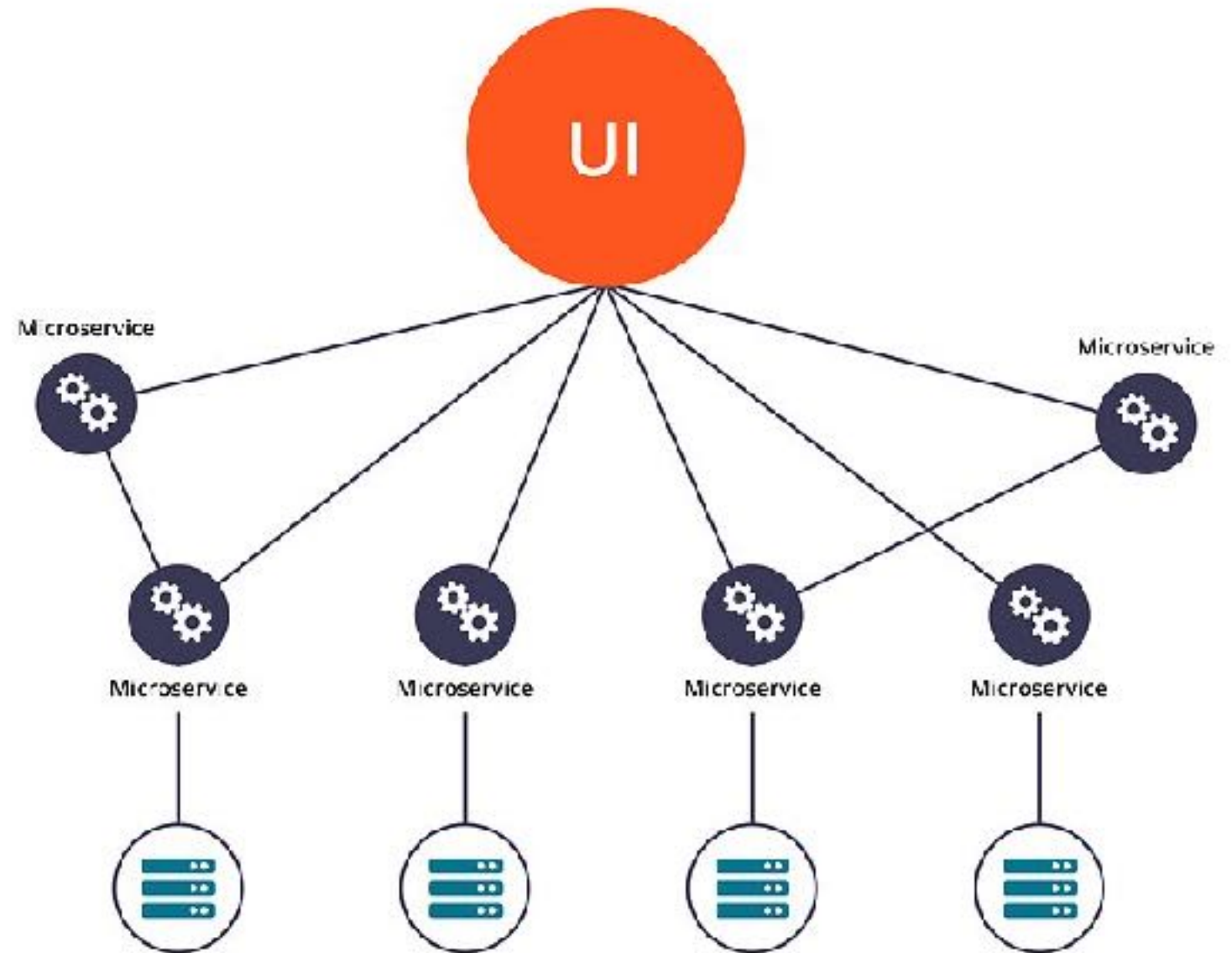
<https://bytebytego.com/guides/what-is-cloud-native/>



Microservices



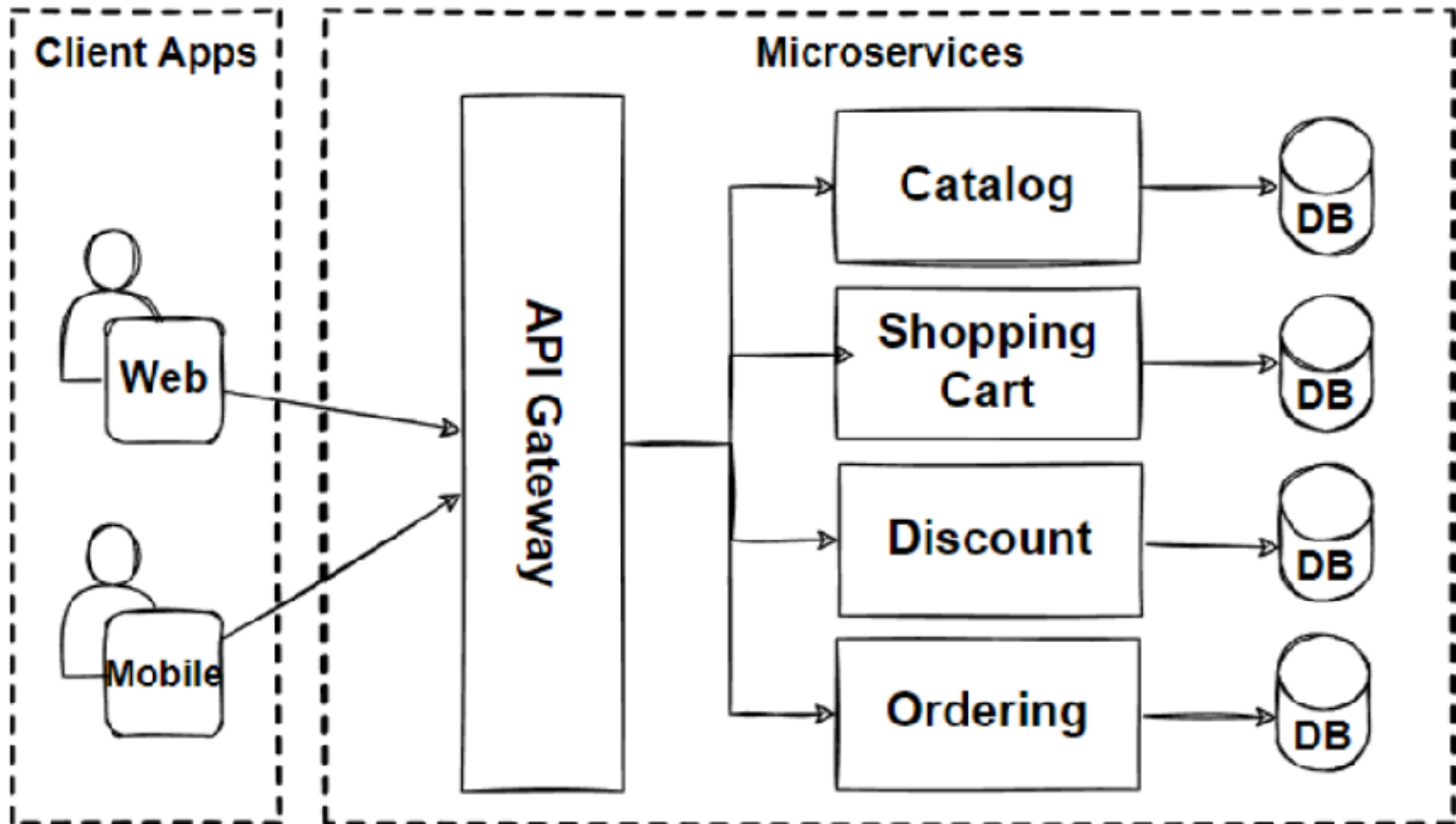
Monolithic Architecture



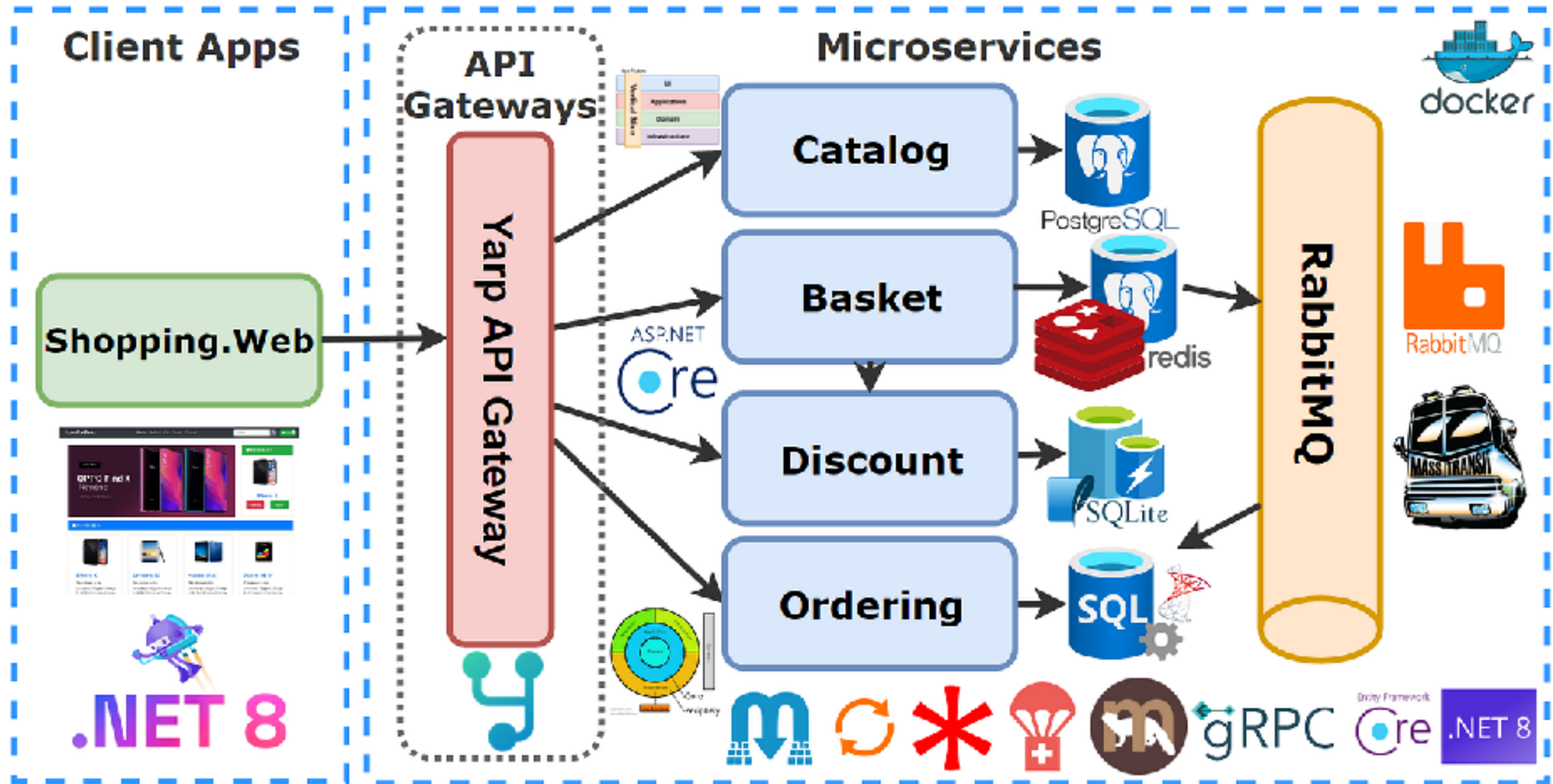
Microservice Architecture



Microservices

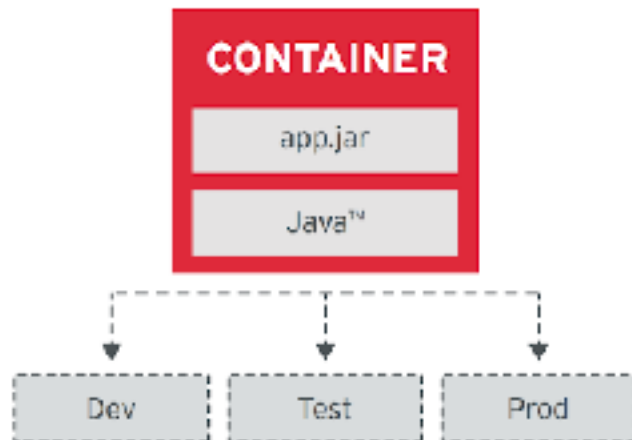


Polyglot Technologies !!



Container Design Principles

Image Immutability Principle



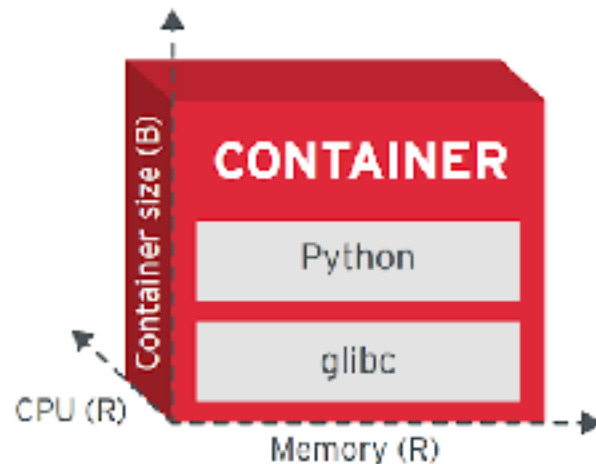
High Observability Principle



Process Disposability Principle



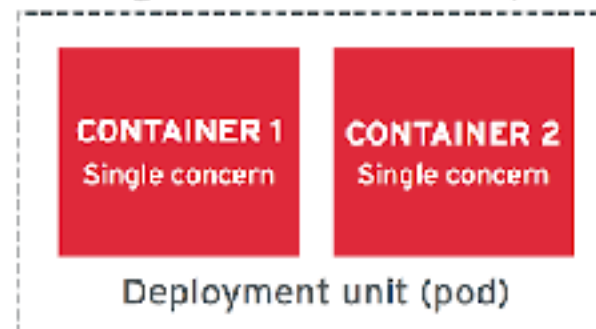
Runtime Confinement Principle



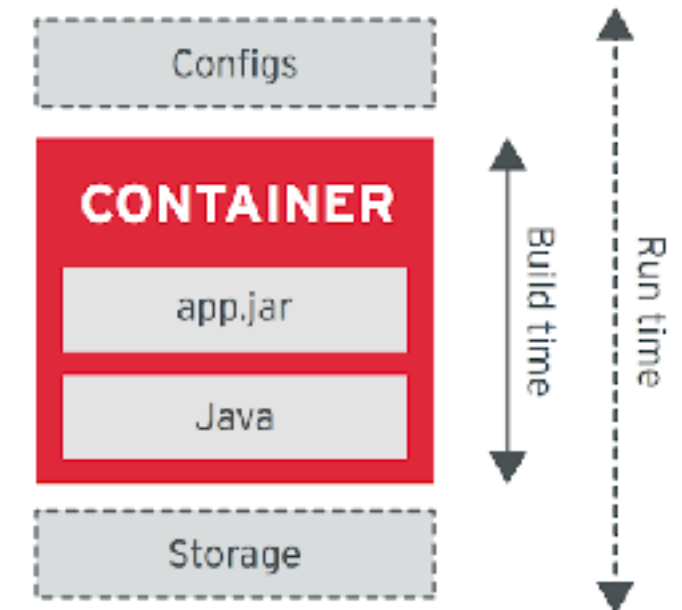
Lifecycle Conformance Principle



Single Concern Principle



Self-Containment Principle



<https://kubernetes.io/blog/2018/03/principles-of-container-app-design/>



12 Factor App



THE TWELVE-FACTOR APP

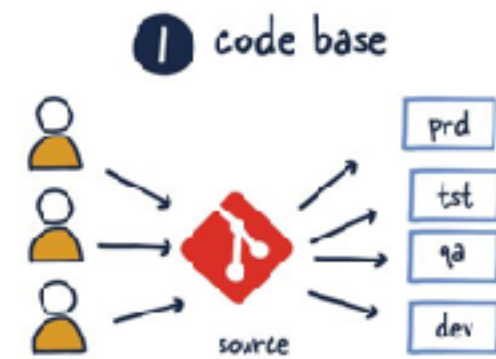
INTRODUCTION

In the modern era, software is commonly delivered as a service: called *web apps*, or *software-as-a-service*. The twelve-factor app is a methodology for building software-as-a-service apps that:

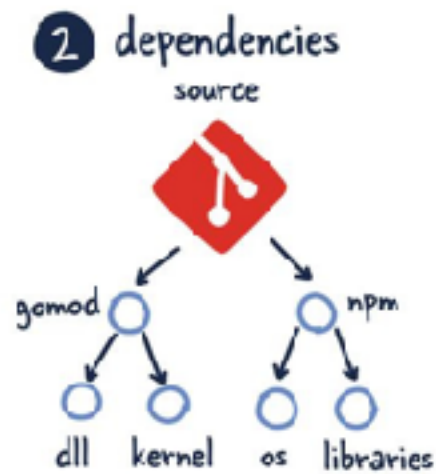
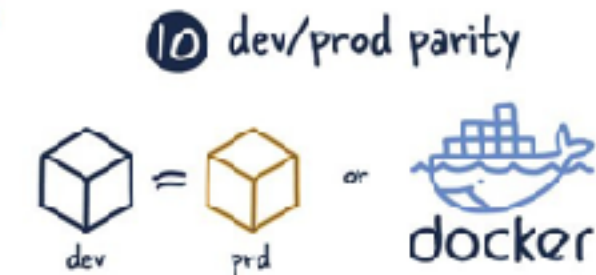
- Use **declarative** formats for setup automation, to minimize time and cost for new developers joining the project;
- Have a **clean contract** with the underlying operating system, offering **maximum portability** between execution environments;
- Are suitable for **deployment** on modern **cloud platforms**, obviating the need for servers and systems administration;
- **Minimize divergence** between development and production, enabling **continuous deployment** for maximum agility;
- And can **scale up** without significant changes to tooling, architecture, or development practices.

<https://12factor.net/>





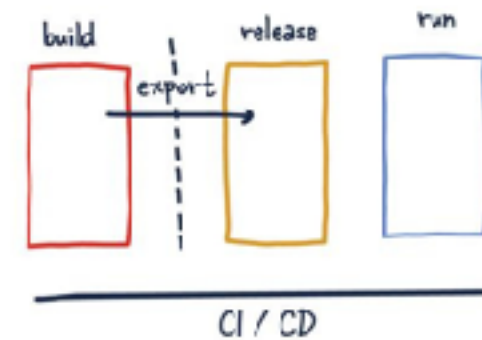
12 FACTOR APP REVISITED



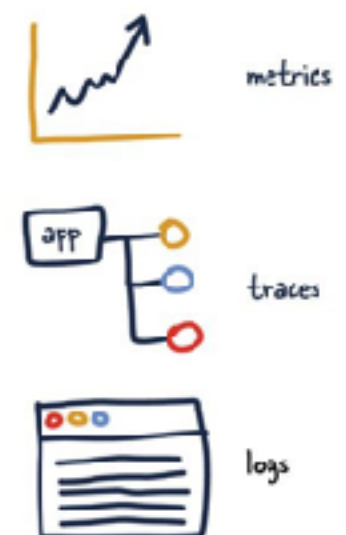
4 backing services



5 build, release, run



11 logs



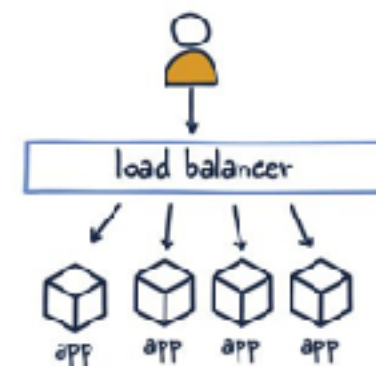
6 Processes



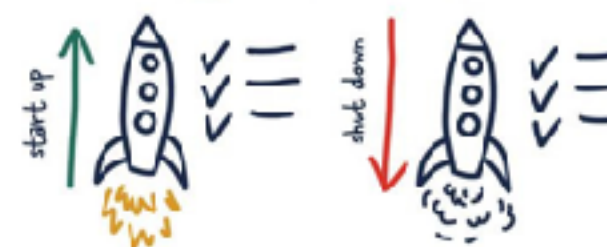
7 port binding



8 concurrency



9 disposability



12 admin processes



<https://architecturenotes.co/p/12-factor-app-revisited>



Container Design Principles

Build time

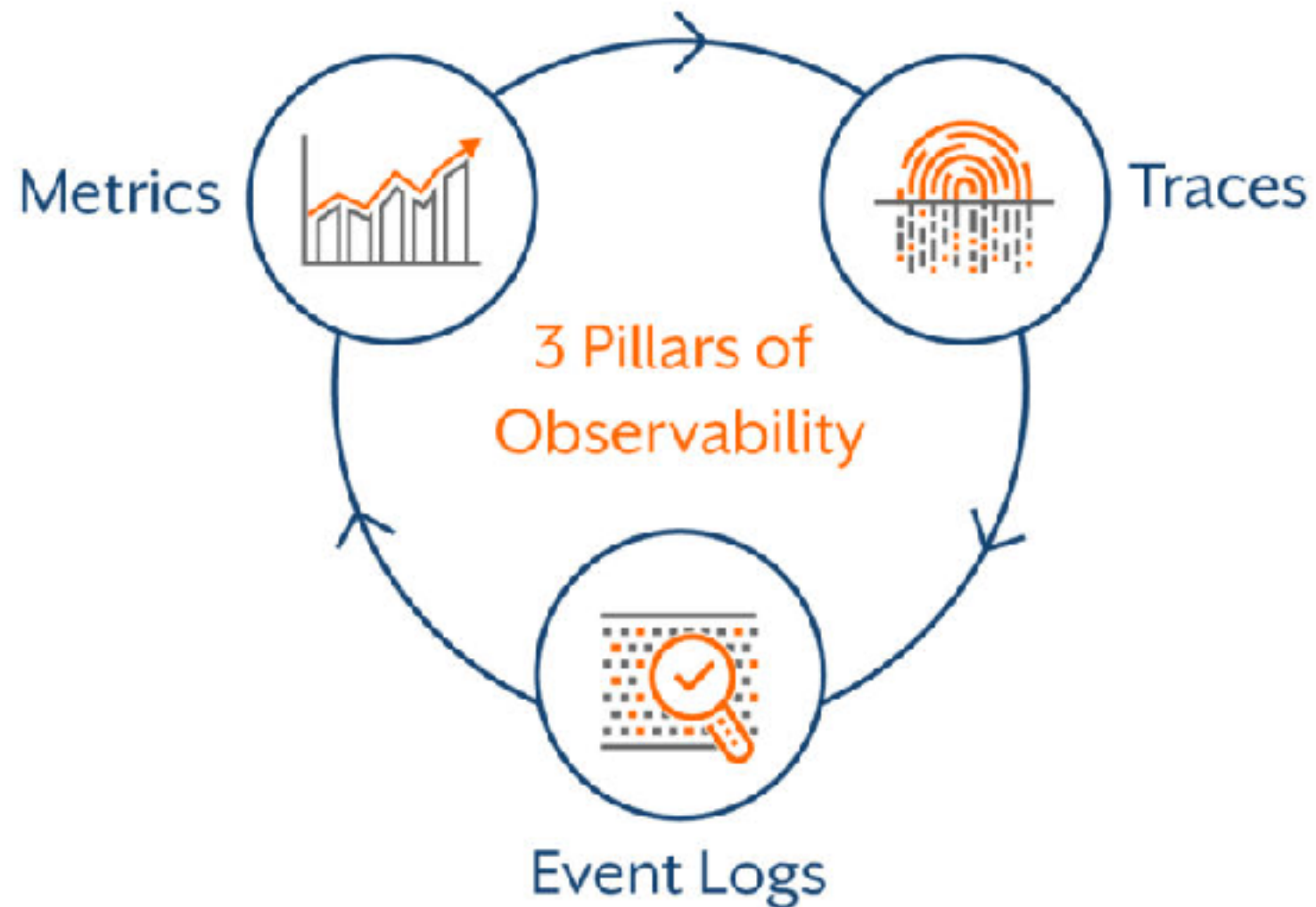
Single concern
Self-containment
Image immutable

Runtime

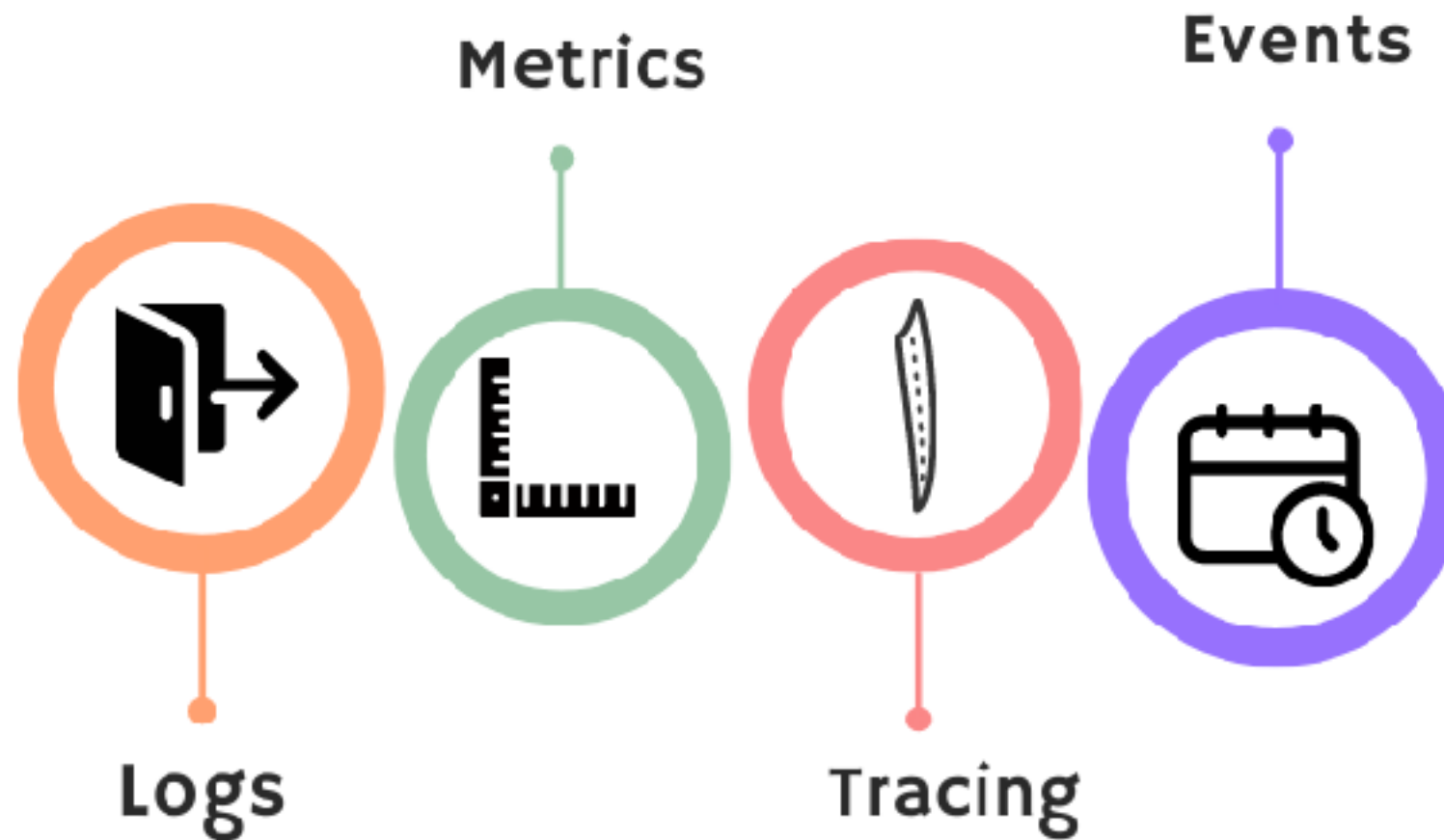
Runtime confinement
Process disposable
Lifecycle conformance
High observability



Pillar of Observability



THE FOUR PILLARS OF API OBSERVABILITY

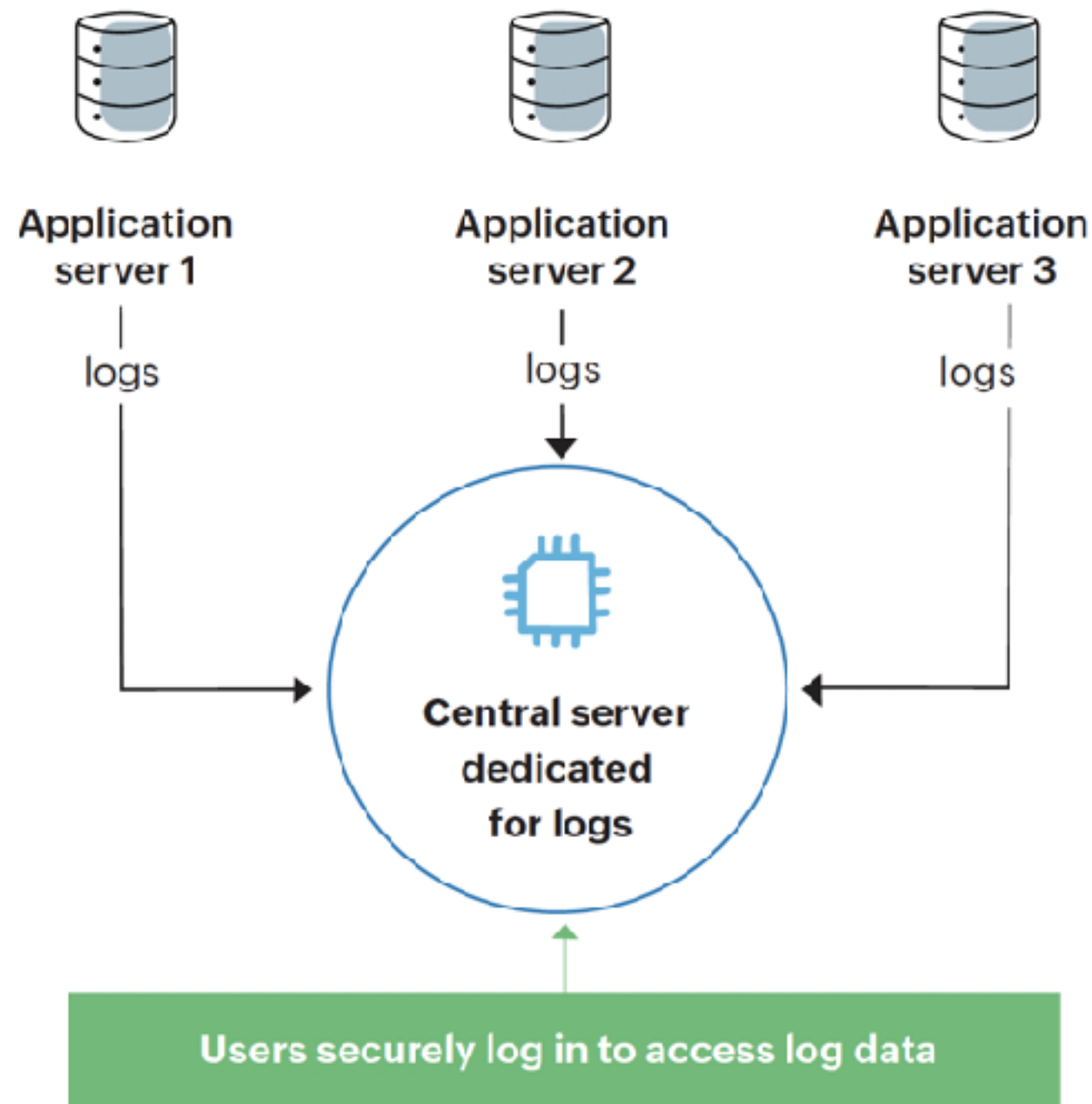


{APIToolkit.io}

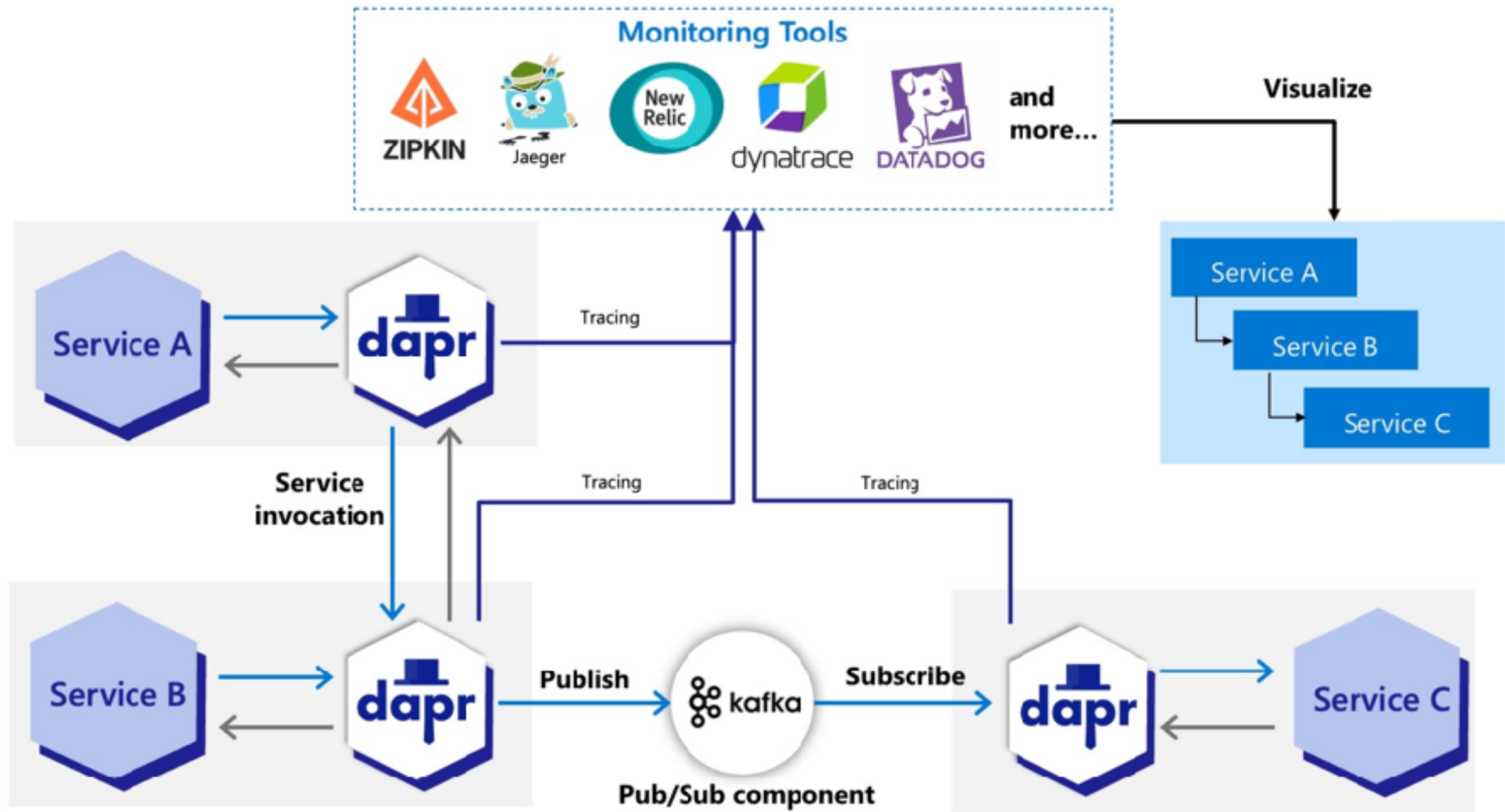
<https://apitoolkit.io/blog/the-four-pillars-of-api-observability/>



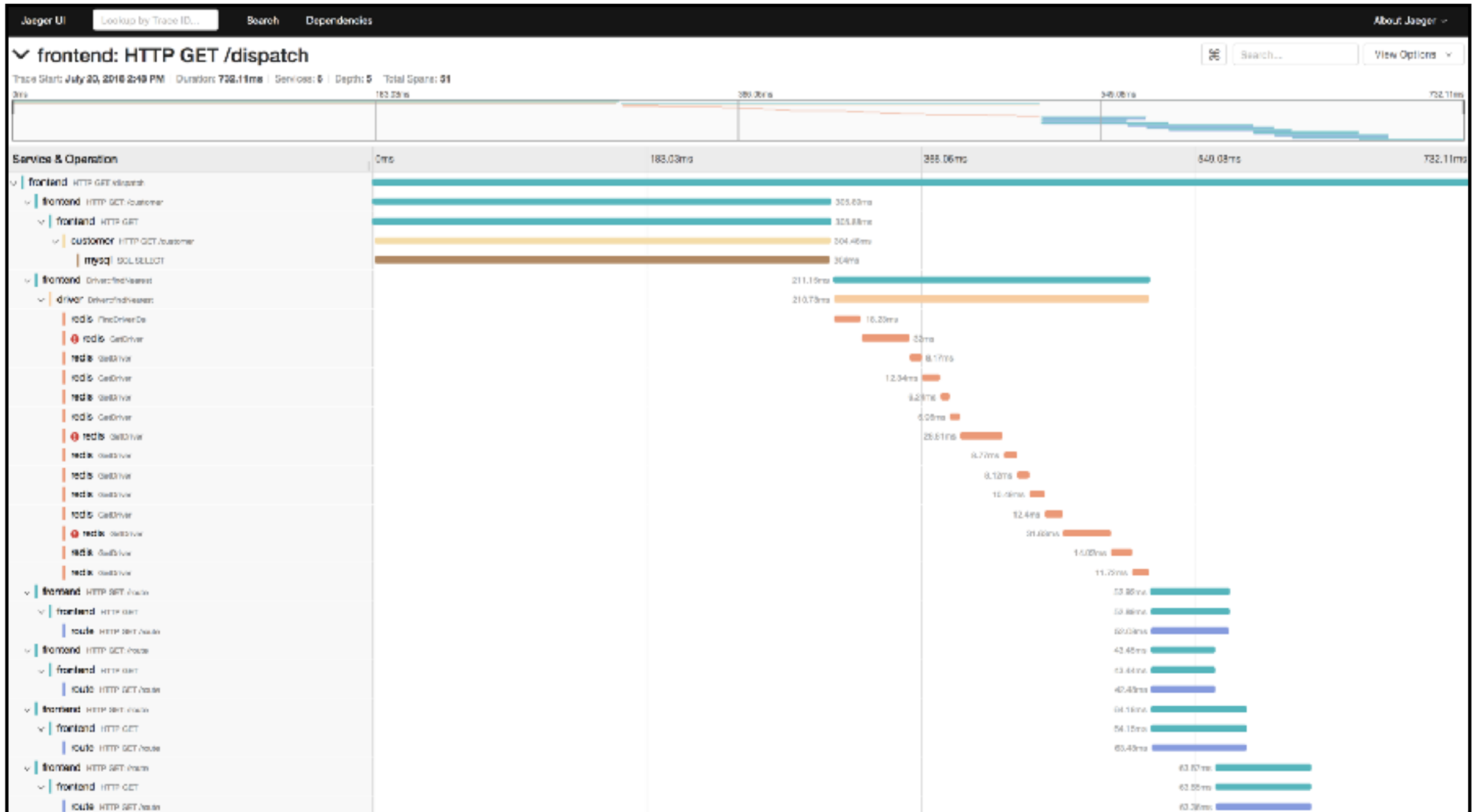
Centralized Logging



Distributed Tracing



Distributed Tracing

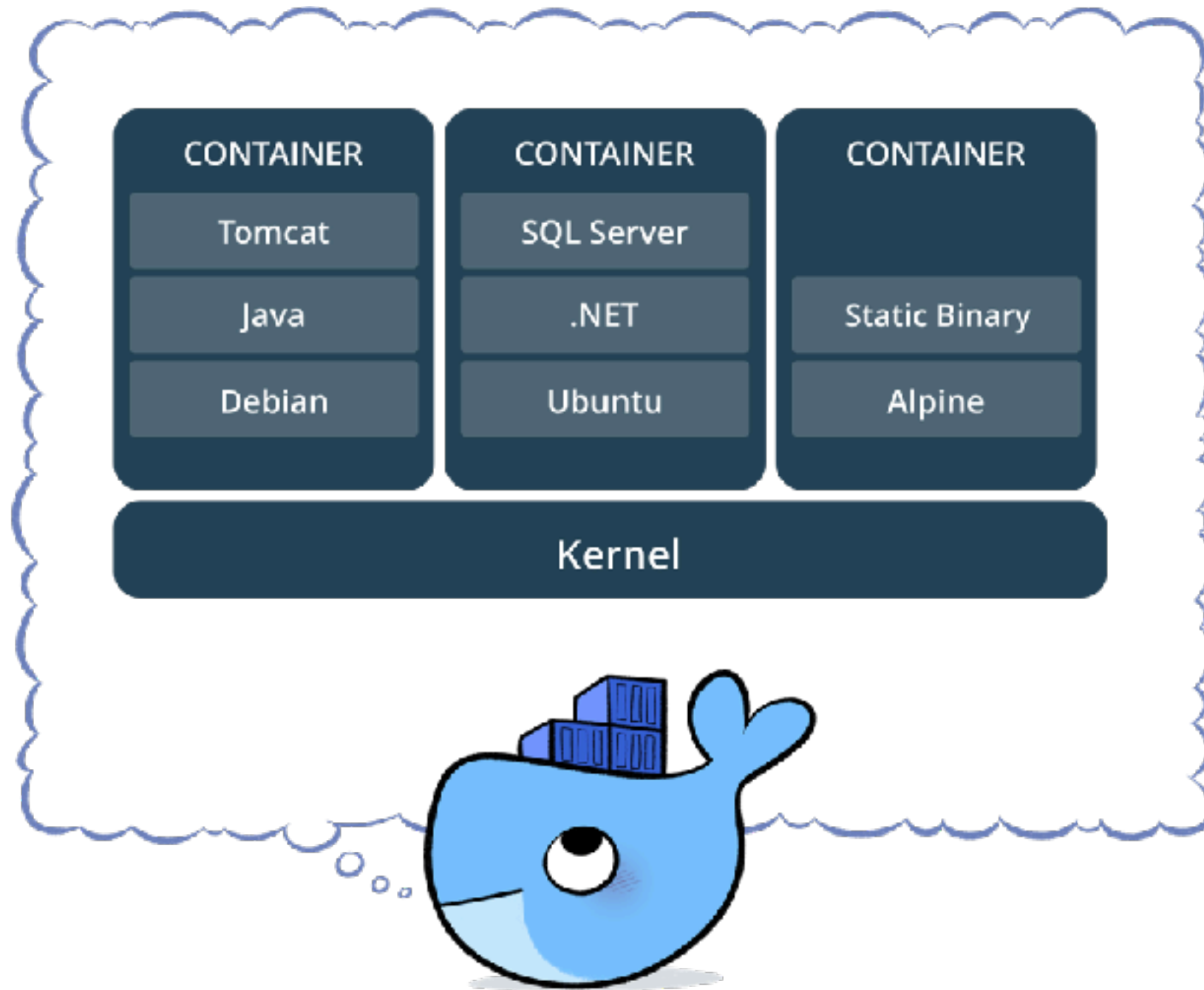


Manage Container with Docker

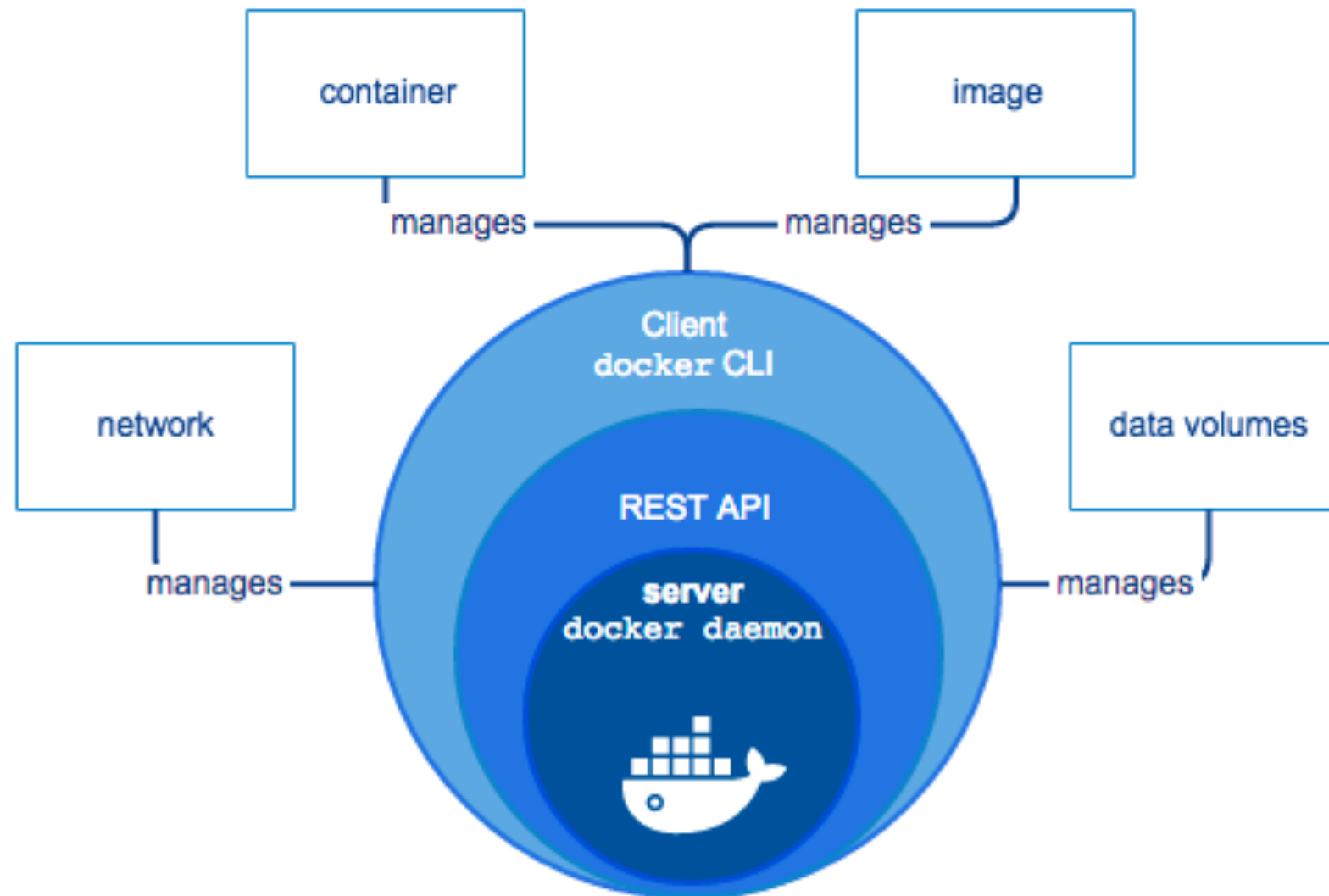
<https://www.docker.com/>



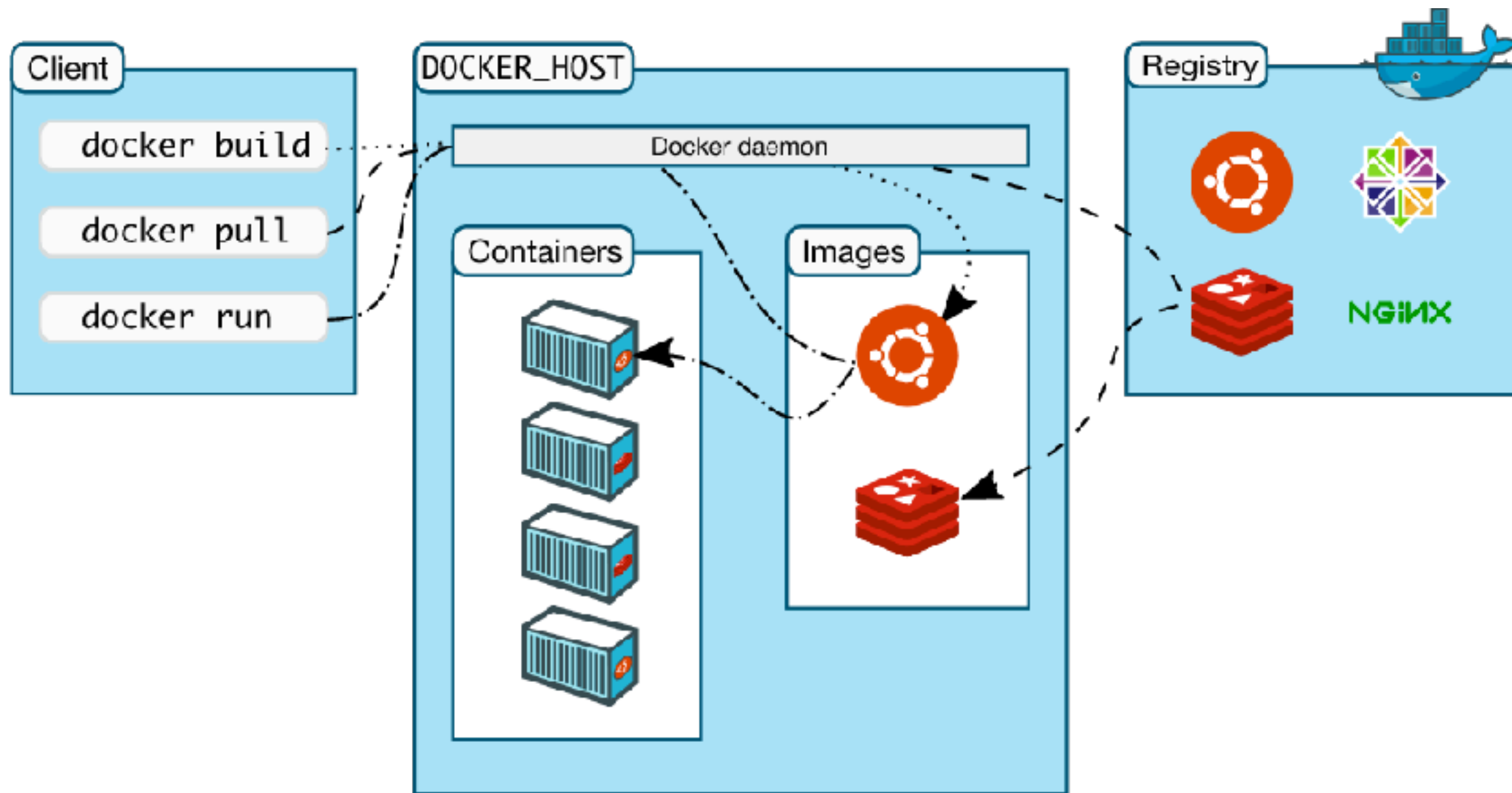
Container with Docker



Basic of Docker



How docker works ?



Docker Components ?

Image
Container
Registry



Image vs Container

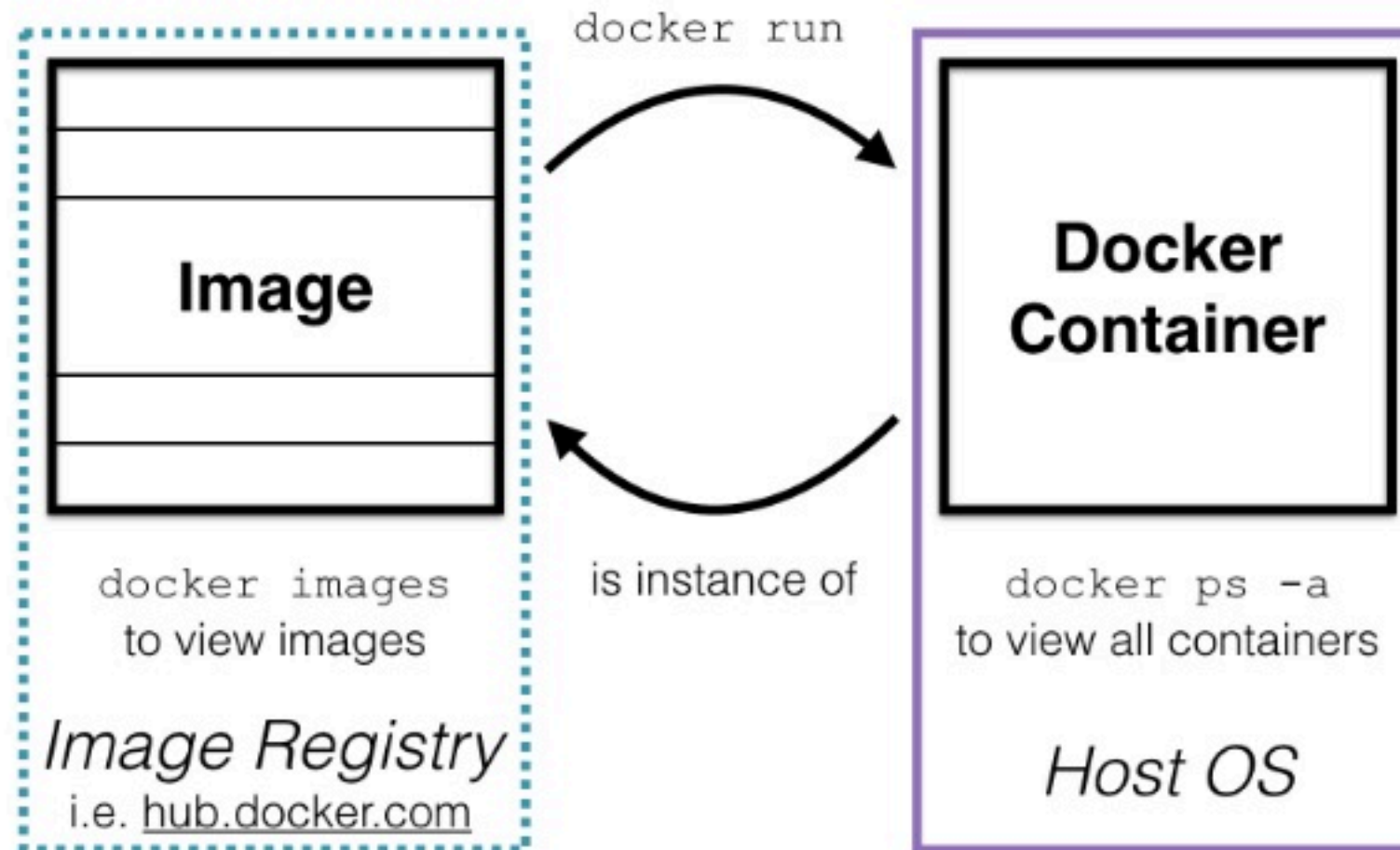
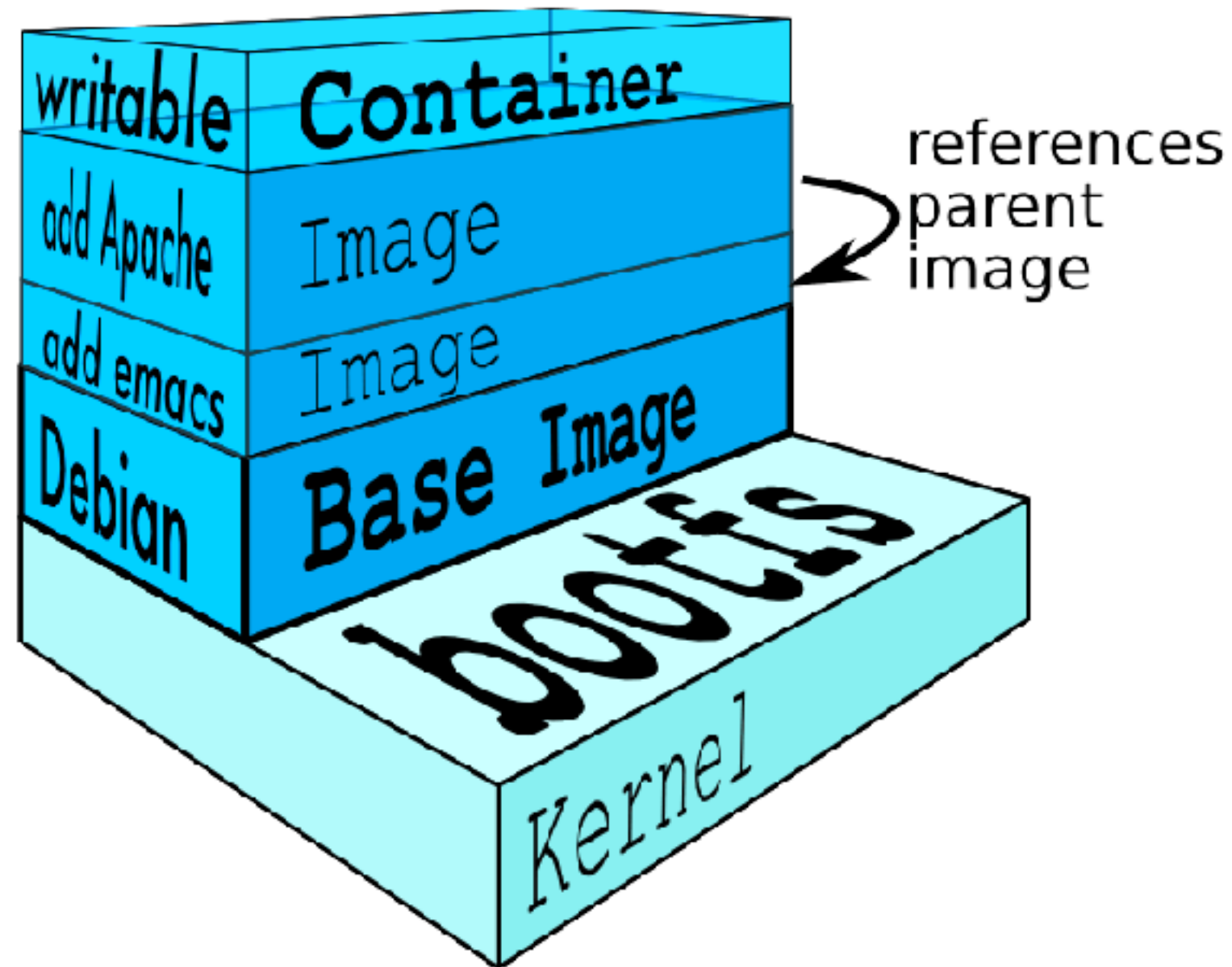


Image vs Container

Image like template/blueprint
Containers are created from image



Docker Image



Building Image ?

Use Dockerfile

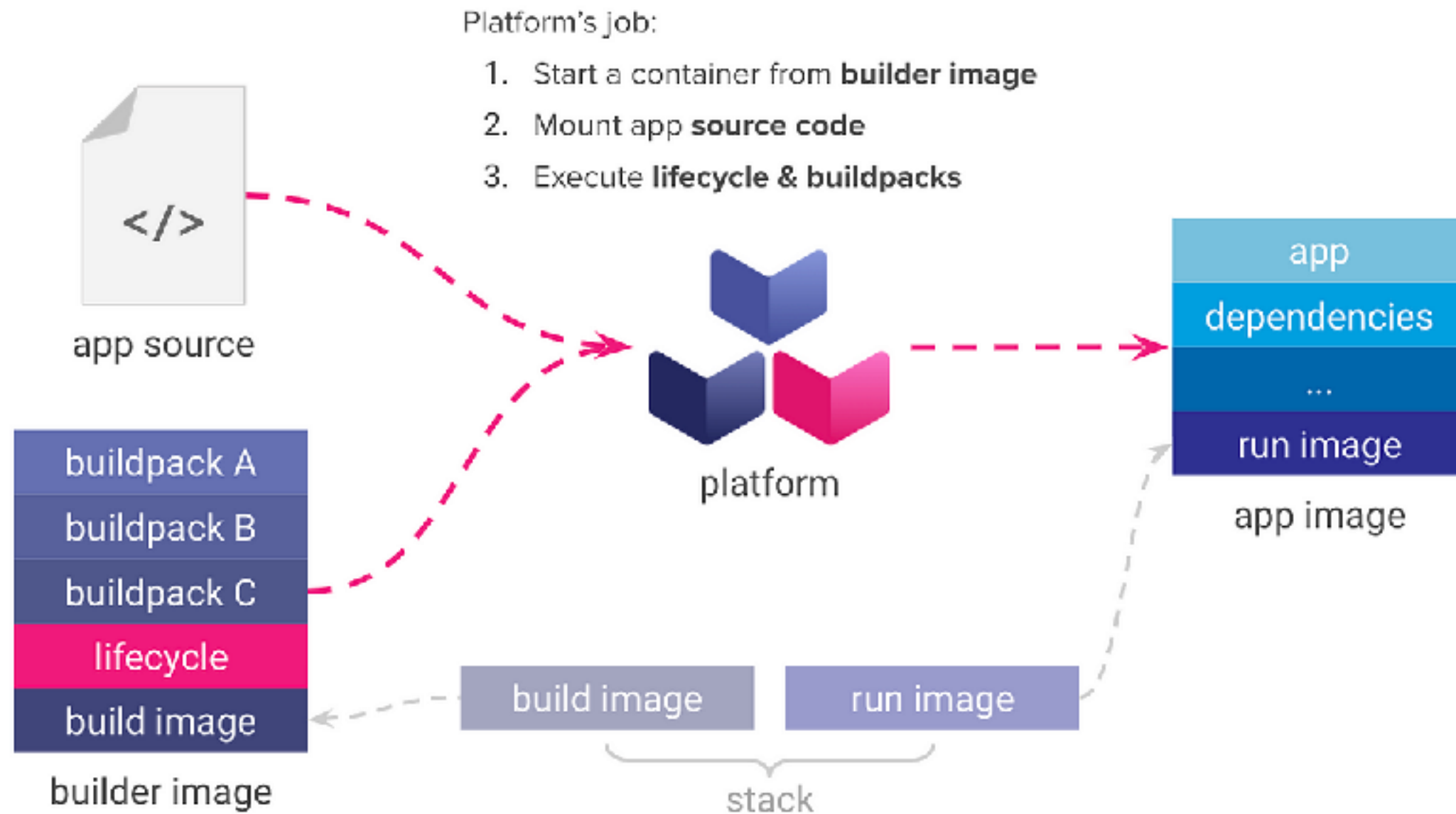
Build from container

Use cloud native buildpacks

<https://buildpacks.io/>



Cloud Native Buildpacks



<https://buildpacks.io/>



Dockerfile

Recipe for a Docker image

Contain series of instructions

Build image from docker cli and docker compose

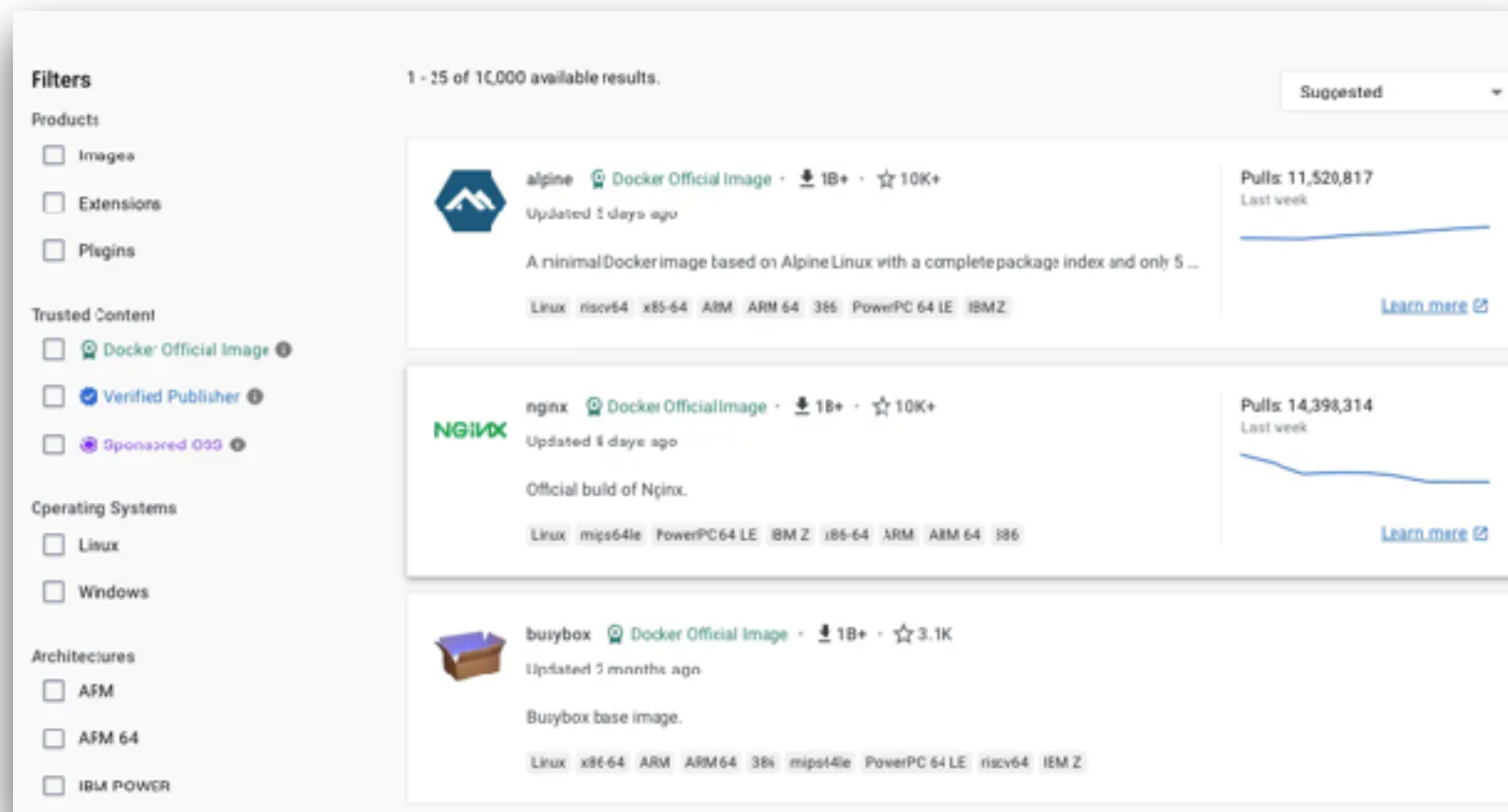
<https://docs.docker.com/build/building/best-practices/>



Dockerfile with best practices

Multi-stage builds

Choose right base image (official, verified publisher)



<https://docs.docker.com/build/building/best-practices/>



Dockerfile with best practices

Use **.dockerignore** file

Don't install unnecessary packages

Decouple application to multiple containers

Pin base image version with digest

<https://docs.docker.com/build/building/best-practices/>



Dockerfile with best practices

Use **.dockerignore** file

Don't install unnecessary packages

Decouple application to multiple containers

Pin base image version with digest

<https://docs.docker.com/build/building/best-practices/>



Workshop with Docker

Create Dockerfile

Build image

Scan image (security)

Push image to container registry server

Pull image from container registry

<https://docs.docker.com/build/building/best-practices/>



Container Security ?

Image
security

Container
security

Orchestration
security

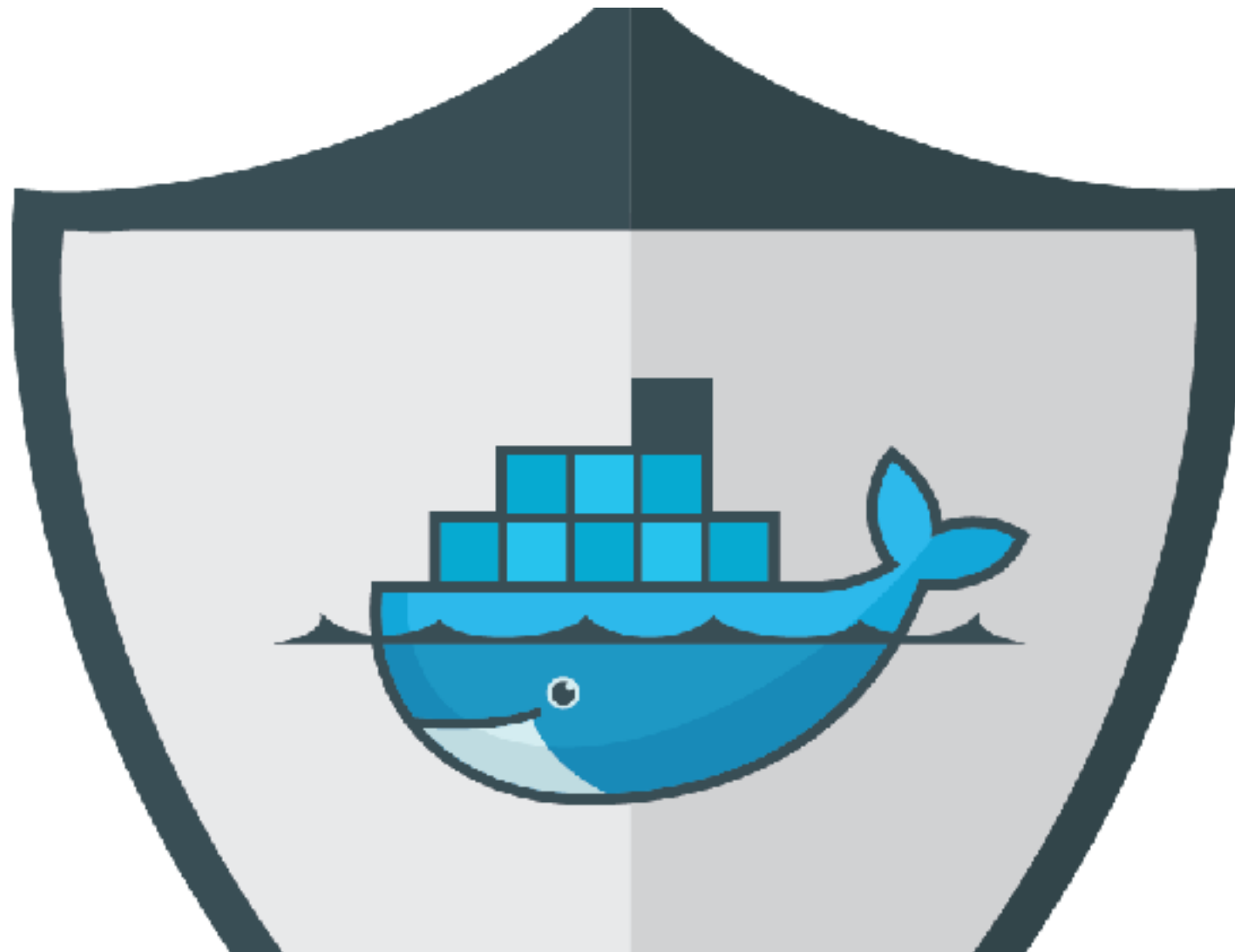
Compliance and
governance



<https://www.securitycompass.com/kontra/what-is-container-security/>



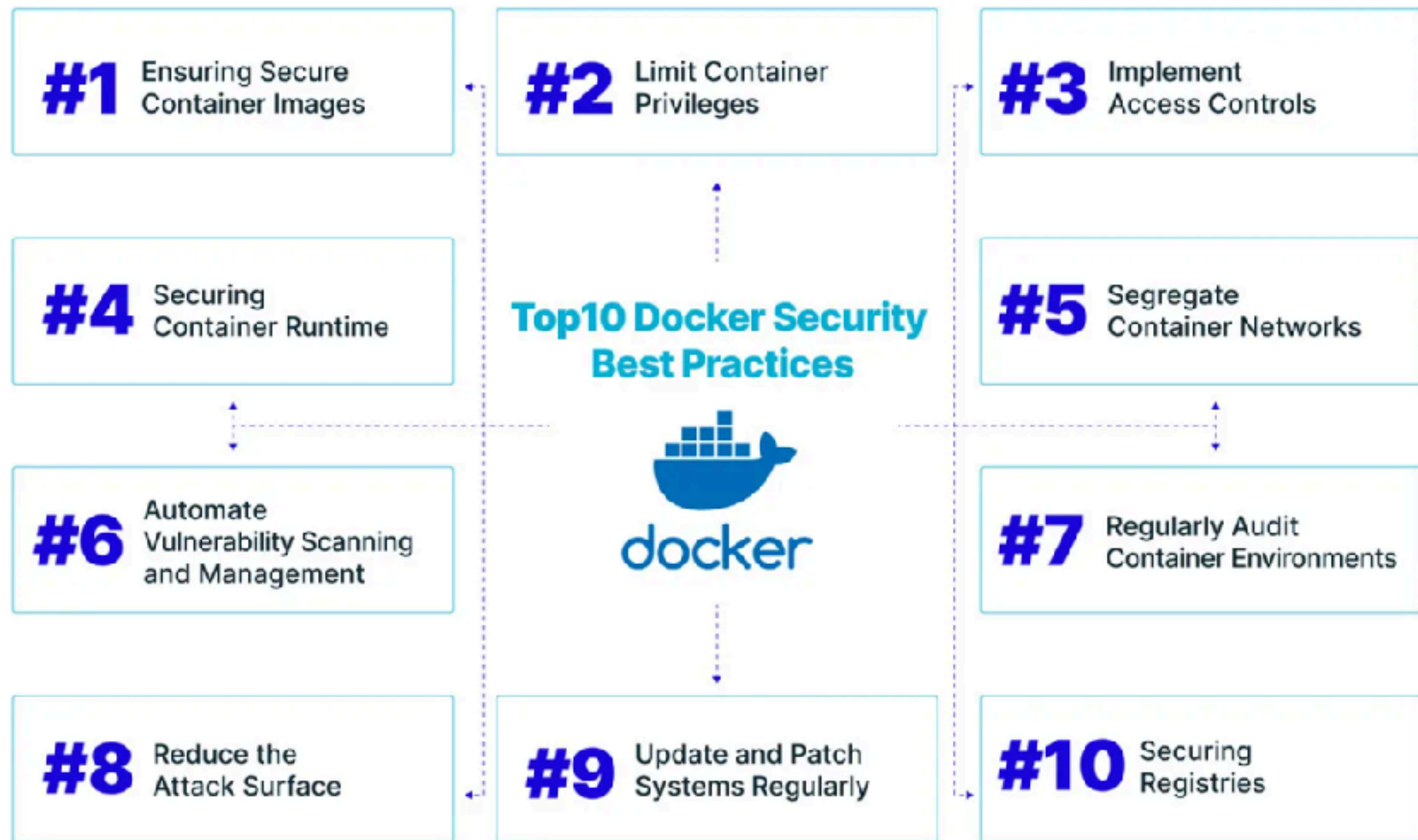
OWASP Docker Top 10



https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html



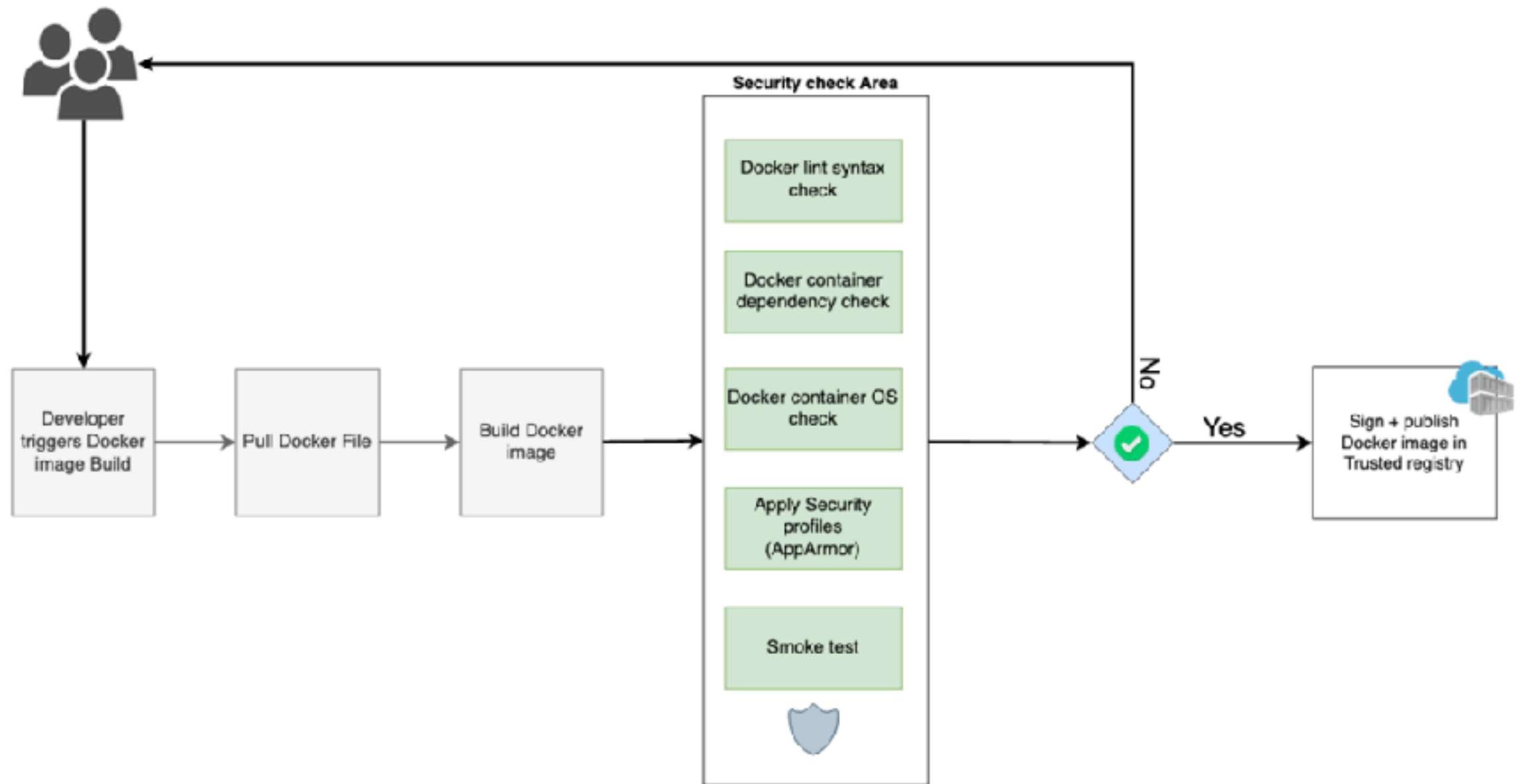
Top 10 Container Security



<https://www.aquasec.com/cloud-native-academy/container-security/container-security-best-practices/>



Container Security Pipeline



https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html



Scan Image



The all-in-one open source security scanner

Use Trivy to find vulnerabilities (CVE) & misconfigurations (IaC) across code repositories, binary artifacts, container images, Kubernetes clusters, and more. All in one tool!

[Get started](#) [Read the Docs](#)

<https://trivy.dev/latest/>



Scan Image



Clair is an open source project for the [static analysis](#) of vulnerabilities in application containers (currently including [OCI](#) and [docker](#)).

Clients use the Clair API to index their container images and can then match it against known vulnerabilities.

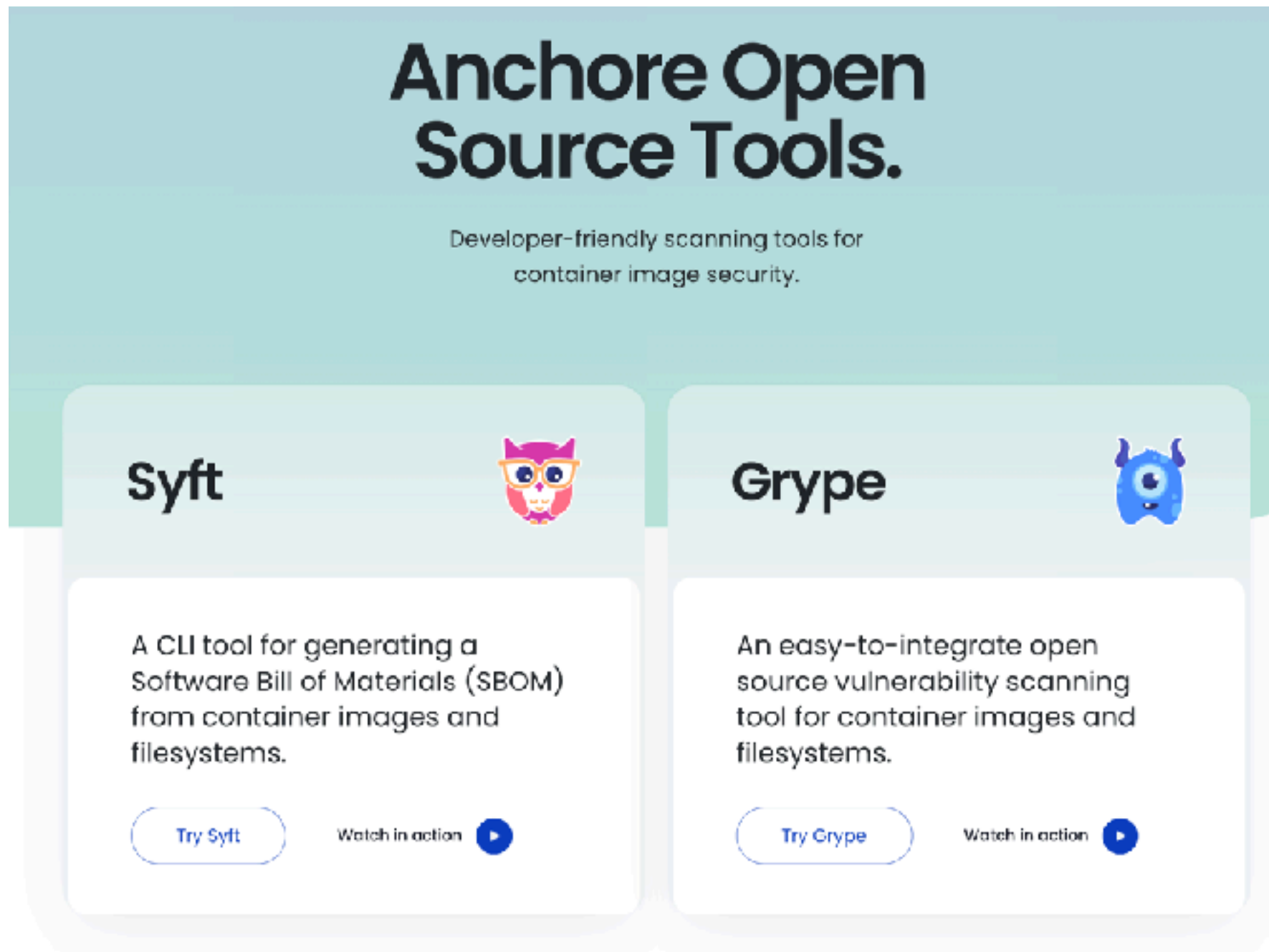
Our goal is to enable a more transparent view of the security of container-based infrastructure. Thus, the project was named `Clair` after the French term which translates to *clear, bright, transparent*.

[The book](#) contains all the documentation on Clair's architecture and operation.

<https://github.com/quay/clair>



Scan Image



The screenshot shows the 'Anchore Open Source Tools' landing page. At the top, the title 'Anchore Open Source Tools.' is displayed in a large, bold, black font. Below it, a subtitle reads 'Developer-friendly scanning tools for container image security.' in a smaller, regular black font. The page features two main tool cards. The left card is for 'Syft', featuring an owl icon with glasses. Its description is 'A CLI tool for generating a Software Bill of Materials (SBOM) from container images and filesystems.' Below the description are two buttons: 'Try Syft' and 'Watch in action' with a play button icon. The right card is for 'Grype', featuring a blue monster icon. Its description is 'An easy-to-integrate open source vulnerability scanning tool for container images and filesystems.' Below the description are two buttons: 'Try Grype' and 'Watch in action' with a play button icon. The background of the page is a light teal color with a subtle gradient.

Anchore Open Source Tools.

Developer-friendly scanning tools for container image security.

Syft 

A CLI tool for generating a Software Bill of Materials (SBOM) from container images and filesystems.

[Try Syft](#) [Watch in action](#) 

Grype 

An easy-to-integrate open source vulnerability scanning tool for container images and filesystems.

[Try Grype](#) [Watch in action](#) 

<https://anchore.com/opensource/>



Distroless image

Minimal container image

Contains only your app and essential dependencies

Exclude standard OS components

Small size

No shell
No package
manager

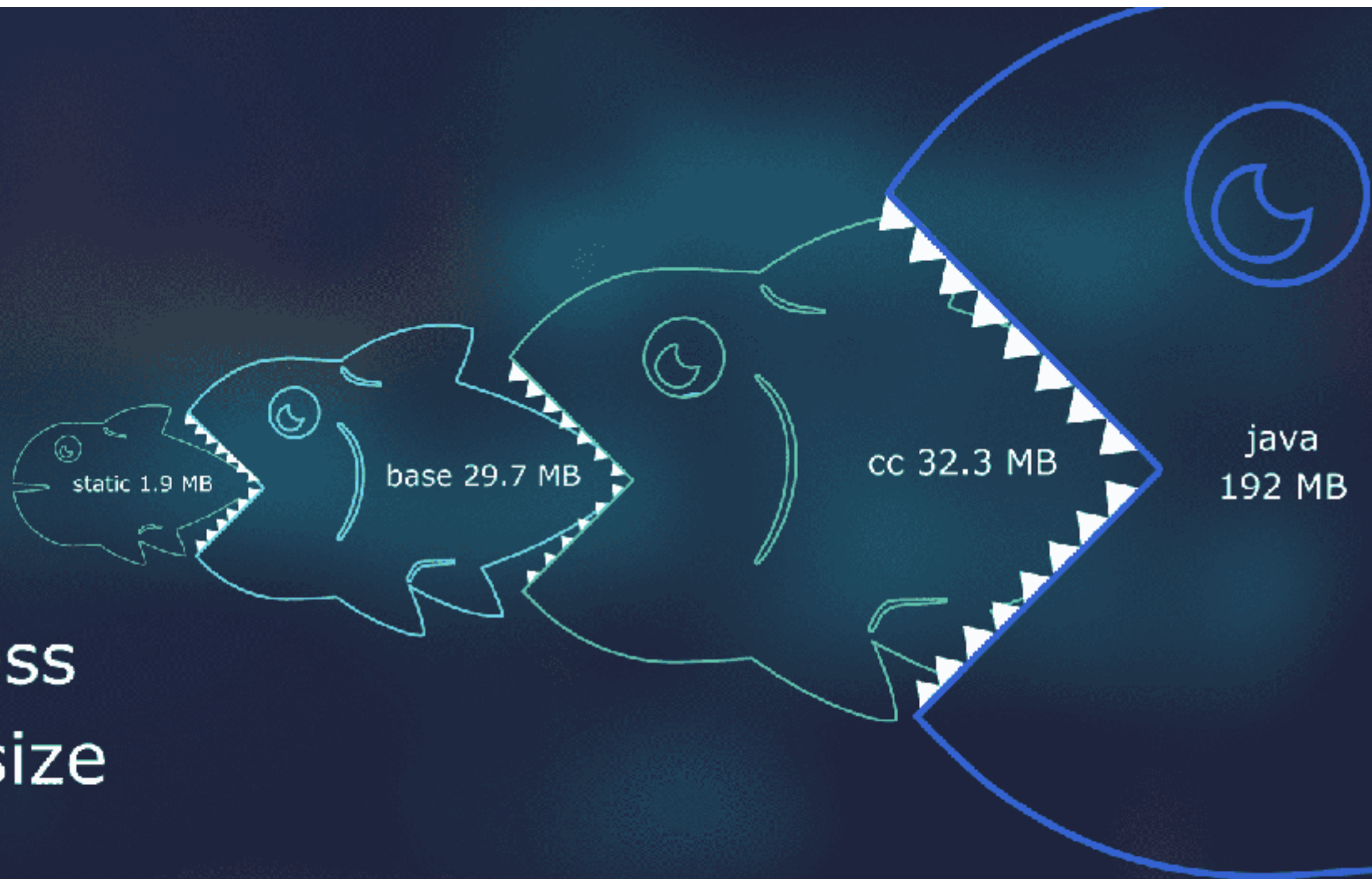
Fast startup

Reduce attack
surface

<https://docs.docker.com/dhi/core-concepts/distroless/>



Distroless image



Distroless
image size



Distroless image

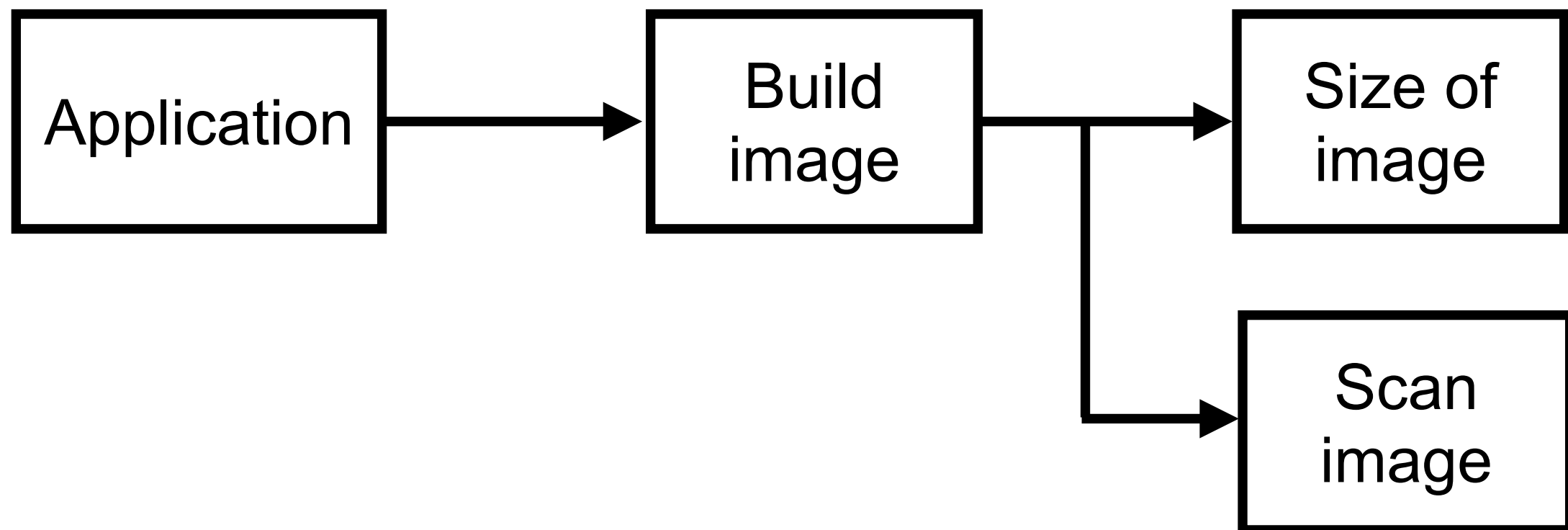


distroless

<https://github.com/GoogleContainerTools/distroless>

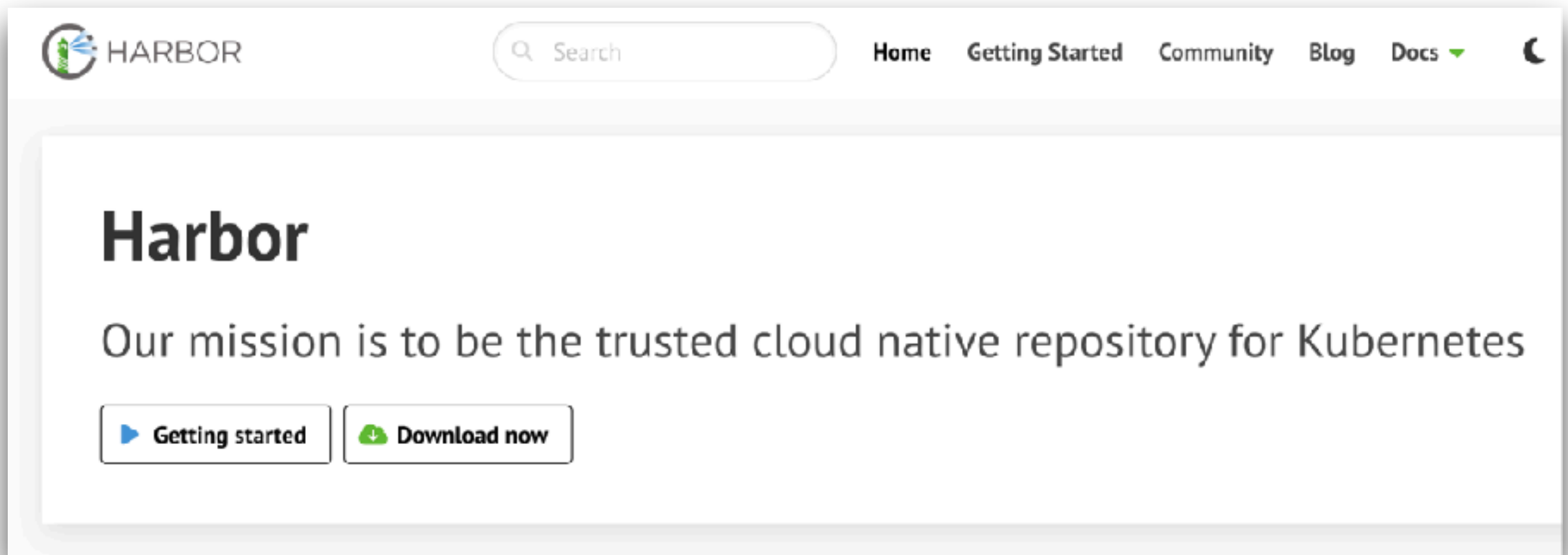


Workshop with Distroless image



Container Registry

Open source registry server



<https://goharbor.io/>



Supply Chain Security Attack !!



Software Bill of Material (SBOM)

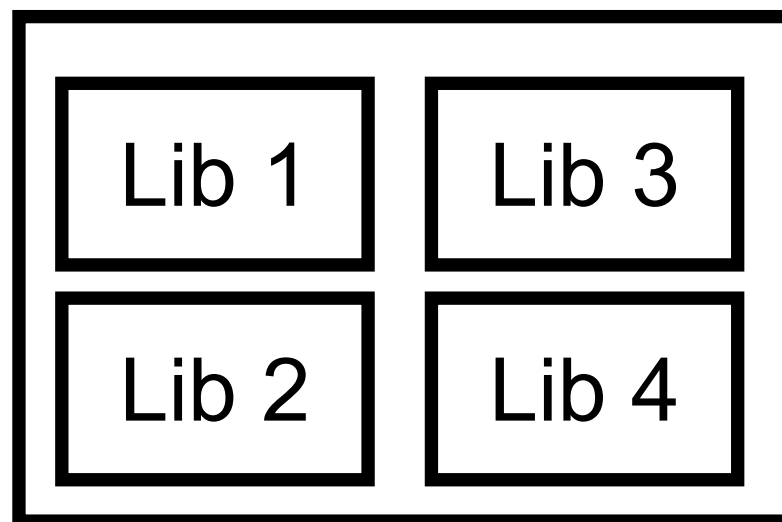
Comprehensive, nested inventory of ingredients

Components

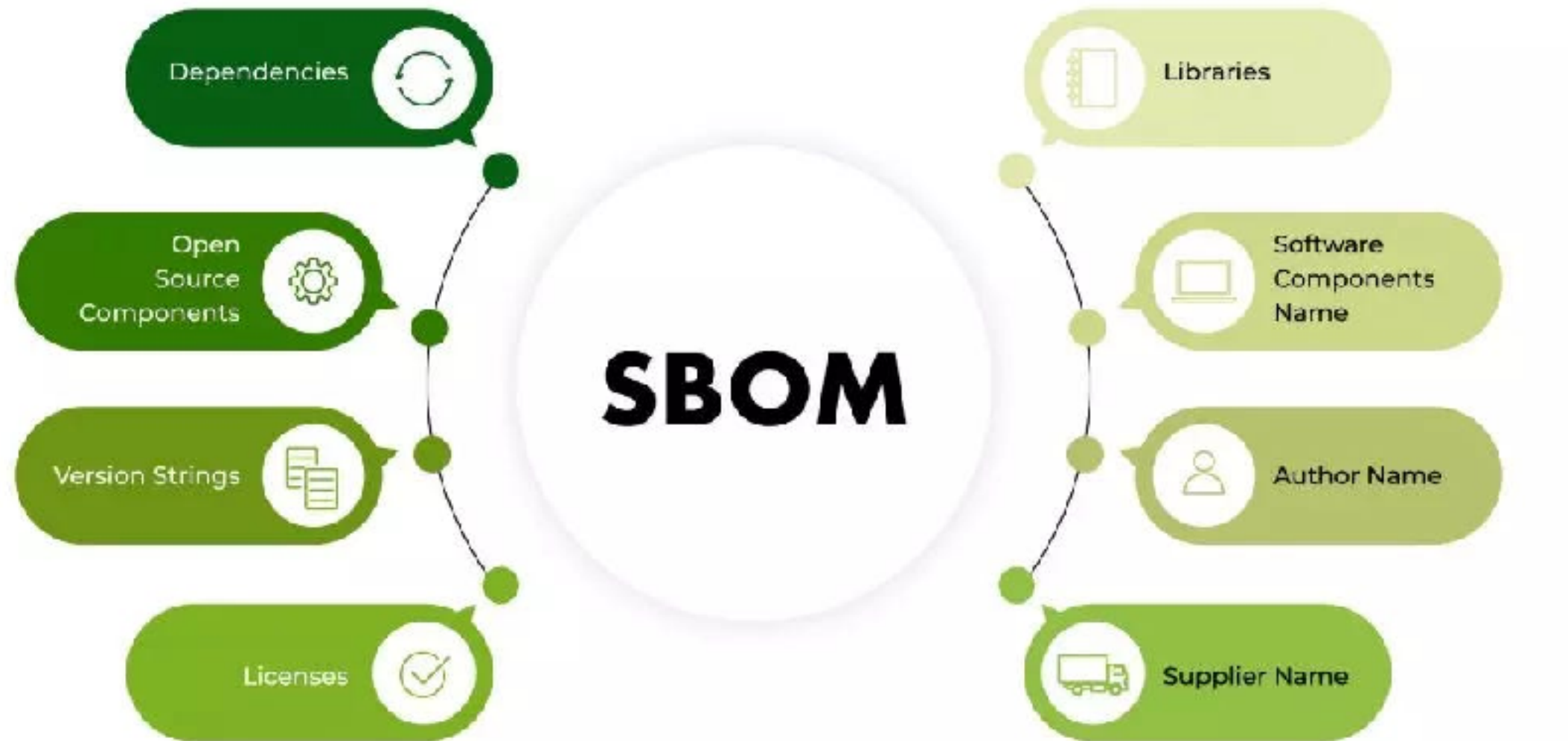
Libraries

Dependencies

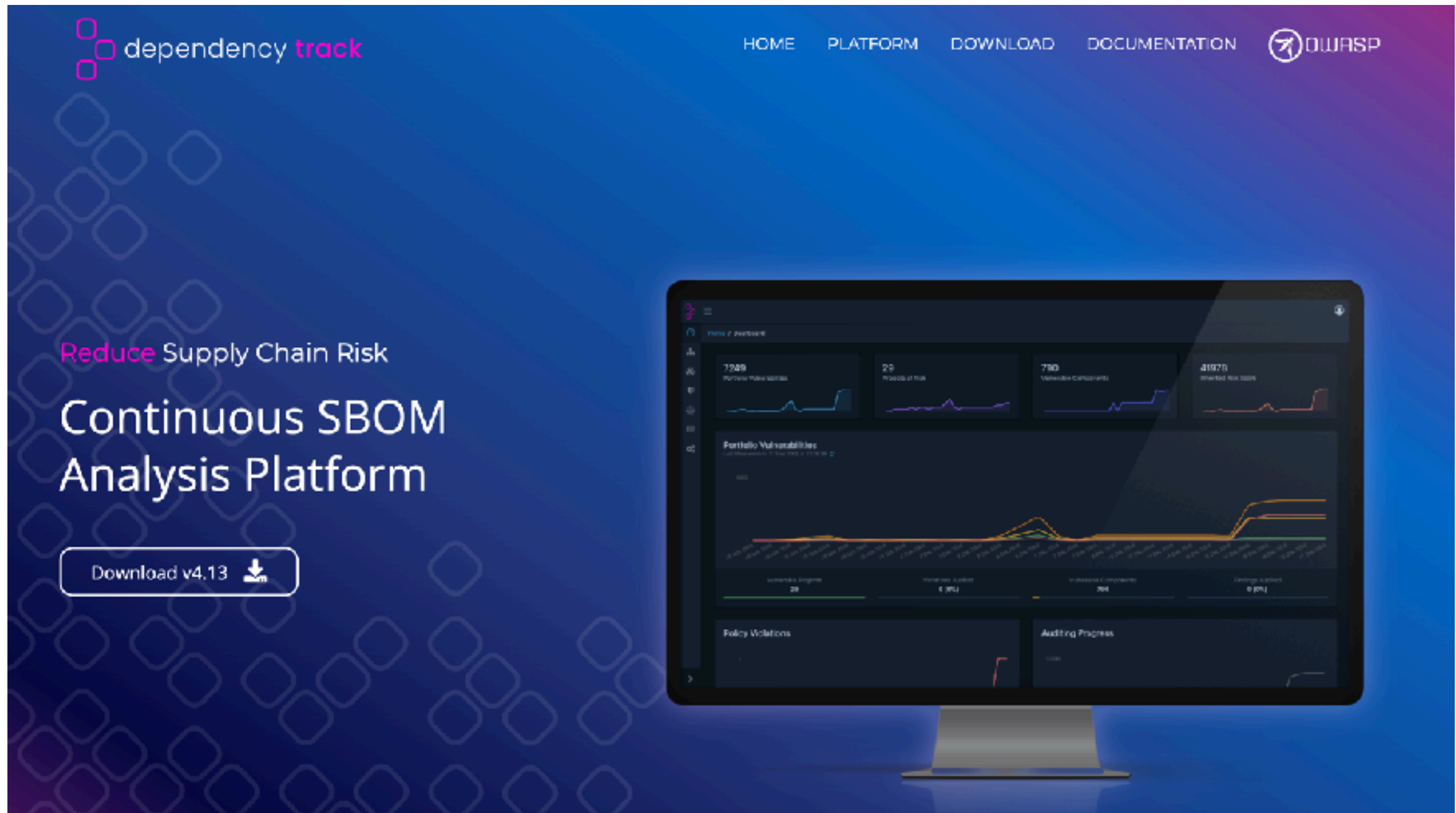
Docker image ?



Software Bill of Material (SBOM)



Dependency Track

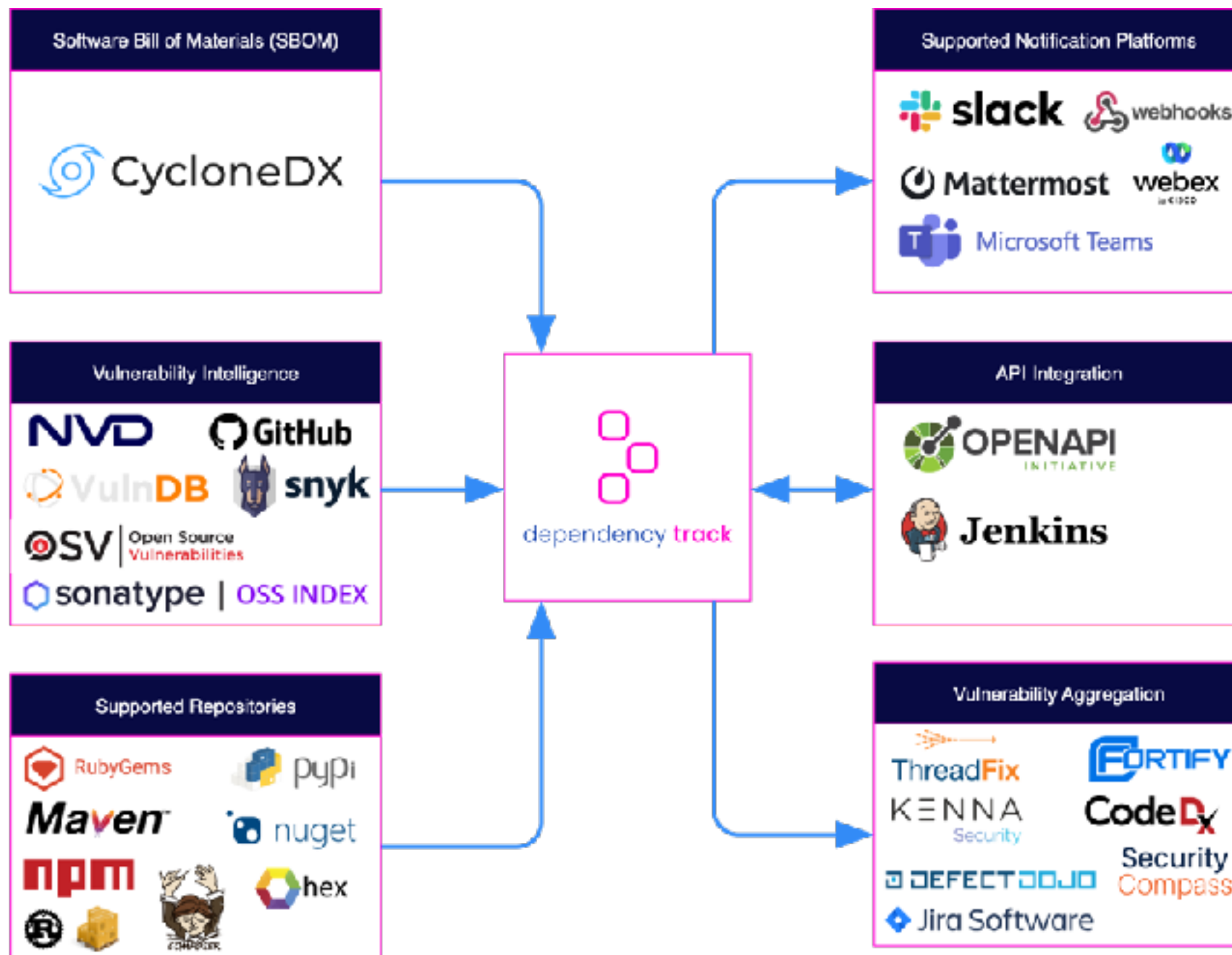


The image shows the Dependency Track website and a monitor displaying the platform interface. The website has a blue background with a pattern of white squares. The top navigation bar includes links for HOME, PLATFORM, DOWNLOAD, DOCUMENTATION, and a logo for OWASP. The main text on the website reads "Reduce Supply Chain Risk" and "Continuous SBOM Analysis Platform". A button labeled "Download v4.13" with a download icon is visible. The monitor displays the platform's dashboard, which includes several charts and metrics: "7249 Portfolio Vulnerabilities", "29 Policies at Risk", "780 Vulnerable Components", and "41870 Shared Risk Score". Below these are sections for "Portfolio Vulnerabilities", "Policy Violations", and "Auditing Progress".

<https://dependencytrack.org/>



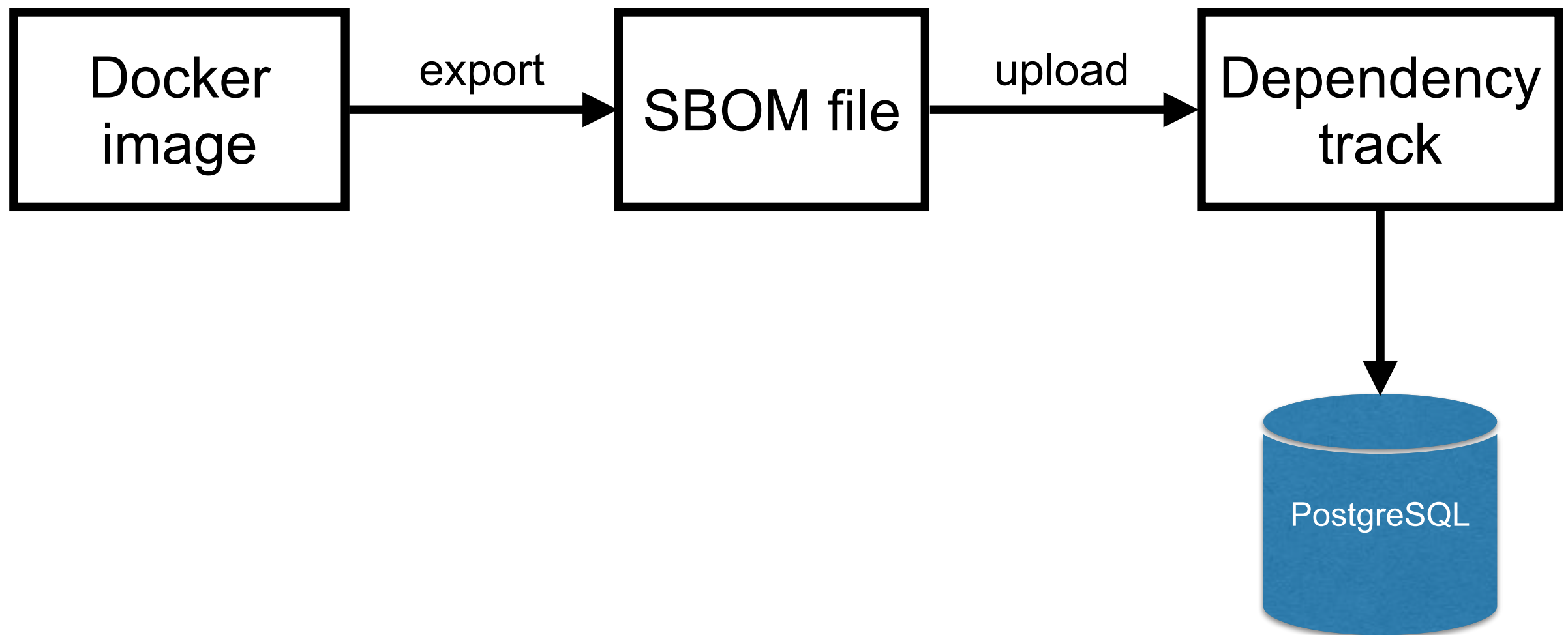
Dependency Track



<https://docs.dependencytrack.org/integrations/ecosystem/>



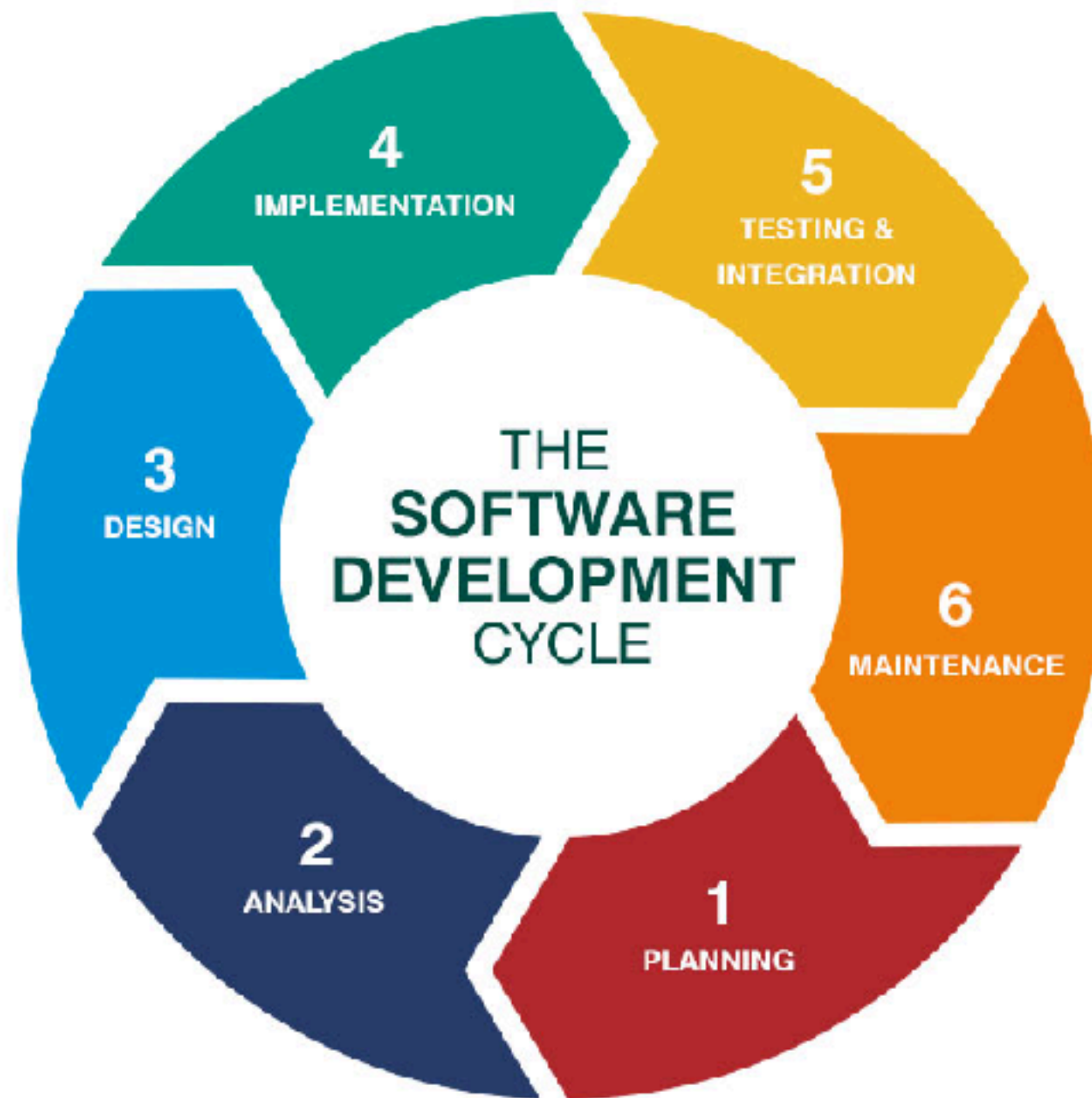
Workshop with SBOM



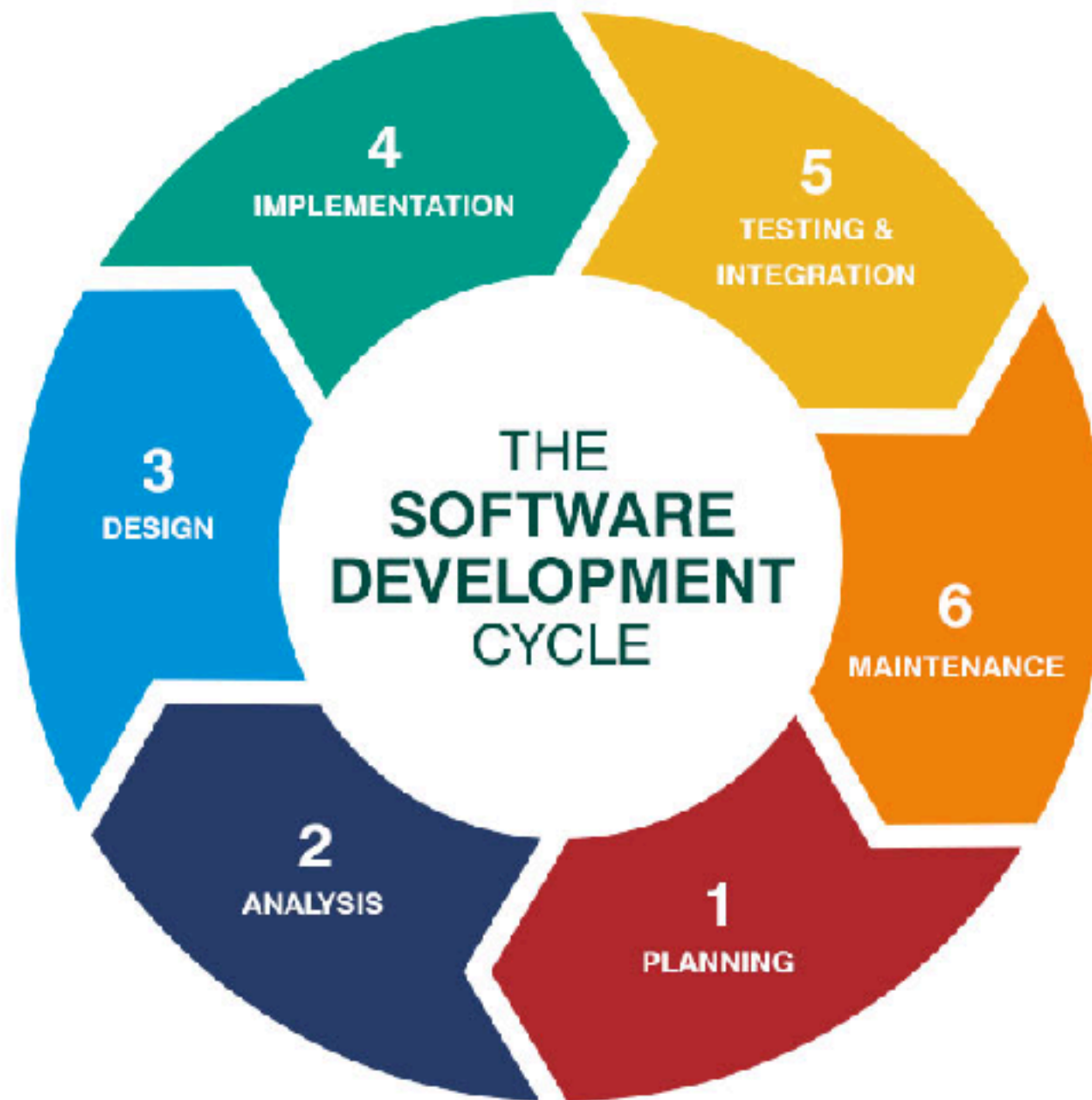
Shift Left Security in CI/CD



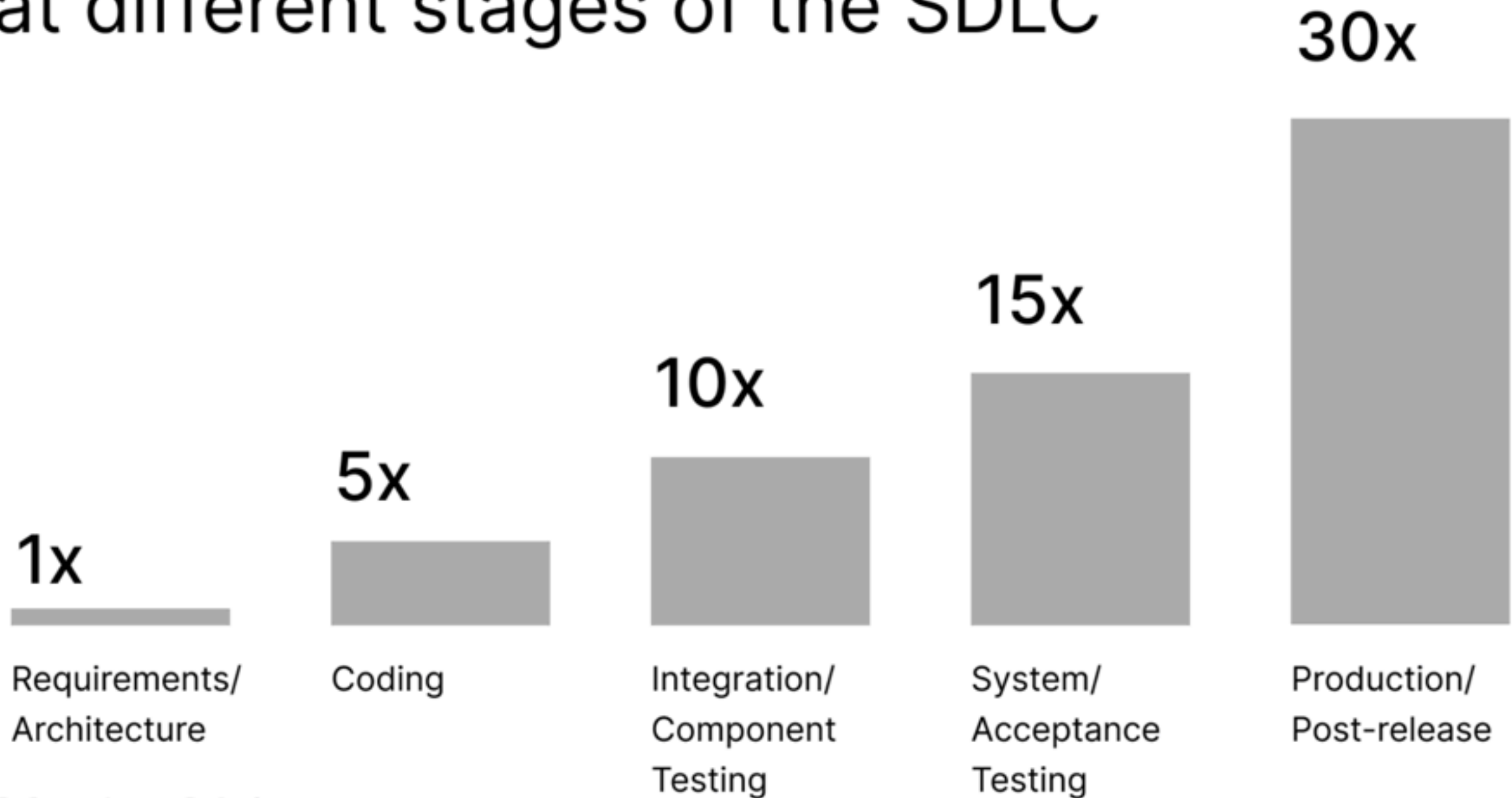
Software Development Life Cycle



Security ?



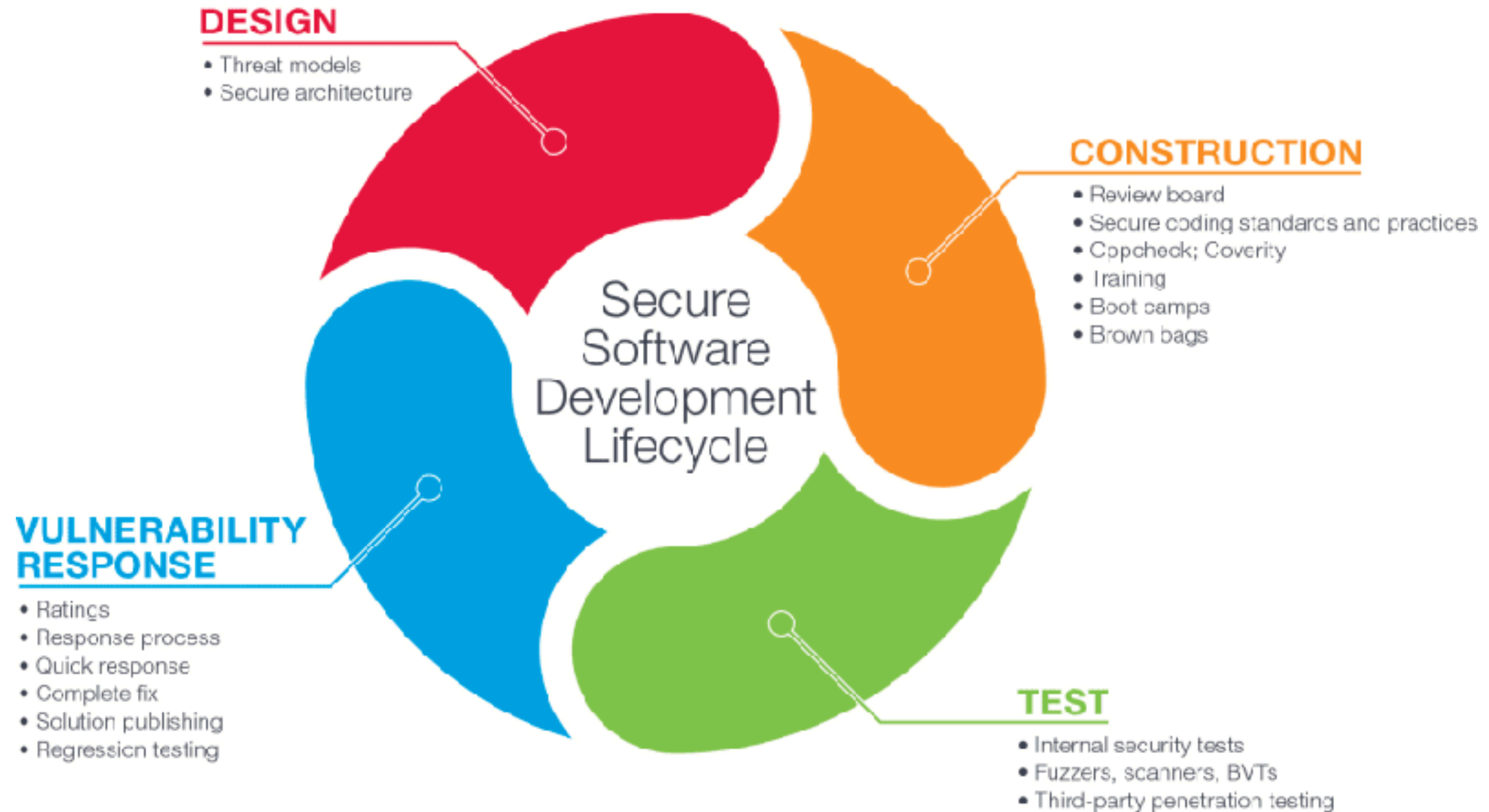
The relative cost of fixing a flaw at different stages of the SDLC



© Symphony Solutions



SDLC with Security



SDLC with Security



Plan and Develop

- ☐ Threat modelling
- ☐ IDE Security plugins
- ☐ Pre-commit hooks
- ☐ Secure coding standards
- ☐ Peer review

Commit the code

- ☐ Static application security testing
- ☐ Security unit and functional tests
- ☐ Dependency management
- ☐ Secure pipelines

Build and test

- ☐ Dynamic application security testing
- ☐ Cloud configuration validation
- ☐ Infrastructure scanning
- ☐ Security acceptance testing

Go to production

- ☐ Security smoke tests
- ☐ Configuration checks
- ☐ Live Site Penetration testing

Operate

- ☐ Continuous monitoring
- ☐ Threat intelligence
- ☐ Penetration testing
- ☐ Blameless postmortems

<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/secure/devsecops-controls>



Application Security Verification Standard (ASVS)

	Applicability	Building		Building, Configuration, Deployment Assurance and Verification			Assurance and Verification		
Level 1	All apps		Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Penetration Testing	DAST
Level 2	All apps	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Level 3	High Assurance	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Legend		Acceptable	Suitable						

<https://github.com/OWASP/ASVS/blob/master/4.0/en/0x03-Using-ASVS.md>



DevSecOps

Continuous improvement process
Plan-Do-Check-Act



When develop committed code ...

Dependency checking

Static Application Security Testing (SAST)

Security pipeline

Dynamic Application Security Testing (DAST)



Static Application Security Testing (SAST)

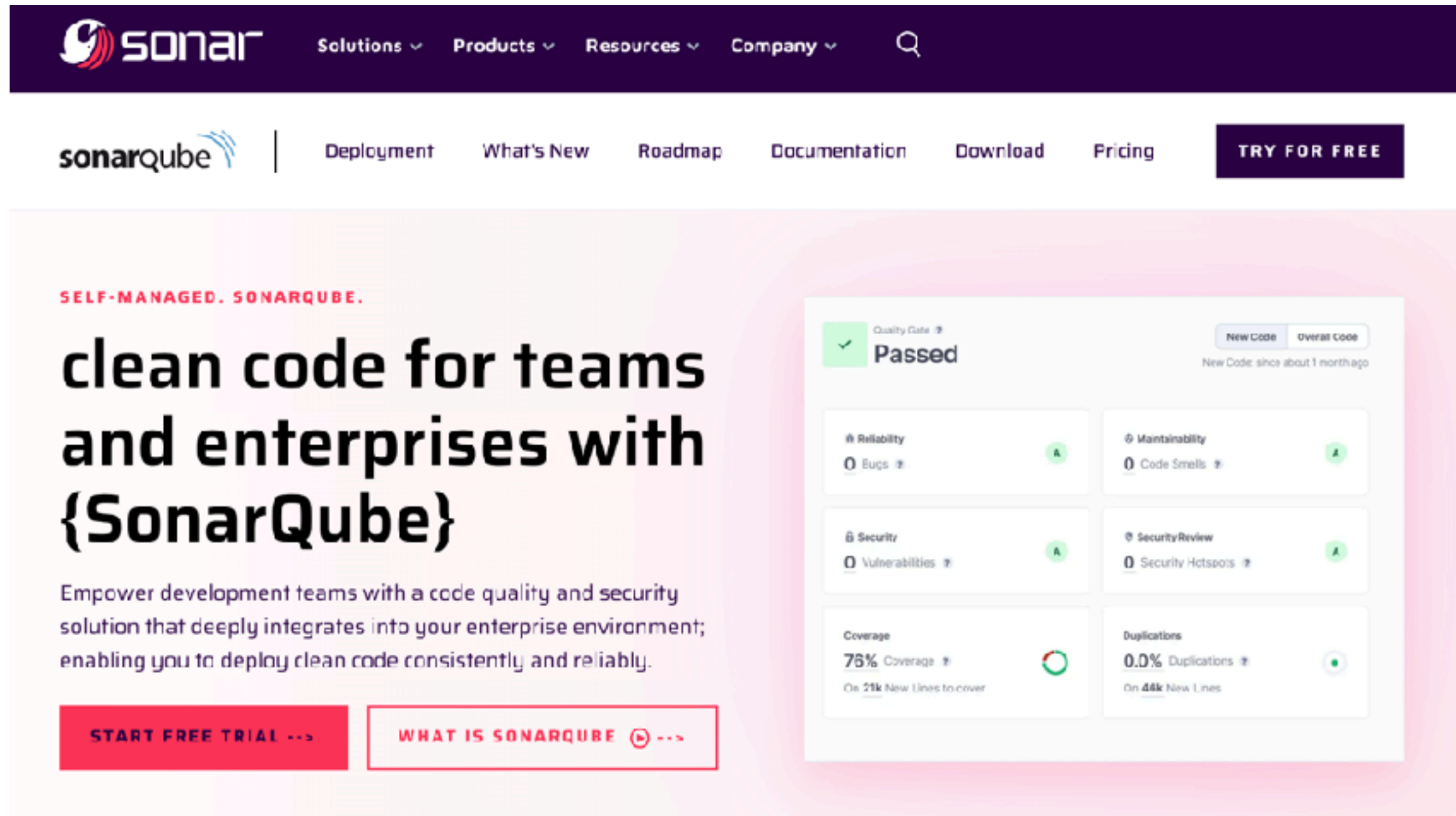
Static Code Analysis (detect 50%)
Scan and review code
Identify source of vulnerabilities
White-box testing

Must be integrate into development process to help development team

https://en.wikipedia.org/wiki/Static_application_security_testing



SonarQube



The banner features a dark purple header with the Sonar logo and navigation links. Below it, a lighter purple section contains the SonarQube logo and more navigation links, including a 'TRY FOR FREE' button. The main content area has a light pink background with a large heading, a descriptive paragraph, two call-to-action buttons, and a screenshot of the SonarQube dashboard showing a 'Passed' quality gate and various code quality metrics.

SELF-MANAGED. SONARQUBE.

clean code for teams and enterprises with {SonarQube}

Empower development teams with a code quality and security solution that deeply integrates into your enterprise environment; enabling you to deploy clean code consistently and reliably.

[START FREE TRIAL -->](#) [WHAT IS SONARQUBE -->](#)

Quality Gate **Passed**

New Code Overall Code
New Code: since about 1 month ago

Reliability 0 Bugs	Maintainability 0 Code Smells
Security 0 Vulnerabilities	Security Review 0 Security Hotspots
Coverage 76% Coverage On 21k New Lines to cover	Duplications 0.0% Duplications On 44k New Lines

<https://www.sonarsource.com/products/sonarqube/>

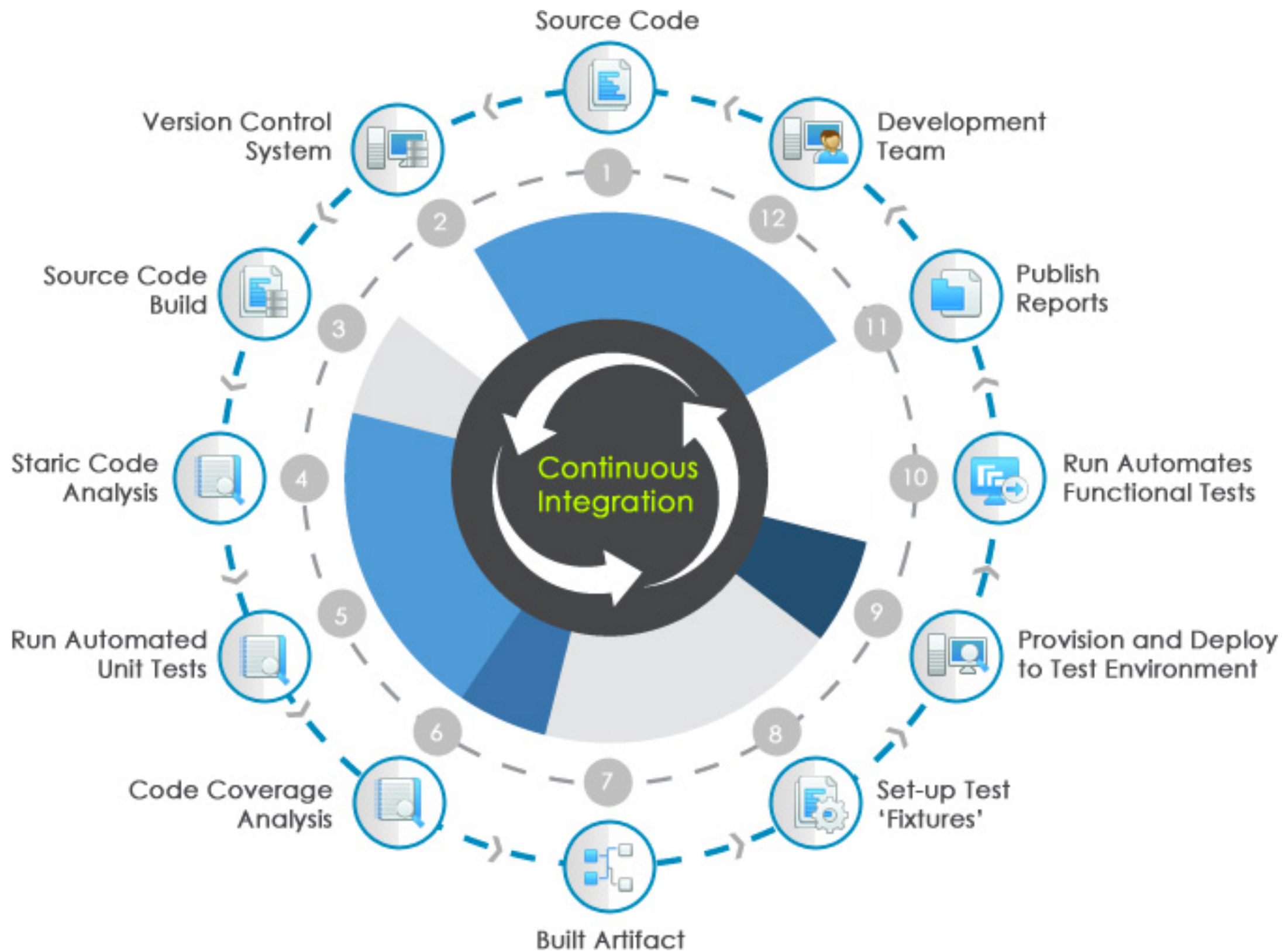


Security Pipeline

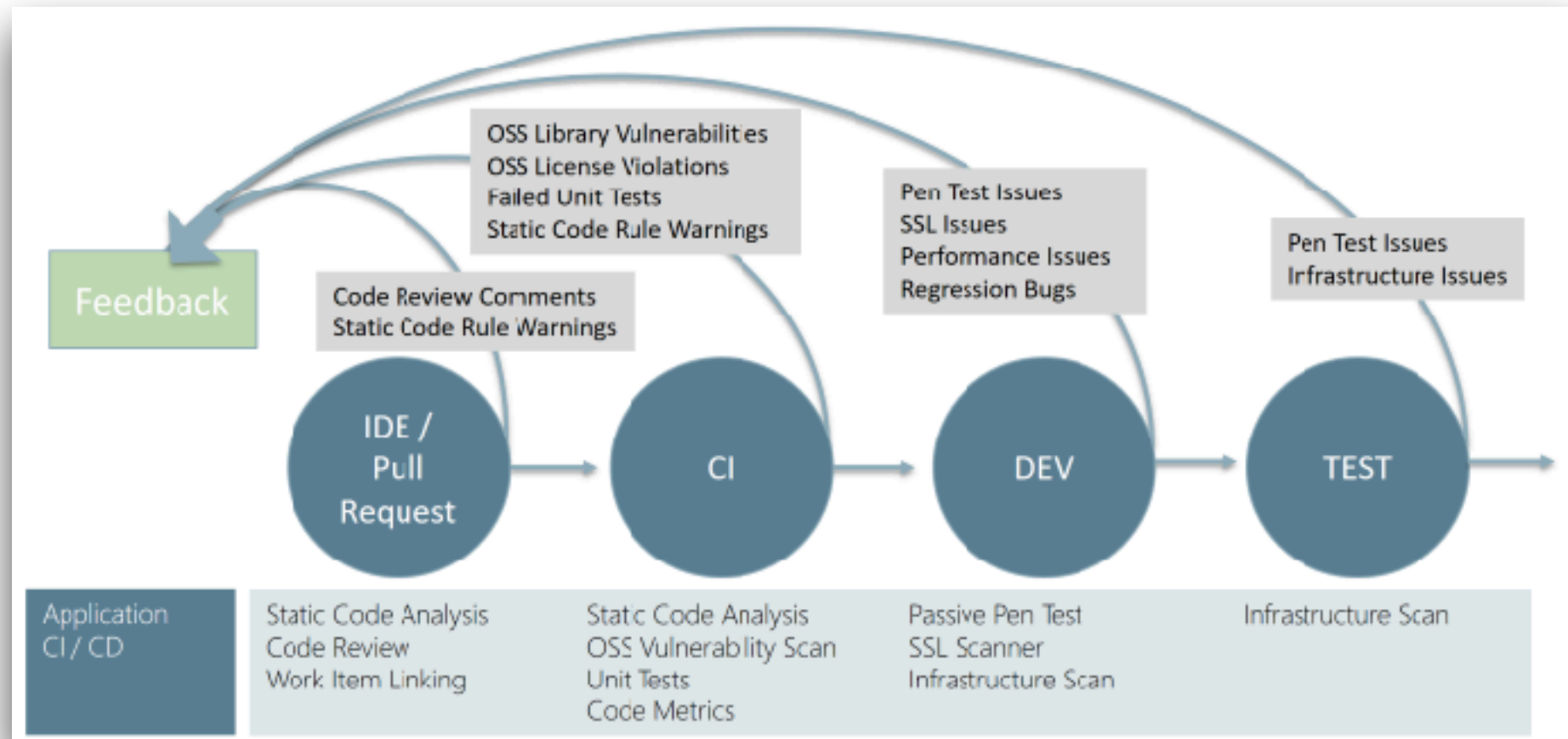
Implement security control in pipeline
Continuous Integration and Delivery

Build -> Test -> Deploy -> Monitoring

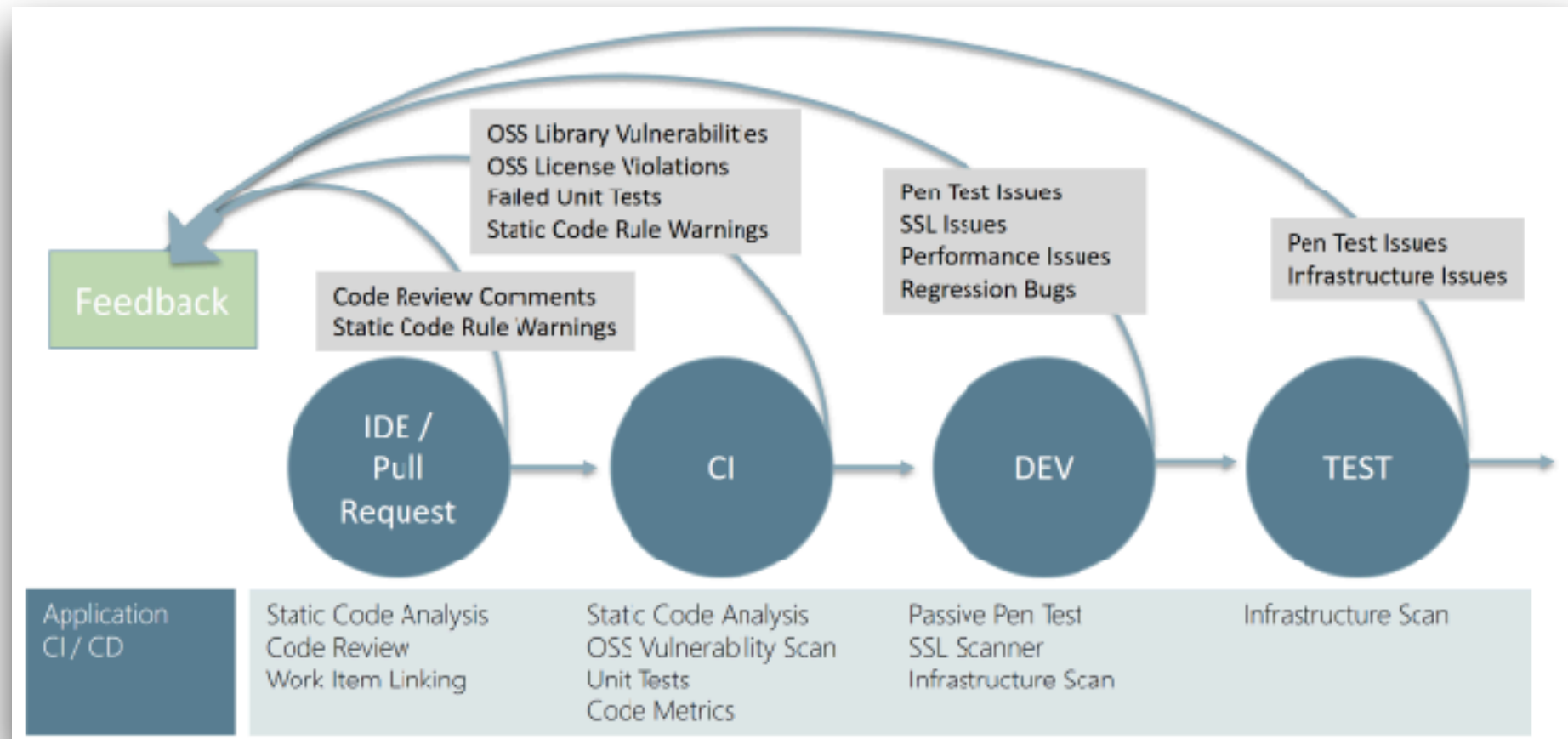




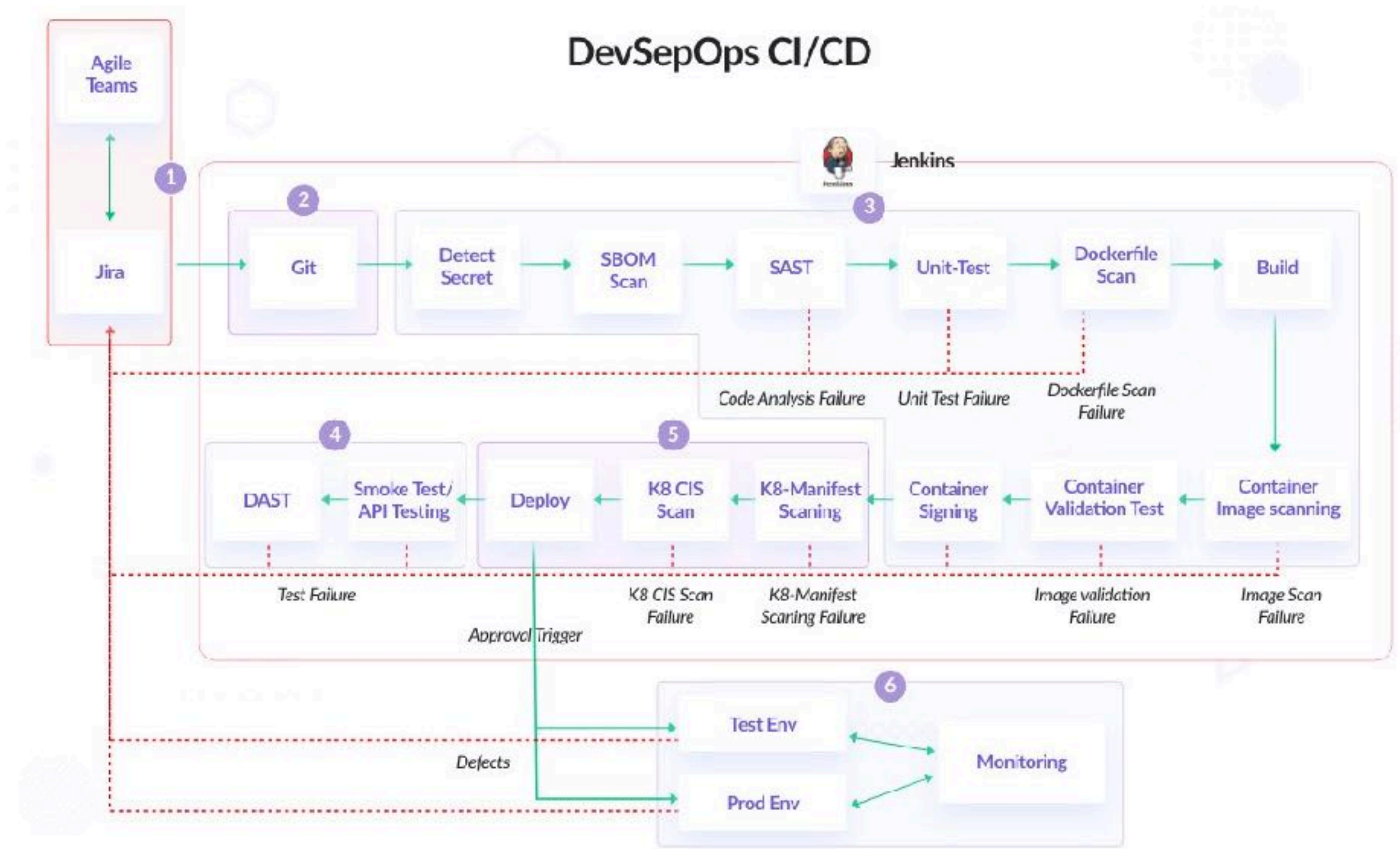
All about feedback loop



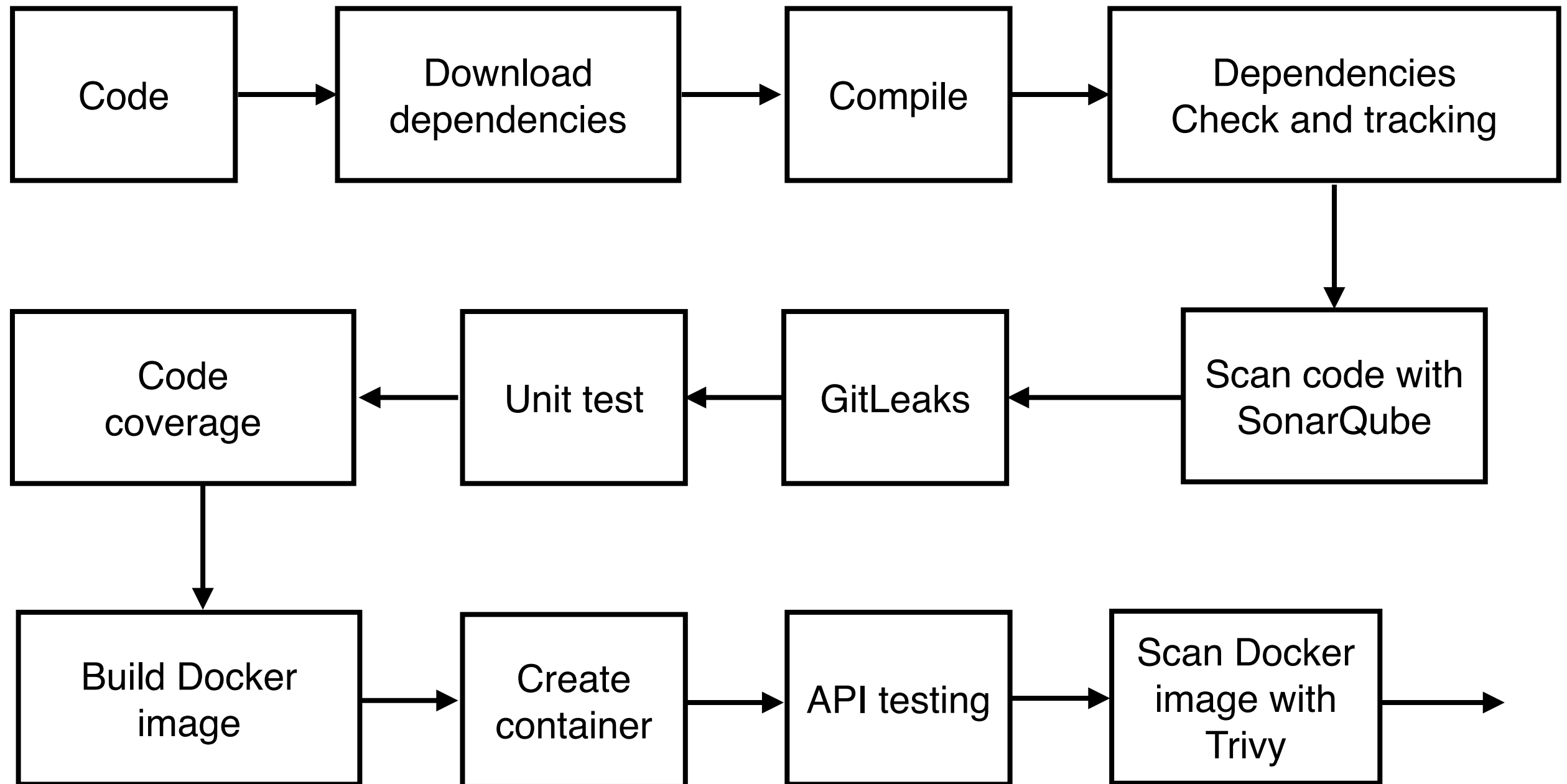
All about feedback loop



DevSecOps Pipeline



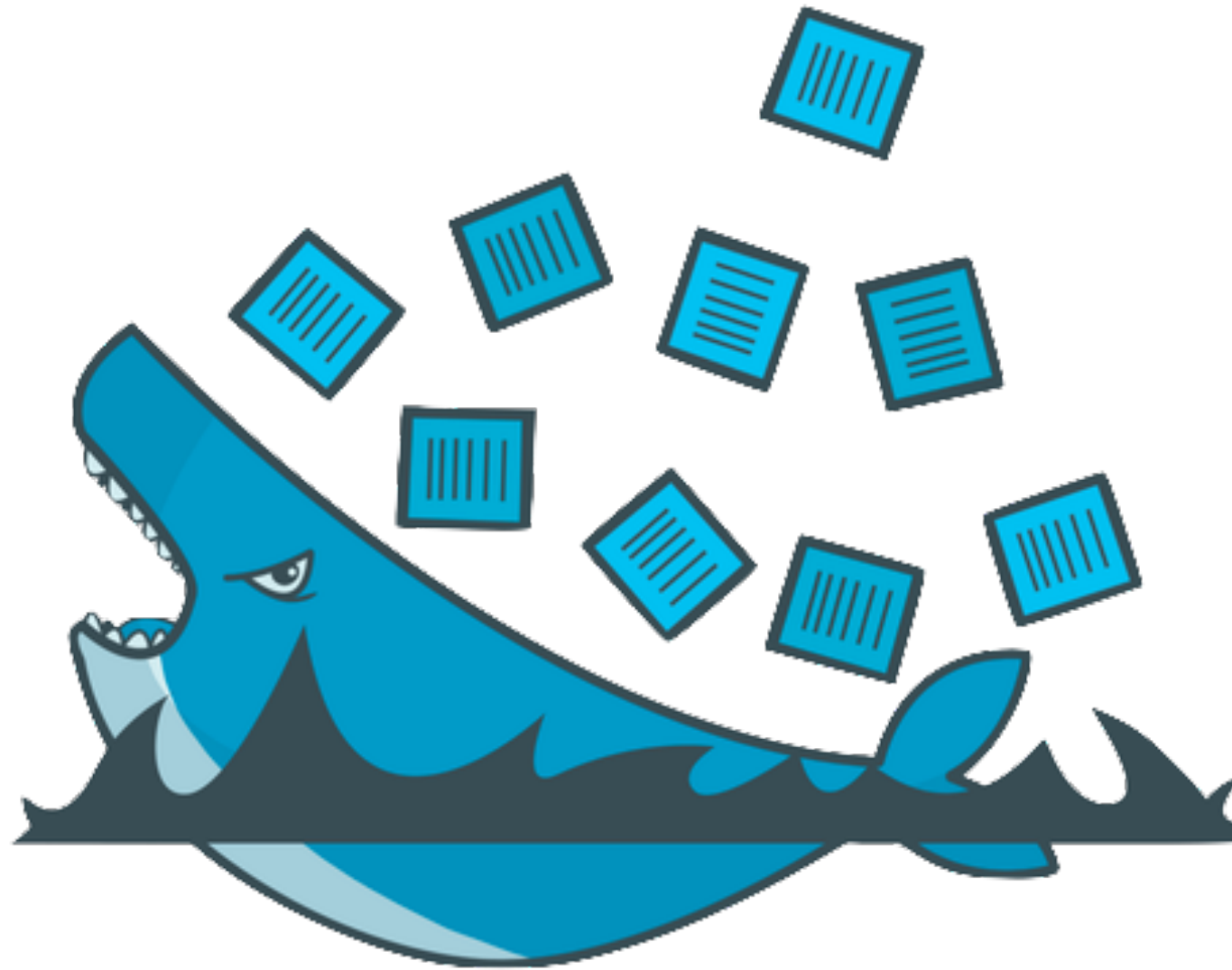
Design your pipeline for API



Working with Kubernetes

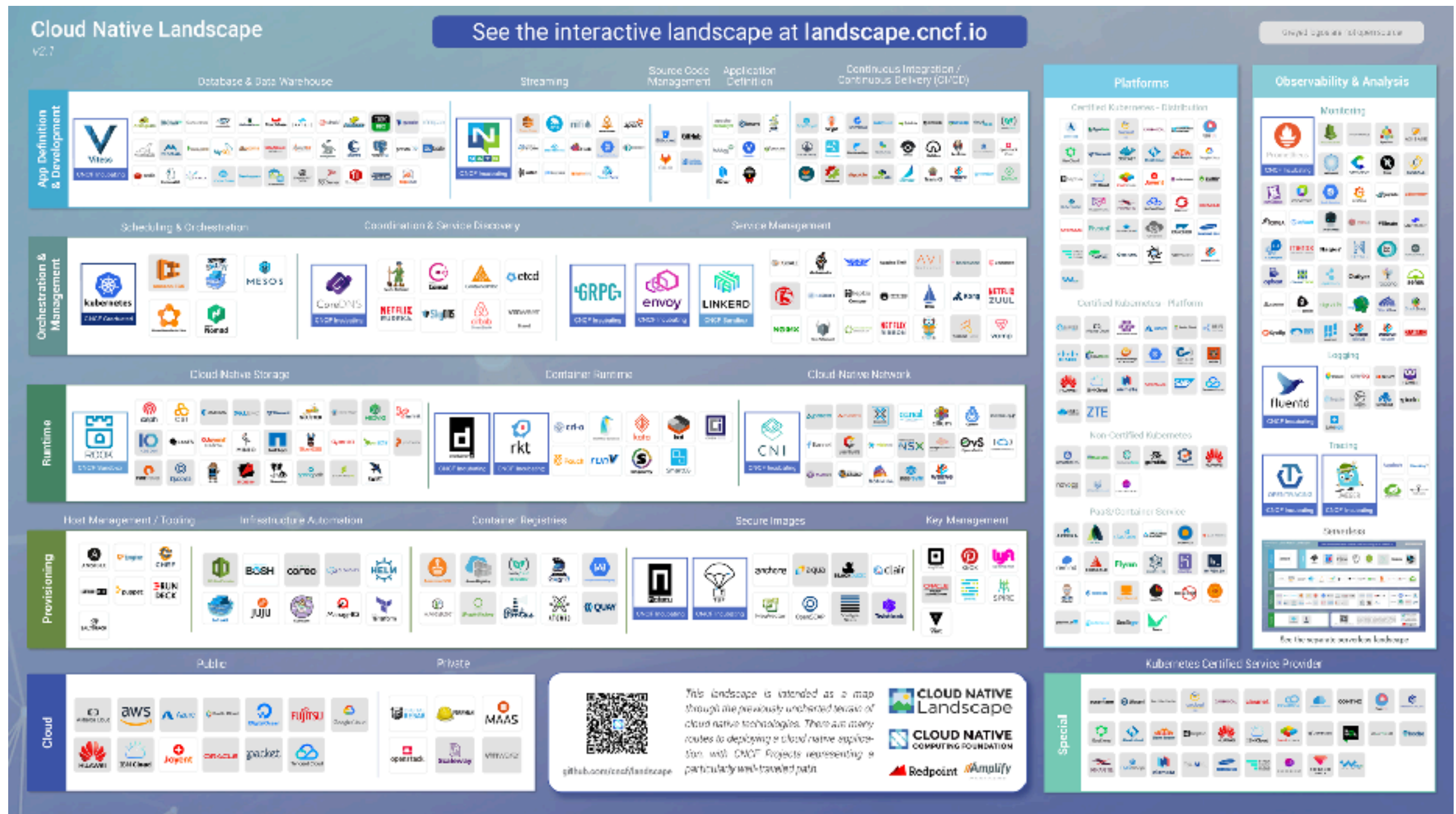


Orchestration Containers ?





Cloud Native Landscape



<https://github.com/cncf/landscape>





kubernetes

<https://kubernetes.io/>



Why we need Kubernetes ?

Managing containers for production is challenging
Need something to manage beyond a container
engine !!



Key capabilities was missing

Using multiple containers with shared resources

Monitoring running containers

Handling dead containers

Moving containers so utilization improves



Key capabilities was missing

Autoscaling container instances to handle load

Making the container services easily accessible

Handling dead Connecting containers to a variety of external data sources

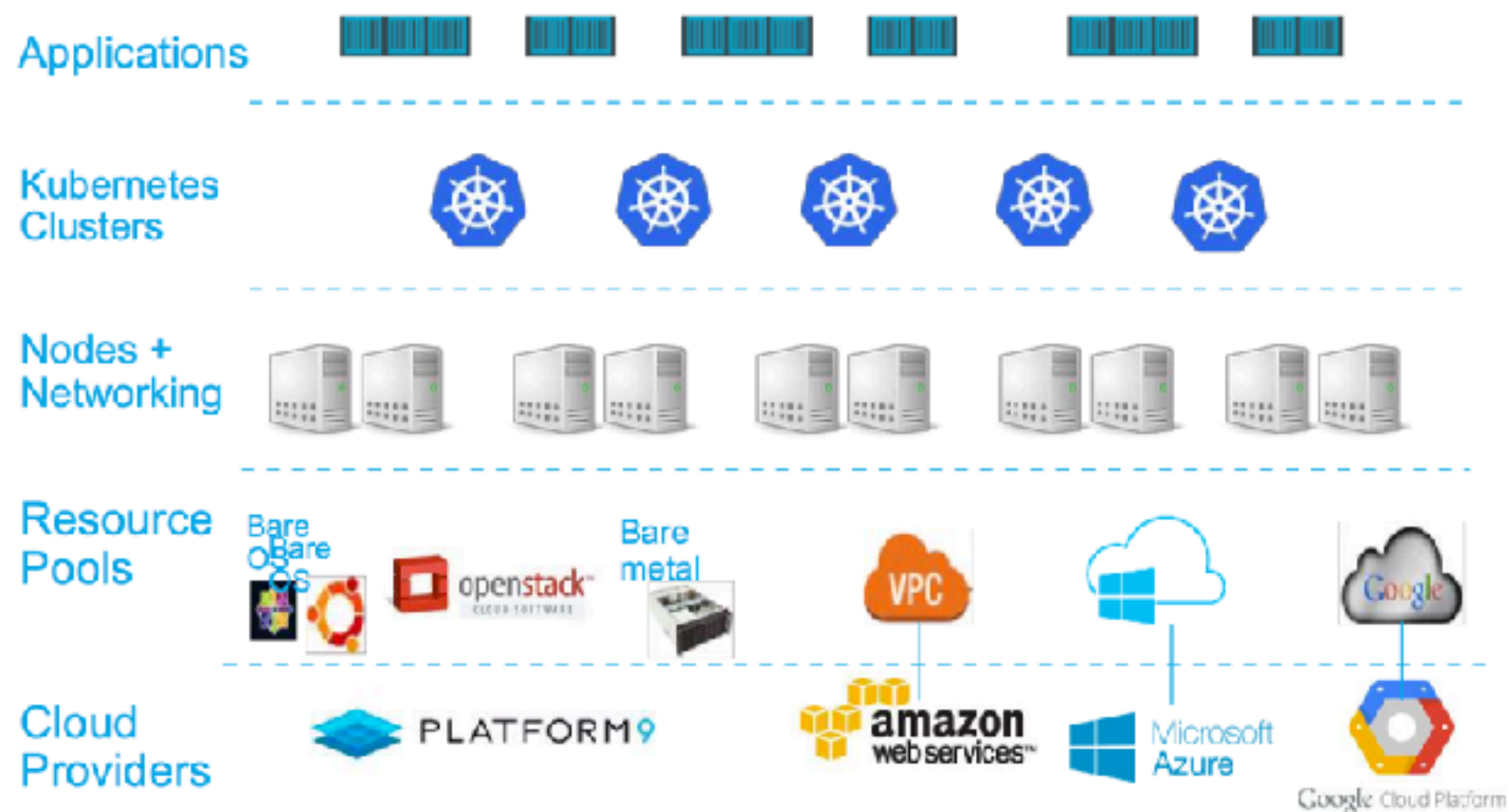


Write once, run anywhere

Eliminate infrastructure lock-in

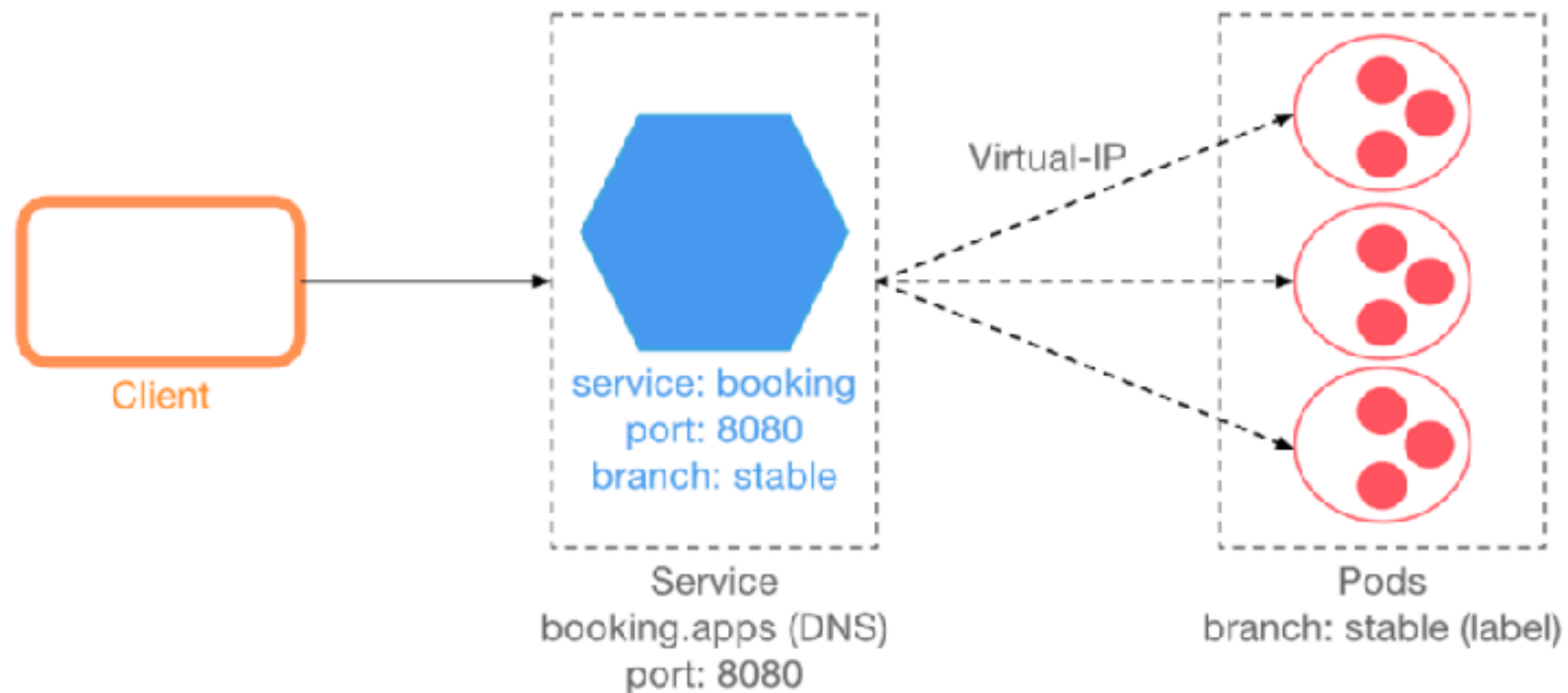
Use containers

Provides management for containers



Modular app design

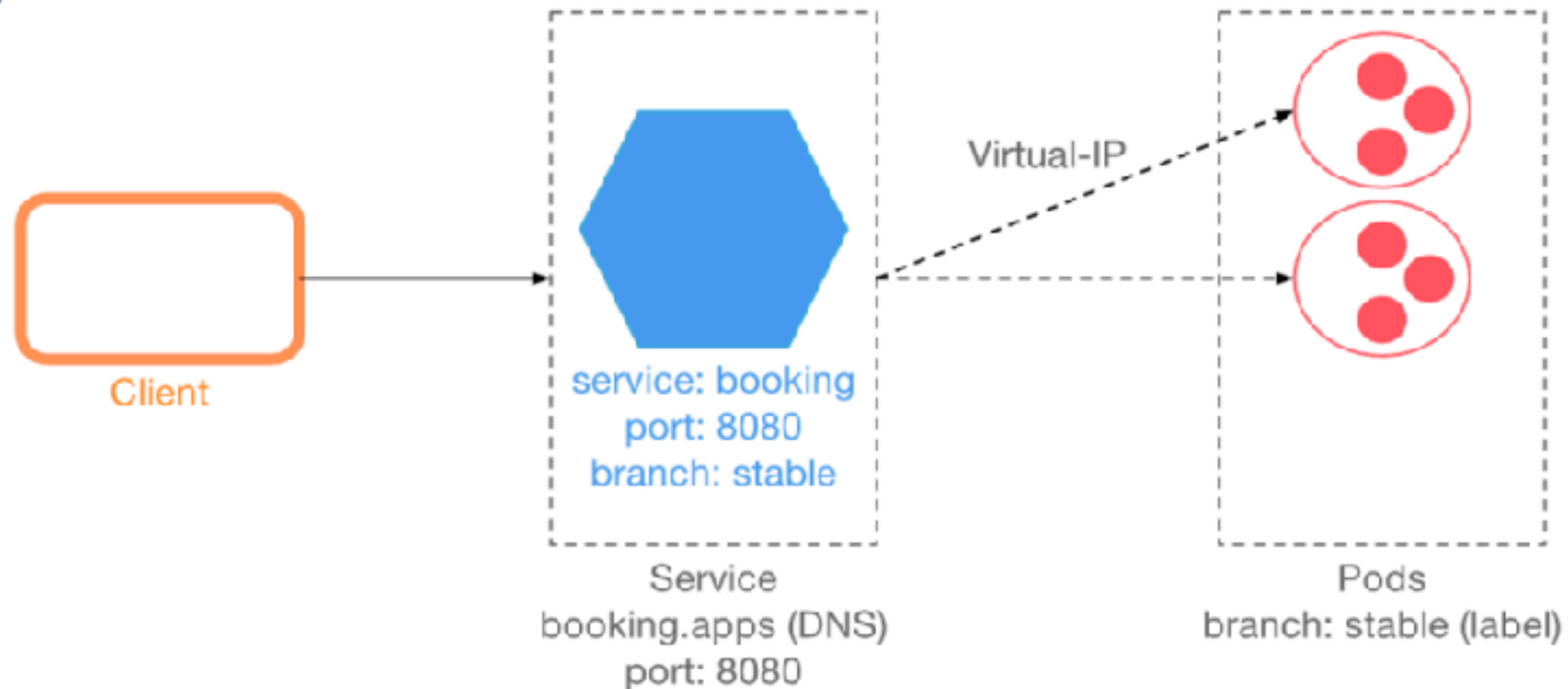
Container/Image boundary similar to class
How to use/manage a collection of container ?



Fault-tolerant by design

Design for failure

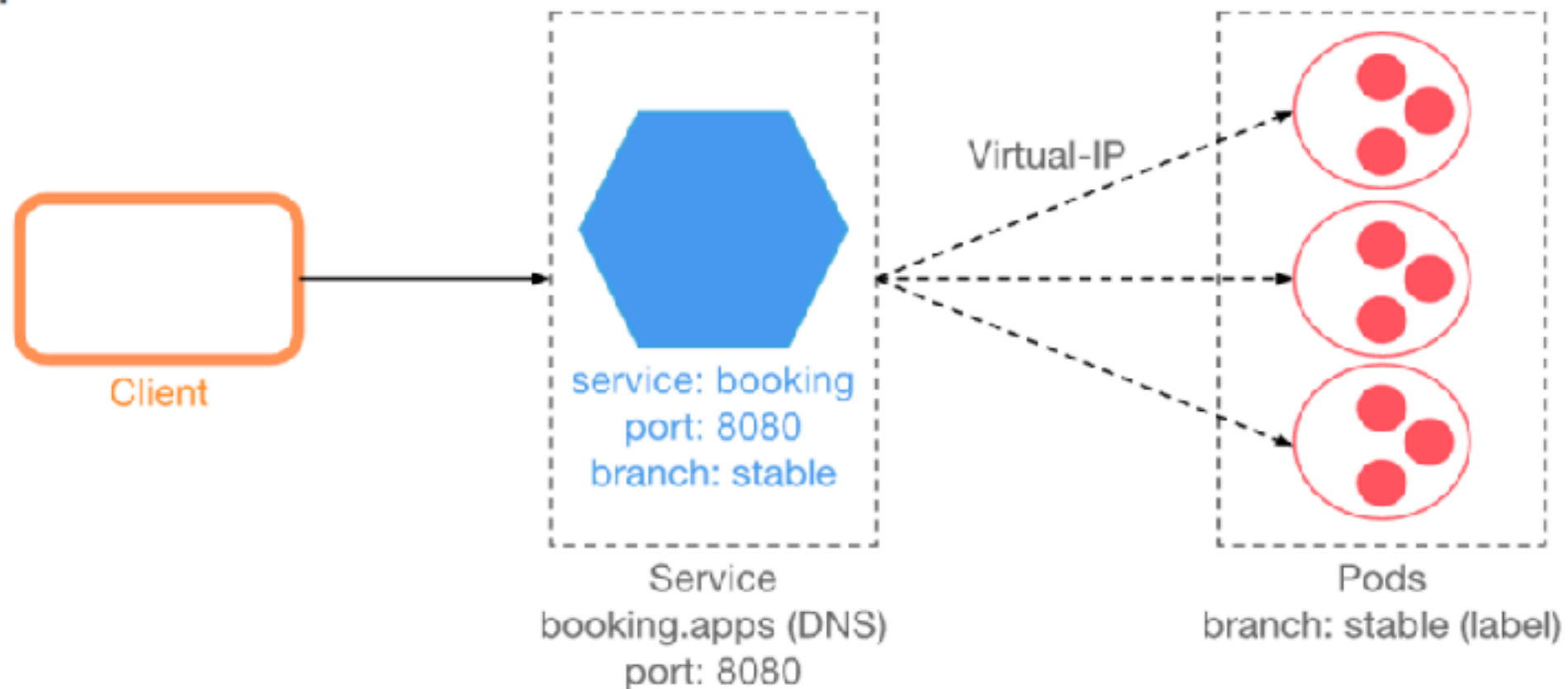
replicas = 2



Fault-tolerant by design

Design for failure

replicas = 3



Deployment, not infrastructure

Software deployment is hard

Infrastructure provisioning/re-provisioning

Configuration networking and load balance

Redundancy (scale-out)

Lifecycle management (Software update)



K8s support for deployment

Scale-out service

Rolling update for new version

Rollback to a previous version

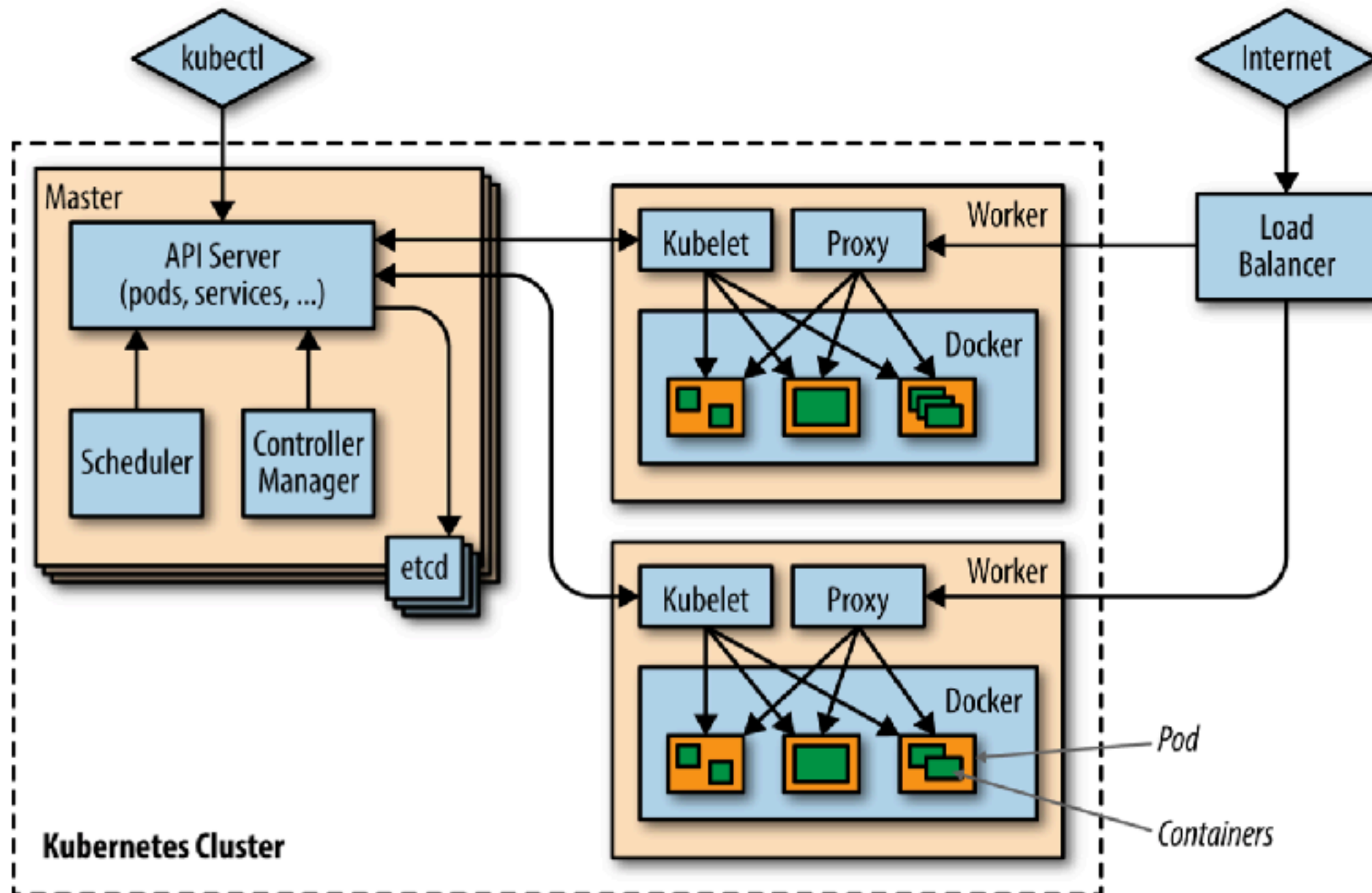
Pause and resume a deployment

Horizontal auto-scaling

Canary deployment



Kubernetes Architecture



<https://kubernetes.io/docs/concepts/architecture/>



K8s components

Pods

Services

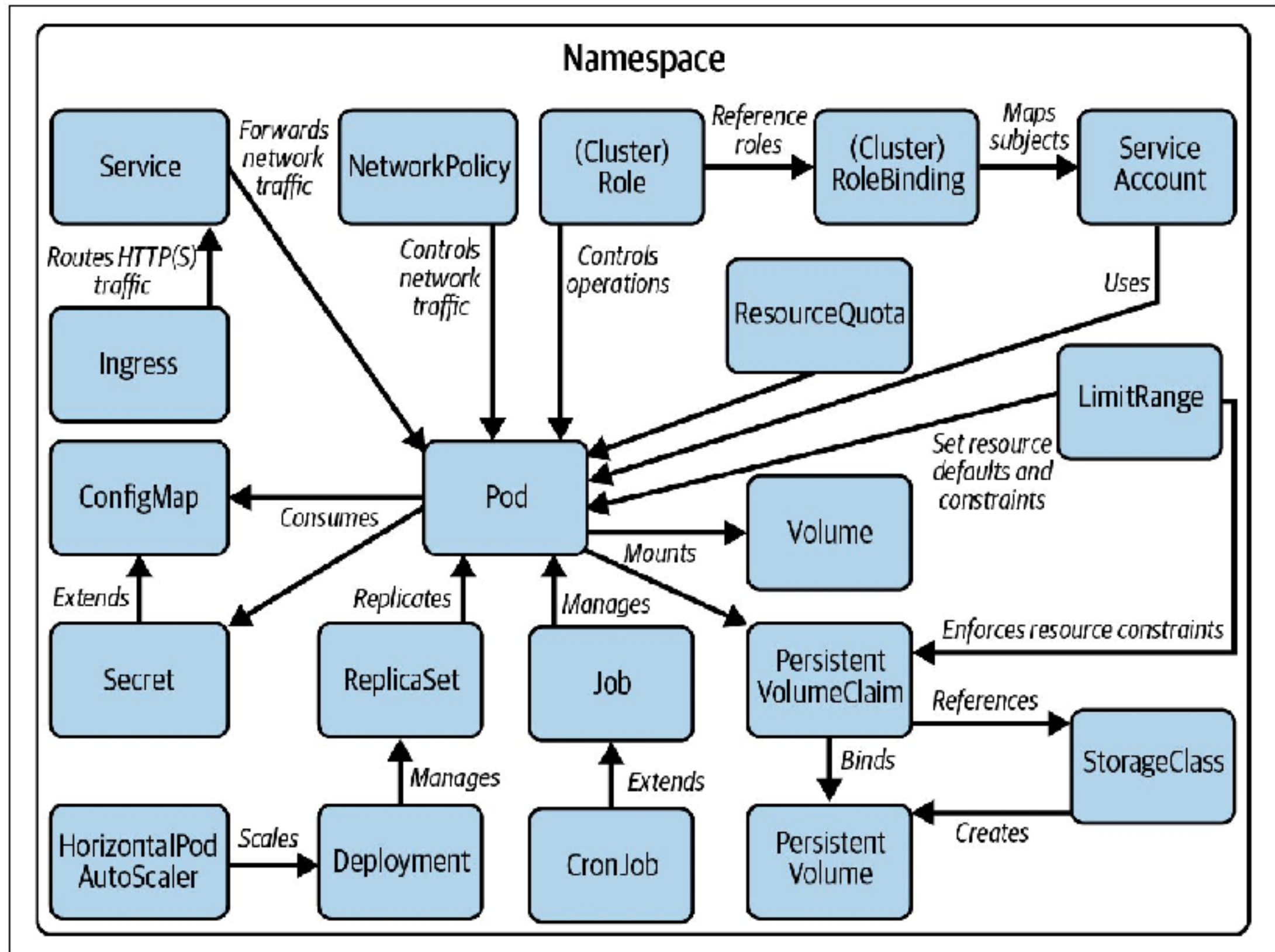
Deployments

ConfigMap and Secret

Volumes

StatefulSets





<https://kubernetes.io/docs/concepts/architecture/>



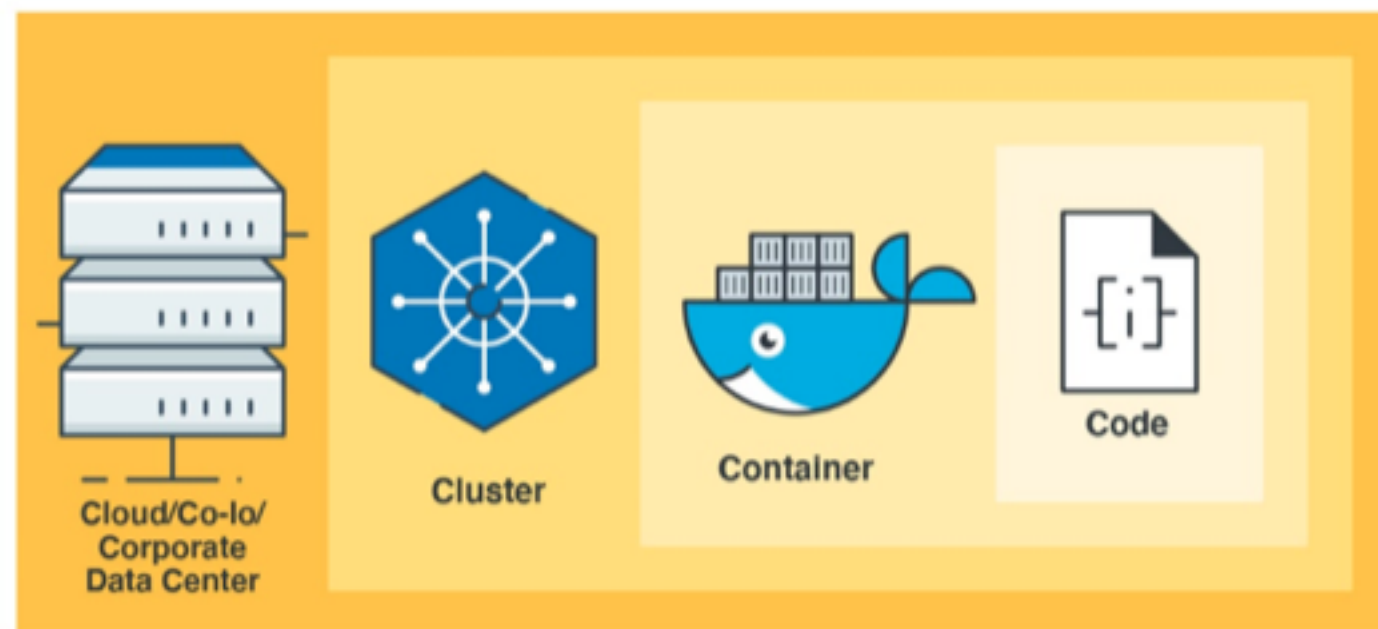
4Cs of Cloud Native Security

Code -> Container -> Cluster -> Cloud

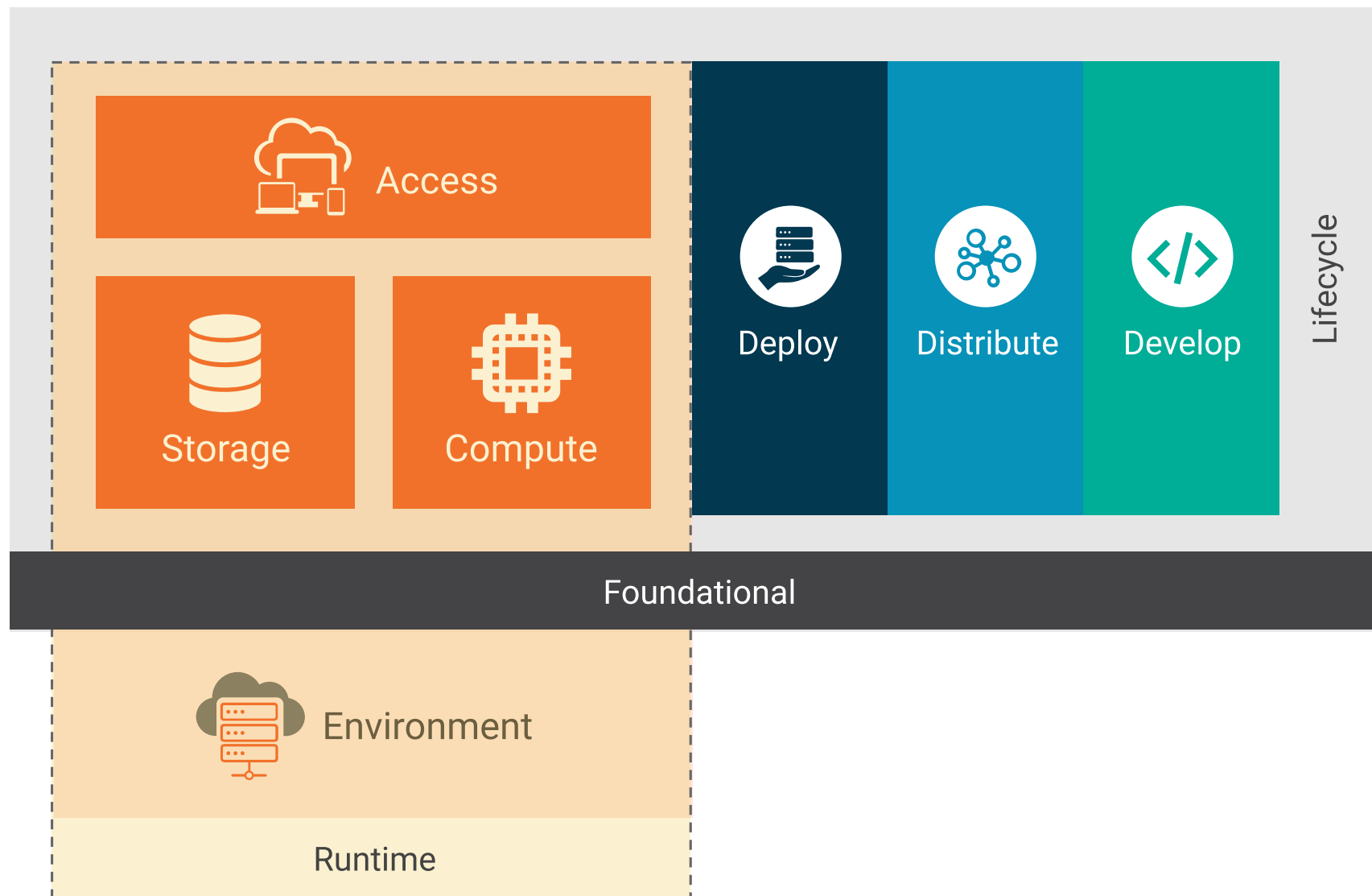
Provide function to maintain versioning of Pods

Update new version (Rollout)

Revert to old version (Rollback)



Kubernetes Security Controls



<https://kubernetes.io/blog/2020/11/18/cloud-native-security-for-your-clusters/>



Kubernetes Security Controls

Pods security policy

Resource requests and limits

Audit log analysis

Control plane authentication and certificate

Secret management



Secret Management in Kubernetes ?

Manage sensitive datas (password, token)

Use **secret** object

Base64 encoding by default (not encrypt)

No automatic rotation

Size limit and namespace limitation



Alternative and best practices

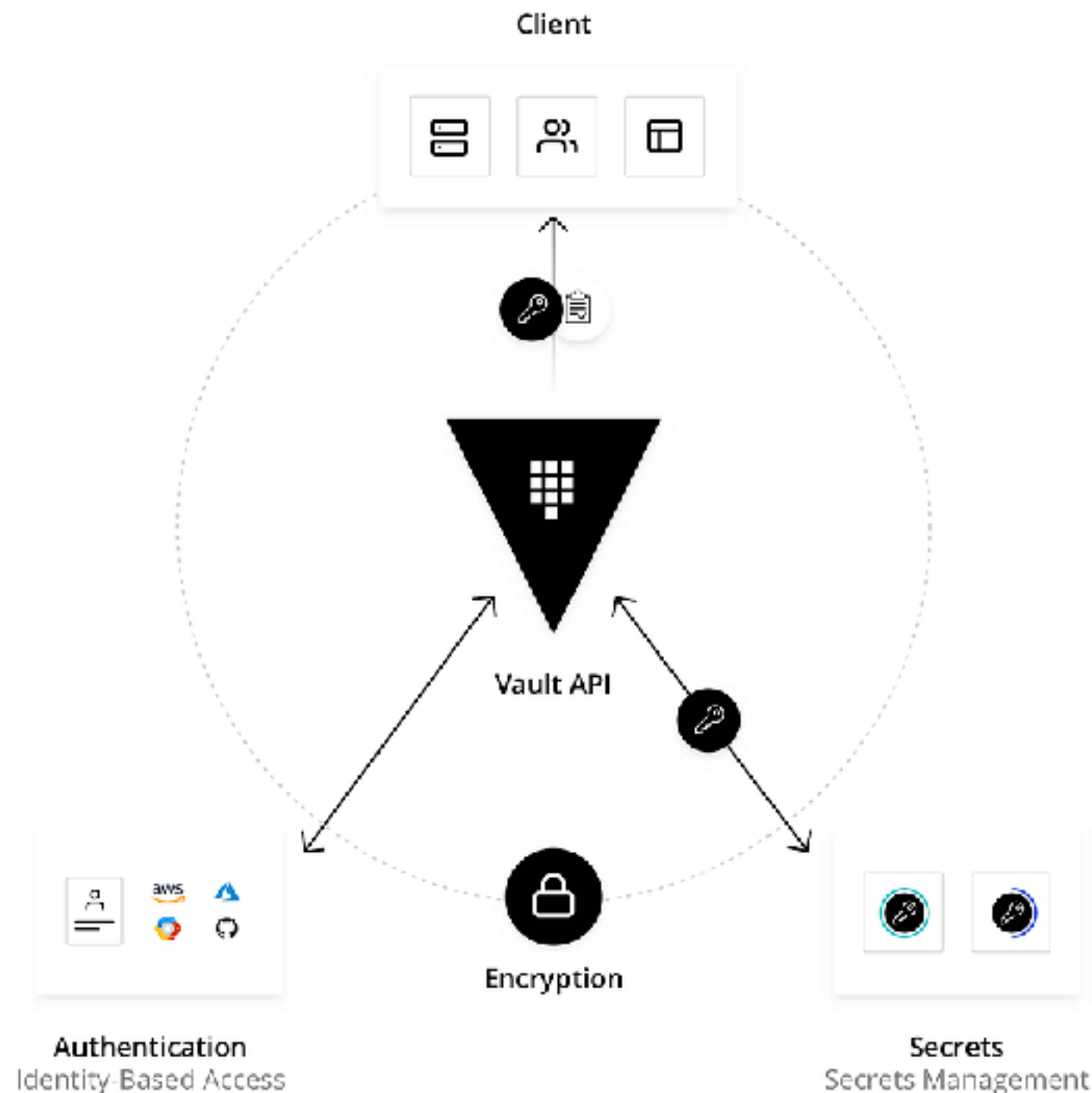
External Secret Operator (ESO)

Secret Store CSI Driver

Hashicorp Vault



Hashicorp Vault



<https://www.hashicorp.com/en/products/vault>



Integrate with Kubernetes

Vault Sidecar Agent Injector

Vault Container Storage Interface (CSI) provider

Vault Secrets Operator

<https://developer.hashicorp.com/vault/docs/deploy/kubernetes>



Vault Sidecar Agent Injector

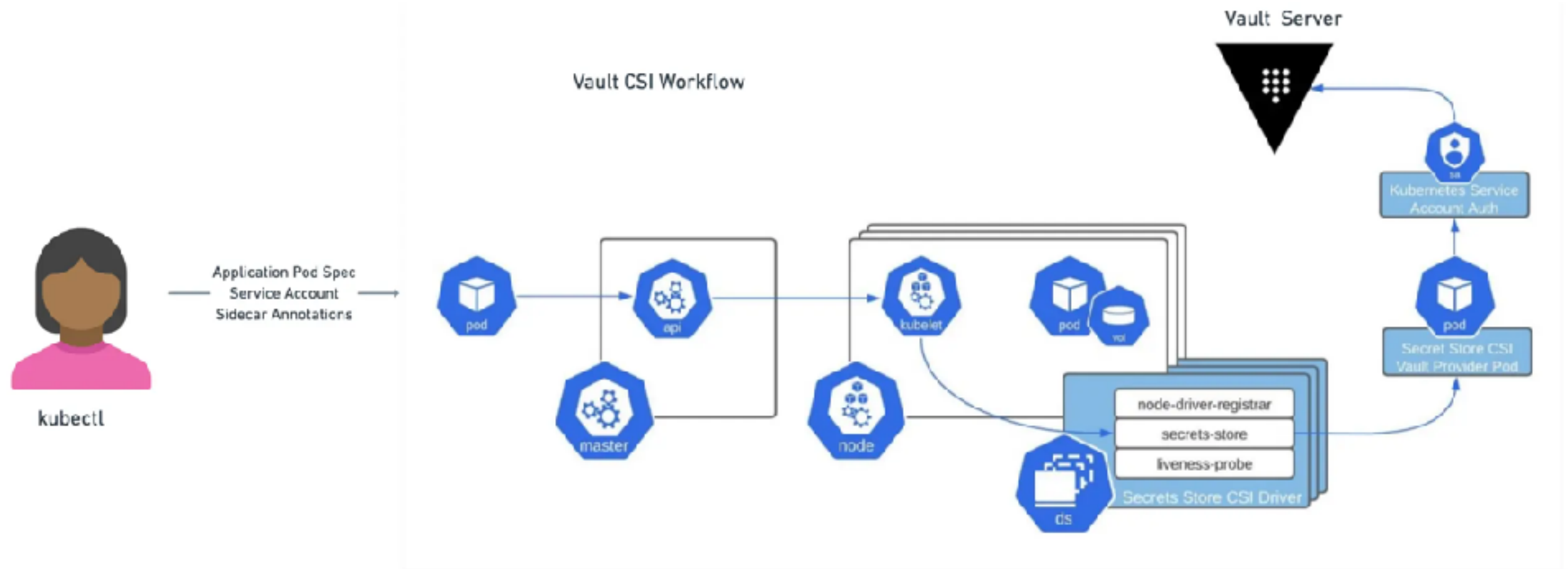
Vault Sidecar Secret Injection Workflow



<https://developer.hashicorp.com/vault/docs/deploy/kubernetes/injector>



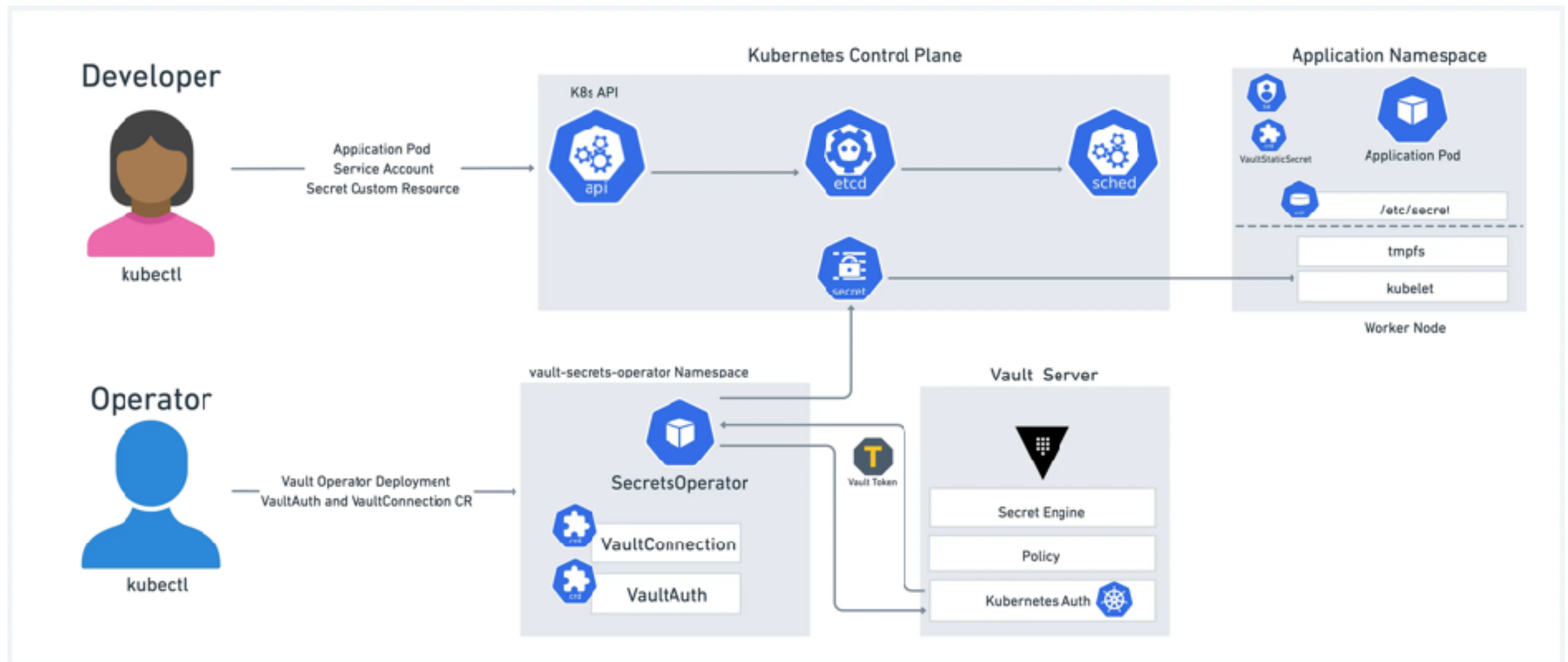
Vault Container Storage Interface (CSI) provider



<https://www.hashicorp.com/en/blog/kubernetes-vault-integration-via-sidecar-agent-injector-vs-csi-provider>



Vault Secrets Operator



<https://github.com/hashicorp/vault-secrets-operator/>

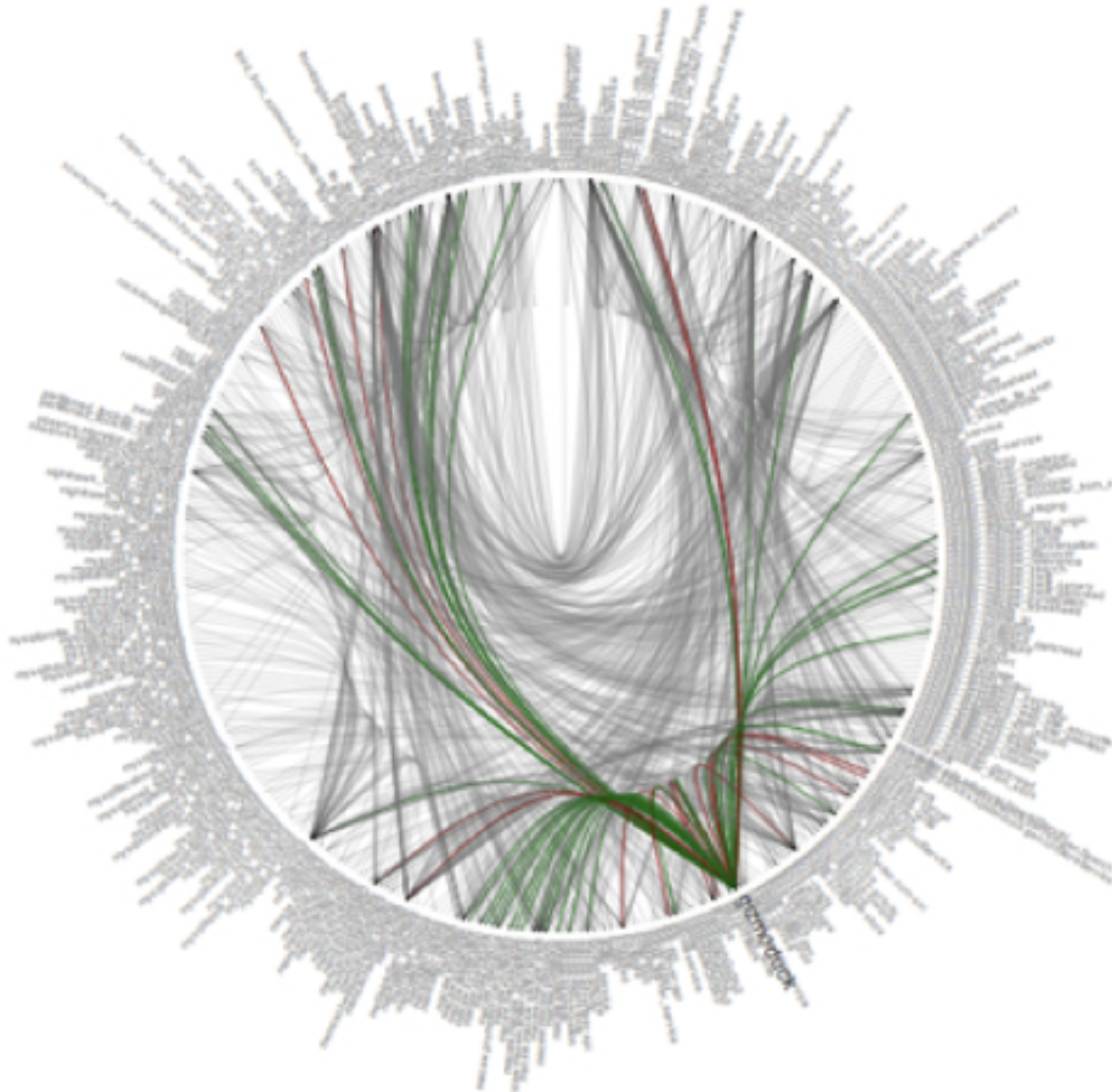


Workshop with Hashicorp Vault

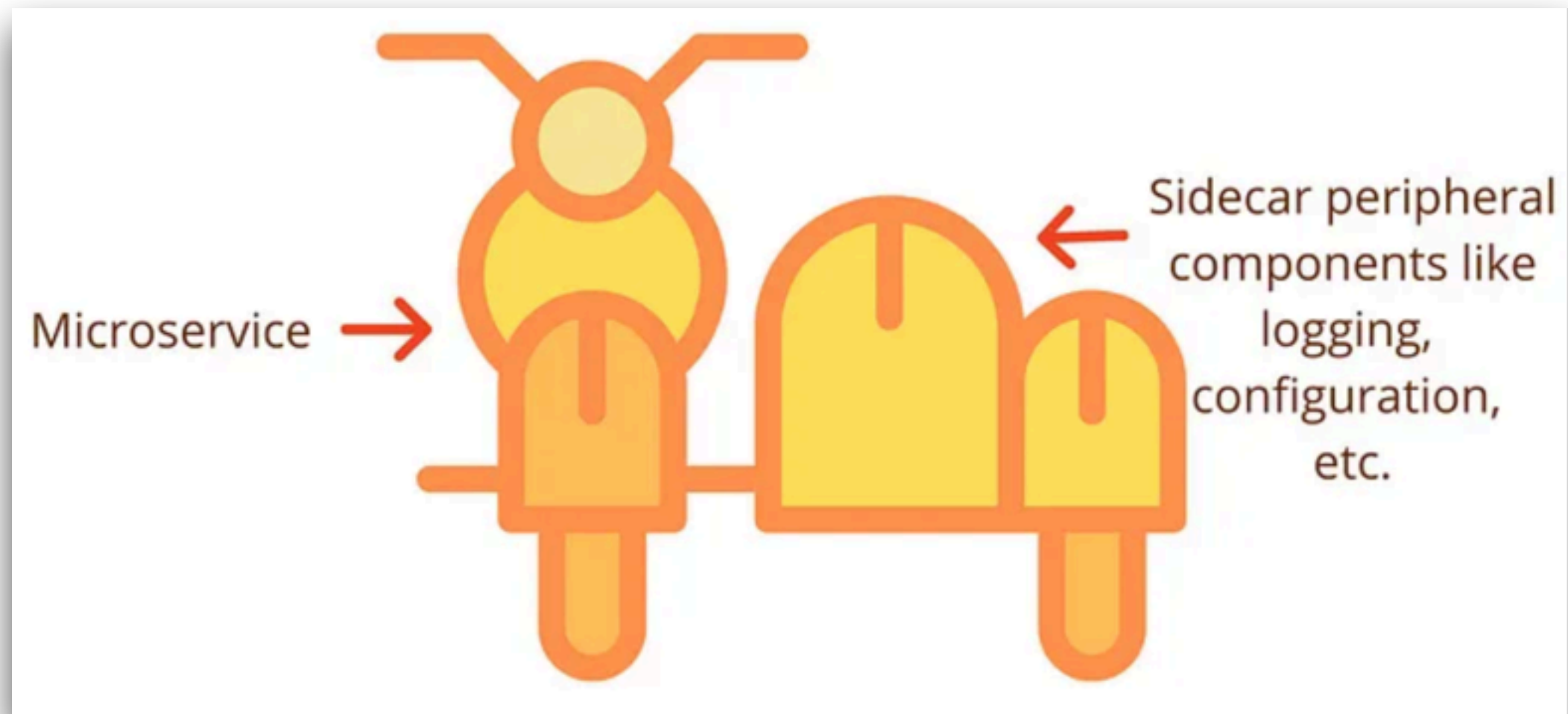


Service Mesh ?





Sidecar Pattern



<https://learn.microsoft.com/en-us/azure/architecture/patterns/sidecar>



Common Use Cases

Service mesh proxy (Istio, Linkerd)

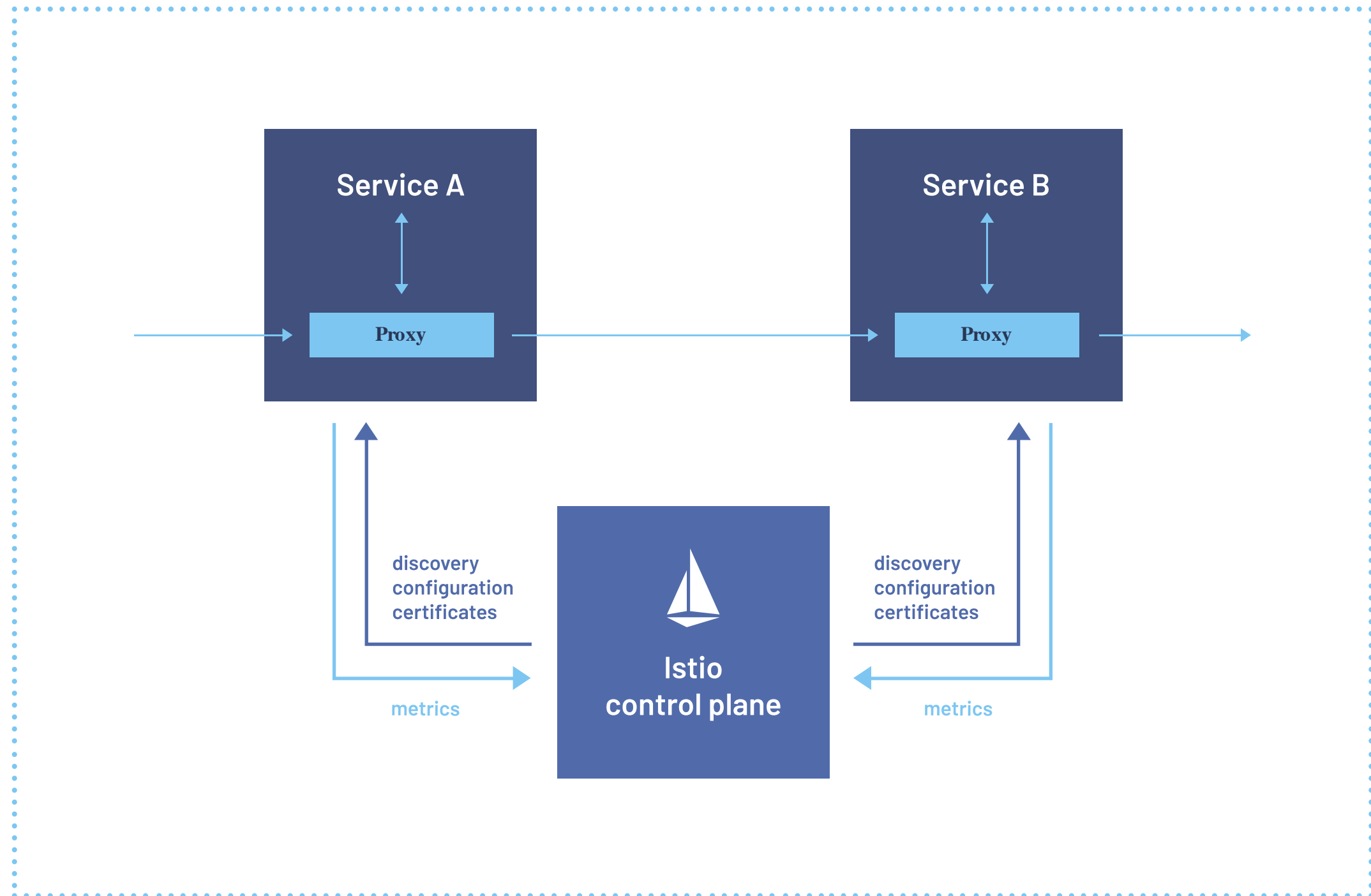
Logging monitoring

Configuration management

Security (authentication and authorization)



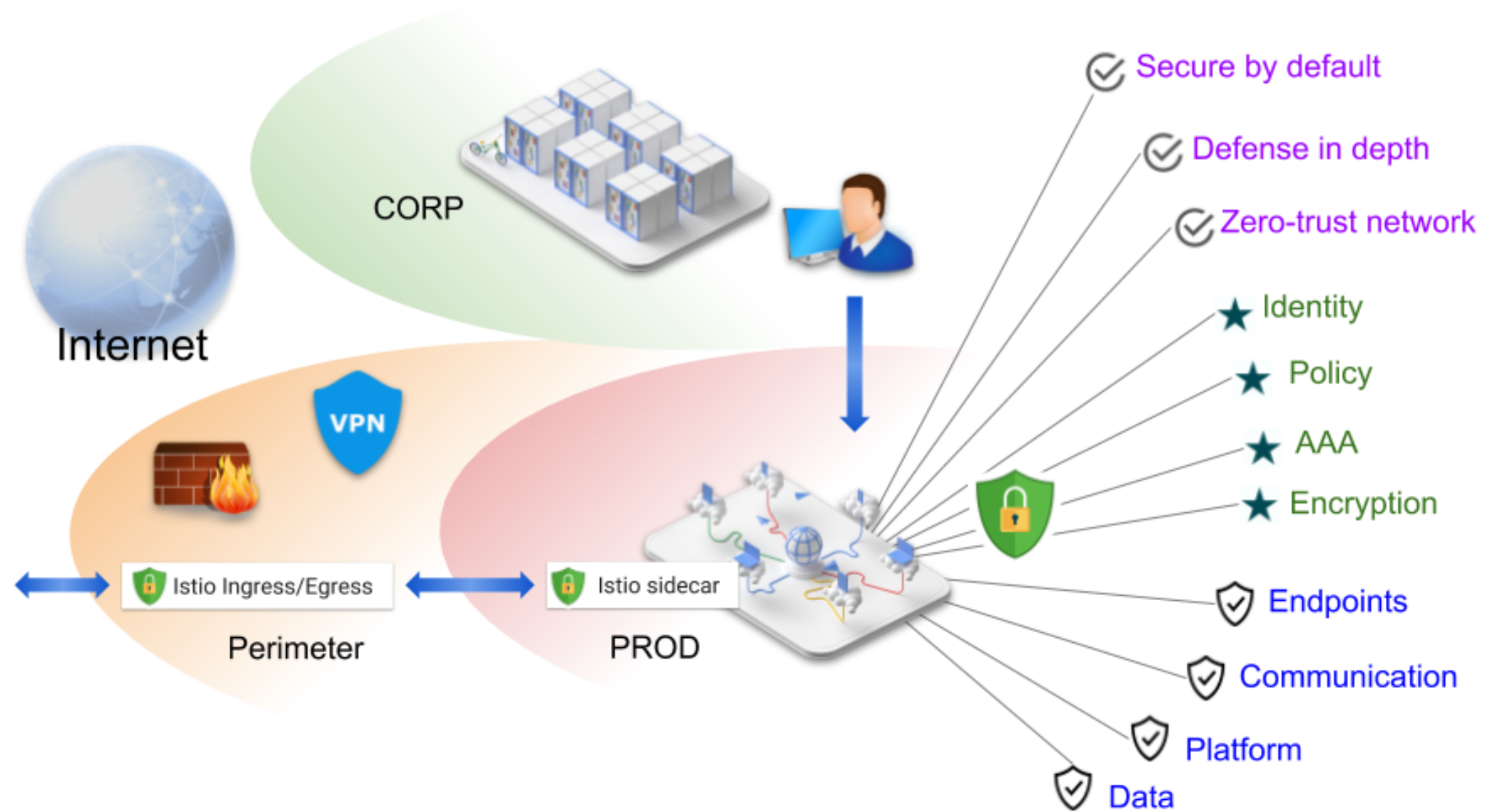
Istio



<https://istio.io/latest/about/service-mesh/>



Istio with security



<https://istio.io/latest/docs/concepts/security/>



Workshop with Istio

Security

Traffic
management



Q/A

